

Supplementary Materials: Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking for Shared Execution Services under Deadline-Driven Bursty Load

Yazhou Guo ^{1,*}, Zexin Lin ², Chengyang Huo ³ and Yurong Song ¹

This file holds supporting material for the manuscript: the visibility audit of the feature pipeline, the predictor families and the predictor scores on the two judging traces and on the four non-judging ones (Section S1); the field audit of the main judging trace, what supports the shared-pool design, two checks on the continuous-integration refit and the cost-concentration screen (Section S2); the experimental configuration, the selected budget points, the budget shapes the main guard table no longer prints, the corrupted scores, the aging comparator, the alternative ranking losses, the identity residual on all nine cells, the single-server configuration and the trace-construction sensitivities (Section S3); and the deferred proofs and remarks of the theory section, the workload comparison and the consistency proposition among them, including the proof of the net-overtake identity (Section S4). Section, table, figure, equation and statement numbers here carry an S prefix; a number without it refers to the main manuscript, and the sentence says so. What the manuscript calls logs are called traces here, and its two grading logs are the two judging traces. In the proofs, a job's rank is its position in arrival order: a job of lower rank arrived earlier. The traces are the ones the manuscript's Table 4 names: the CodeBench judging logs [1], the Azure Functions invocation trace [2], the Intel Netbatch logs [3] and the two Firefox continuous-integration pools collected from Mozilla's public APIs [4,5]. The additional reproducibility specification is Section S5; capacity, physical queueing, feedback and LPC-EGEE results are Section S6. Additional theory is Section S7; the extremal constructions and remaining proofs behind Section 6 of the manuscript, which carries no appendix of its own, are Section S8; and the per-paper detail behind Section 2 of the manuscript is Section S9; the tables of the sealed run are Section S10; and the main comparisons replayed under the online protocol are Section S11. Outside Section S10, every empirical figure quoted here is a development figure in the sense of Section 7.3 of the manuscript: nothing in those sections comes from a sealed part of any trace, and nothing there was recomputed for it.

S1. Cost Prediction

S1.1. The Visibility Audit in Full

Section 4.2 of the manuscript states that every historical feature is recomputed by streaming the log in availability order and reading each entity's state as of the query instant, and that 0 rows disagree in any of the eight configurations. This is what that test is.

On 3,000 randomly drawn development jobs all 34 history and relational features are recomputed from scratch under each configuration's own visibility rule and compared against the pipeline's values. The same test applied to the natural but wrong construction, in which a feature may use any record with a smaller index, fails on 97.67% of rows under the primary rule, so the test does discriminate. The graph pipeline is checked separately: an index assertion over 863,149 rows finds 0 timeline entries pointing at or after the read, brute-force recomputation on 600 rows matches on all 2,400 root states and all 2,400 neighbour sets, and perturbing or deleting every record after a cut time leaves all earlier predictions unchanged while a positive control moves the later ones.

S1.2. Predictors on the Non-Judging Traces

Table S1 is the cross-trace counterpart of Table S2, and Section 4.3 of the manuscript points here for the four logs from outside education. The AUROC column of Table 9 of the manuscript is the ALL row of each trace.

A task on the first judging platform is attempted by one cohort within a few days, so who is writing still carries information. A task on the second is attempted by thousands of users over years, so the task's own record has already fixed the cost. The non-judging traces push this further. On the serverless trace the identity of the function explains 0.867 of the variance of $\log(1 + C)$ on its own, and static and calendar features alone are close to useless for ranking (Spearman 0.021). The two CI pools sit at opposite ends: entity history gives AUROC 0.9968 on one and 0.6334 on the other, because that pool's job labels do not separate its costs. The compute-farm trace is predictable in rank and not in level.

Table S1. The same protocol on the non-judging traces, by feature group. ENT uses the entity's own visible history, STATIC uses only what is declared before the job runs, ALL is both. RMSE and Spearman are on $\log(1 + C)$.

Trace	Features	AUROC	AUPRC	RMSE	Spearman
Serverless invocations	ENT	0.9828	0.816	0.3401	0.953
	STATIC	0.6238	0.084	0.8088	0.021
	ALL	0.9847	0.830	0.3185	0.932
CI pool, hardware A	ENT	0.6334	0.157	0.3964	0.833
	STATIC	0.4990	0.045	0.7659	0.277
	ALL	0.6512	0.177	0.3956	0.837
CI pool, hardware B	ENT	0.9968	0.984	0.0887	0.985
	STATIC	0.6915	0.077	0.6411	0.500
	ALL	0.9970	0.985	0.0860	0.988
Compute farm	ENT	0.8903	0.265	1.4214	0.883
	STATIC	0.8704	0.242	2.2559	0.750
	ALL	0.8902	0.260	1.3485	0.892

S1.3. The Release Delay as a Sensitivity

The delay d with which results reach the feature store (the rule of Section 4.2 of the manuscript) is treated as a sensitivity and not as an assumption. At $d = 0$ the full tabular model reaches AUROC 0.9283 and at $d = 600$ s it reaches 0.9066. On the second judging platform, where the log carries no clock and lag has to be expressed in records, the same model moves from 0.9266 at lag 0 to 0.8902 at lag 1,000. Ranking survives a ten-minute feedback delay at the top of the curve; retrieval of the small heavy class degrades, average precision falling from 0.289 to 0.139.

S1.4. Predictor Families

Three model families are compared under one visibility protocol and one split: gradient-boosted trees, a sequence model, and a relational ladder. The negative relational result of Section 4.3 of the manuscript is read off the last of them.

The tabular family is gradient-boosted trees [6] over five feature groups: task history (M1), user history (M2), static features of the submitted artefact (M3), everything tabular (M4), and M4 plus one-hop relational aggregates over the task-assignment and user-cohort neighbourhoods (M5). A control (M6) permutes task identity across assignments, so that M5's aggregates are keyed on a scrambled neighbourhood; it reaches AUROC 0.9288 in the primary configuration against M5's 0.9302, so the aggregates lose almost nothing when the identity they are built on is destroyed. The sequence family is a GRU over the earlier jobs of one user whose results are already available, used alone (R1) and as a state vector appended to M4's columns inside the boosted model (R1S). The relational family is a heterogeneous network over user, task, assignment and cohort nodes [7] with

no node-identity embeddings, so an unseen task or user is handled by the same weights. Its ablation ladder runs N0 (a multilayer perceptron on exactly M4's columns), G0U (the relational architecture with every neighbour slot empty and the assignment and cohort roots removed, which leaves exactly M4's information), G0 (empty neighbour slots, roots kept), G1 (the real edges) and G2 (the same network on edges permuted within a period and time block, so that neighbour counts and recency are preserved and only the identity of the neighbour changes).

S1.5. Predictors on the Two Judging Traces

Table S2 scores every predictor family on the two judging traces, and is what Section 4.3 of the manuscript quotes from.

Table S2. Predictors on the two judging traces under one visibility protocol and one split. Heavy is the training-period 95th percentile of executed cost. RMSE and Spearman (Sp.) are on $\log(1 + C)$. R-net is the heterogeneous relational network.

Model	Information	AUROC	AUPRC	RMSE	Sp.
<i>CodeBench</i> , $n = 40,844$					
M1	task history	0.7960	0.053	0.3000	0.219
M2	user history	0.8591	0.242	0.2605	0.302
M3	static features	0.8632	0.082	0.2892	0.290
M4	all tabular	0.9283	0.289	0.2519	0.312
M5	M4 + one-hop	0.9302	0.299	0.2507	0.344
N0	MLP on M4's columns	0.9209	0.306	0.2463	0.278
G0U	R-net, no edges or roots	0.9348	0.322	0.2417	0.321
G0	R-net, no edges	0.9325	0.316	0.2402	0.324
G1	R-net, real edges	0.9324	0.322	0.2397	0.329
G2	R-net, permuted edges	0.9305	0.318	0.2396	0.323
R1	GRU over user history	0.9239	0.324	0.2396	0.313
R1S	M4 columns + GRU state	0.9366	0.315	0.2414	0.329
<i>ACcoding</i> , $n = 647,465$; the neural models were not fitted here					
M1	task history	0.8990	0.431	0.2904	0.683
M2	user history	0.7855	0.292	0.3190	0.471
M3	static features	0.7884	0.165	0.3429	0.493
M4	all tabular	0.9266	0.527	0.2714	0.692
M5	M4 + one-hop	0.9264	0.527	0.2713	0.696

S1.6. The Cost of Predicting

Prediction has to be cheap enough to sit in front of a queue whose median job is 204 ms, which is what makes the choice between these predictors a queueing decision and not a latency one. Measured in a streaming loop over 20,000 jobs, the tabular path costs a median of 0.076 ms per job and a 99th percentile of 0.188 ms; the relational network costs 0.434 ms per row plus 0.121 ms of graph lookup, and the GRU 0.576 ms. Charging the measured feature and inference latency to the predictive policies before they enqueue changes the closed gap by less than 0.001 at every load we ran, so the choice between these predictors is not a latency decision at this job size.

S2. Traces and Workload Characterisation

S2.1. The Field Audit of the Main Judging Trace

Section 7.1 of the manuscript summarises how the main trace is read. This is the evidence for each reading, and the two archive defects that follow from reading the trace this way.

The timestamp is a result stamp.

Among later-period runs the gap to the same user's previous event grows one for one with the cost (slope 0.99 of the median gap on C) while the gap to the next event barely moves (slope 0.16), and the feedback after a user kills a run arrives within about 0.1 s whatever its length. A result stamp predicts both readings; a receipt stamp neither.

The whole-second leak.

Stamps are recorded to the whole second while costs are recorded to the microsecond, so the fractional part of a job's cost appears in the gap between its own stamp and the next event, and a feature like "seconds since the user's previous record" reads the label off the clock. We close it by replacing t with $t + u$, where $u \in [0, 1)$ is drawn deterministically from a fixed key and the record's identifier.

The cost field and the limit.

The cost field is wall-clock occupancy of one evaluation including interpreter start-up, read from the 10th percentile of correct submissions (0.12–0.18 s across the two periods) and from its weak dependence on the number of test cases (median $0.196 + 0.007n$ s in the later period). The platform limit is 60 s: the last run to reach it ends at 2019-08-28 10:32, after which the largest observed cost is 59.97 s, with 13 runs in $[59, 60)$ s and none above. Before that date there was no fixed limit, the longest run is 8,901 s, and the 1,189 runs that end with a kill are almost all above 360.8 s, pointing at a container watchdog rather than judging demand.

Co-ending groups, zero-cost blocks and two short archives.

Groups of runs by the same user that end in the same second are 7.53% of blocks and 26.28% of capped work; every prediction table in the manuscript is repeated on rows outside such groups. A further 1,671 blocks record a cost of exactly zero with empty outputs; they leave the simulated trace and stay in the features, the labels and every total. The 2019-2 archive stops at 2019-09-19, months before that term's assessments end, and 2020-1 is short by about 3% of its blocks; both serve per-submission modelling and are excluded from anything that treats a term as a whole.

What the recorded costs mean.

Part of the recorded duration is contention, which we then re-impose inside the pool: between 38% and 79% of jobs longer than a second end in the same second as another block of that student. The observed 60-second regime is not documentation of an enforced timeout, and how the platform enforces such a limit is undocumented in the sources we found. We wrote to the maintainers about both points and had no answer. ACcoding [8] records no submission timestamps, so its arrival process is synthetic.

Against the published release statistics.

We match the class count in 18 of 18 terms and the student and exercise counts in 16 of 18, and parse 4.08% fewer execution blocks than the official column sums, whose printed totals disagree with those same columns by -248 exercises and $+246,246$ blocks. We quote the columns.

S2.2. The Continuous-Integration Refit: Sharpness and Circularity

Section 7.2 of the manuscript reports the held-out refit of the effective capacity k_{eff} and the setup term on the two Firefox CI pools. Two checks on that fit sit here.

The objective is sharp in k rather than flat: on a step-of-one grid only 5 to 8 values of k put the simulated mean wait within 10% of the recorded one, so the capacity the fit chooses is not one of many equally good answers.

The circularity of the original whole-window fit can be read off the two pools directly. On pool A the whole-window and held-out rows are almost identical, because every split picks nearly the same capacity; on pool B the reverse split's held-out capacity gives a mean ratio of 1.14 on its test part where the whole-window capacity gives 0.99. The difference between those two numbers is the size of the circularity that fitting on the reported window would have hidden.

The split leaves two things uncovered. Out of sample the 90th percentile lands up to 21% from the recorded one and is systematically low on the larger pool. The split is chronological inside one 21-day and one 7-day window of two pools, not a different cluster, quarter or workload, and the capacity is held constant over the test part although the real machine count varies.

S2.3. What Supports the Shared-Pool Design

Section 7.2 of the manuscript names three things in support of the shared-pool design and gives each of them one sentence. The evidence behind all three is here.

Dedicated per-entity capacity is known to sit idle in the workloads this one resembles, and two measurements outside our own traces point the same way. In the serverless characterisation behind our invocation trace, 45% of applications are invoked once an hour or less and 81% once a minute or less, and the authors report that keeping them warm can cost far more than their billable execution time [9]. A cluster manager measured the opposite design directly: splitting each large user's workload into its own cell would need 2 to 16 times as many cells and 20 to 150% more machines [10]. Neither source is about judging, and neither licenses a number of our own.

The second is a measurement on the same demand. Replaying the overlay of Section 7.2 of the manuscript under the dedicated design, the 99th percentile of containers alive at once is 1,636 and each container executes code for 0.12% of the time it is alive; since only 11% to 35% of logins have a logout written after them, session boundaries are reconstructed, and across six reconstructions that percentile lies between 1,003 and 2,525 containers and the utilisation between 0.07% and 0.17%. A shared pool holds the 99th percentile of deadline-window waits under 1 s with 14 servers under FCFS and 13 under the predictor-ordered policy. Pooling is worth two orders of magnitude here and the scheduler is worth one server (14 to 13 at 1 s, 11 to 10 at 5 s, 8 to 7 at 30 s). It does not support a claim that the platform ever queued, the pooled side being simulated, nor that the containers are over-provisioned, since they also run the editing session the pool does not replace, nor that any of it is a cost saving.

The third is that the simulator reproduces waits somebody else recorded. The CI logs carry the queue wait of every run, so we replay the recorded arrivals and services through our own k -server non-preemptive kernel. Two quantities are fitted: the effective capacity k_{eff} , the shortfall against the listed machine count being machines that are up but not claiming work, and a setup term, the median short gap between consecutive runs on the same machine, added to every service because a machine is not reusable the instant a run resolves. Fitting both on the window and then reporting agreement on that same window would be circular, so we refit both on one chronological part of each window and test on the other, five splits over the two pools. This split is internal to the recorded window and is held out of the fit only; it is not sealed in the sense of Section 7.3 of the manuscript, and the whole window has been read.

The capacity is stable across splits. On pool A the three splits choose 155, 155 and 157 of 173 listed machines, and on pool B the two splits choose 96 and 92 of 105; the setup term moves between 54.2 and 55.3 s on the first and between 126.2 and 128.6 s on the second. The objective is sharp in k . With those parameters frozen, the held-out part comes back at 0.90 to 1.14 times the recorded mean wait, 0.86 to 1.16 at the 99th percentile and 0.89 to 1.28 at the 99.9th, with a per-hour mean-wait correlation of 0.936 to 0.981, a per-job rank correlation of 0.940 to 0.963, and a median per-job absolute error of 16 to 84 s.

The 90th percentile does not survive the held-out test. It lands at 0.81 to 1.21 of the recorded one, and on the larger pool it is below the recorded value on all three splits while the 99th is above it, so the simulated distribution is too steep in its upper middle; the two-term objective hides this in sample by trading the mean against the p90, so we fit out of sample and say so. What the fit absorbs is nonetheless a property of the pool and no artefact of fitting: told instead that every listed machine is available, the same simulator underestimates the held-out mean wait by a factor of 1.3 to 2.6 and its per-hour correlation falls to 0.75. Section S2.2 gives the step-of-one sharpness check and reads the circularity of the original whole-window fit off the two pools directly.

S2.4. Cost Concentration and the Provenance of the Analysis Bound

The cost-concentration screen is read trace by trace below, with the arrival and service fields each trace supplies and what each analysis truncation removes. Section 1 of the manuscript quotes its headline shares.

Table S3. Share of executed work carried by the longest jobs, and provenance of the analysis bound L . An analysis cap is applied to the replay, not evidence of a platform-enforced timeout.

Trace	Top 1%	Top 5%	Bound and provenance
CodeBench, 60 s regime	0.375	0.432	60 s, observed regime ceiling; analysis cap
CodeBench, no-limit periods, raw	0.941	0.960	none
CodeBench, no-limit periods, capped	0.494	0.658	60 s, imposed analysis truncation
ACcoding, measured cost	0.275	0.628	30.0 s, observed maximum of measured test-cost sum
ACcoding, with the assumed fixed term	0.122	0.290	30.5 s, observed maximum after assumed overhead
Serverless invocations	0.443	0.879	286.8 s, training p99.9 analysis cap
Compute farm	0.372	0.628	493,857 s, training p99.9 analysis cap
CI pool A	0.034	0.107	5,066 s, training p99.9 of run plus setup; clipped
CI pool B	0.052	0.159	4,140 s, training p99.9 of run plus setup; clipped

Arrival and service provenance across traces.

CodeBench's jittered result stamp minus the recorded duration is an execution-start proxy, not a measured receipt time. ACcoding has no submission clock: arrivals are drawn, including the Poisson comparison and the deadline-shaped construction; its submission ids determine ordering, not elapsed time. Over 469 contests, the Spearman correlation between a contest's first submission id and its recorded start time is 0.9995, but in 207 of 468 consecutive transitions the id ranges of two contests overlap. ACcoding's measured cost sums test-case execution times, each subject to its problem limit, and the replay adds an assumed 0.5 s overhead. Thus neither the observed 30.0 s sum nor the resulting 30.5 s maximum is a verified whole-job timeout.

For Azure, the replay arrival is the documented end timestamp minus duration, again a start proxy; the publisher explicitly states that invocation timestamps were modified from the production trace [2]. The Netbatch replay uses the recorded SWF submission time and run duration. Their bounds in Table S3 are training p99.9 truncations, not observed platform timeouts. Both FCFS and every reordered policy receive the same truncated service sequence. The truncation affects 1,187 Azure jobs and 7,696 Netbatch jobs in the analysed pools; it removes 3.18% and 2.16% of raw work.

In the CI pools, arrival is the recorded scheduled instant, when dependencies have cleared and a worker may claim the run; recorded wait is started minus scheduled. Service is resolved minus started plus a fitted same-worker setup term. The analysis first adds setup and then clips this occupancy at the training p99.9 L ; longer jobs are retained with truncated service, not excluded. This clips 227 runs in pool A and 27 in pool B. FCFS and all comparison policies use exactly that same capped occupancy sequence. The source scheduler's priority-then-FIFO rule and operational capacity are distinct from the counterfactual FCFS model used for the guarantee.

An enforced L in a deployed service would instead kill an unfinished job at the limit and charge that executed work to the same-input FCFS comparison. The killed job would not yield its intended result; these replays do not estimate the resulting completion-quality cost or users' resubmission response. The table therefore supports finite-input bounds under its stated transformations, not a future operational timeout claim.

S2.5. The Steady-State Formula against This Load

Section 3 of the manuscript reports that the non-preemptive priority formula of [11] is out by an order of magnitude on this load. This is the comparison behind the two figures it quotes.

The synthetic control is a Poisson stream whose service distribution matches the first two moments of the real one. On it the formula agrees with simulation to 0.4%, so the implementation of the formula is not what fails. On the same workload replayed at its recorded timestamps the measured wait is 10.7–13.7 times what the formula predicts on the development trace. A class threshold tuned on the formula would therefore be tuned against a number that is off by an order of magnitude, which is why the manuscript states every promise on the realised sample path instead.

S3. Experiments

S3.1. The Experimental Configuration

Table S4 sets out the experimental configuration and the selection protocol for the guard parameters, unchanged from the version Section 8 of the manuscript refers to.

S3.2. What the Selection Rule Chose

Table S4 gives the three pre-stated grids, the feasibility rule, the objective and the tie-break, and names the shape the rule selected at each promise. Table S5 gives the selected points in full and the nine per-shape winners. The same values are in `protocol_lock.draft.json`, which is the artefact the selection was frozen into.

Table S4. Experimental configuration and the selection protocol for the guard parameters.

Item	Value
Trace and pool	CodeBench development terms under the 60 s limit, overlaid as in Section 7.2 of the manuscript; 17,634,760 jobs, 30 weeks, five overlays
Load levels	$k = 8/5/4$ on four of the five overlays and $7/5/4$ on the fifth, whose work per busy hour is the lowest; busy-hour $\rho = 0.529/0.847/1.059$ on overlay 0
Single-server configuration	separate trace, 321 copies, 3,209,347 jobs, busy-hour $\rho = 0.80$ at $k = 1$
Primary metric	99th percentile of the wait among jobs arriving in the 24 h before their own deadline; secondary metrics as defined in Section 3 of the manuscript, the gap closed printed beside the percentage reduction
Intervals	paired week-block bootstrap, 2,000 resamples of the 30 whole weeks, one draw of week multiplicities applied to every policy and to all five overlays
Promise to parameter	$B_{\max} = k(G - (3 - 2/k)L)$ with $L = 60$ s, no data used
Budget shapes	one mechanism under one cap, three shapes: constant ($\gamma = \eta = 0$), age-relative ($+\eta k(t - a_q)$), queue-length ($+\gamma n_q$). Three pre-stated grids, $42 + 162 + 54 = 258$ points
Selection rule	a point is feasible if its harm is $\leq G/2$ at each of the three load levels of each of the five validation overlays (15 cells); the objective is the deadline-window gap closed in the worst cell; ties to smaller worst harm, then smaller B_0 , then η , then γ . Guard(G) is the winner over the three grids together; the winner inside each grid is an ablation row
What the rule selected	Age-relative at $G = 300$ and 600 s ($B_0 = 60$ and $120k/4$ s, $\eta = 0.50$ and 0.75), queue-length at $G = 1200$ s ($\gamma = 16k/4$ s). Table S5 gives the three selected points with their worst-cell gap and harm, the feasible counts and the nine per-shape winners; the same values are in <code>protocol_lock.draft.json</code>
Comparators	As named in Table 2 of the manuscript. Fixed(G) is the equal-promise budget $B_{\max}$, which the harm rule rejects at every promise; Skip(G) takes the largest dispatch-charged count (S1) allows at that G and that cell's k

Table S5. The three selected budget points and the nine per-shape grid winners. Budgets in work scale as $k/4$; the harm limit is $G/2$, the feasibility threshold of the selection rule. “Feasible” counts the grid points that met it in all 15 validation cells.

Item	Value
Selected, $G = 300$ s	age-relative: $B_0 = 60k/4$ s, $\eta = 0.50$, $\gamma = 0$; worst-cell gap 0.4243, worst-cell harm 122.8 s against a 150 s limit, 44 of 99 candidate points feasible
Selected, $G = 600$ s	age-relative: $B_0 = 120k/4$ s, $\eta = 0.75$, $\gamma = 0$; worst-cell gap 0.7216, worst-cell harm 293.5 s against 300 s, 70 of 107 feasible
Selected, $G = 1200$ s	queue-length: $B_0 = 120k/4$ s, $\eta = 0$, $\gamma = 16k/4$ s; worst-cell gap 0.7275, worst-cell harm 152.2 s against 600 s, 95 of 111 feasible
Per-shape winners	constant: $B_0 = 329.671, 981.126, 2,029.958 k/4$ s at the three promises; age-relative: $(60, 0.50), (120, 0.75), (360, 0.75)$; queue-length: $(B_0, \gamma) = (120, 4), (120, 16), (120, 16) k/4$ s

S3.3. The Guard Table in Full, and the Two Budget Shapes

Table 7 of the manuscript prints the nine Guard(G) rows. Table S6 is the same table with every comparator row: Guard-fixed(G), the equal-promise Fixed(G) and the skip-count baseline at each of the three promises.

Table S6. The guard at three promises and the comparators, in full; Table 7 of the manuscript prints the nine Guard(G) rows of it. Guard(G) is the shape the selection rule picks over all three grids and Guard-fixed(G) is the winner inside the constant grid; the age-relative and queue-length winners are Table S7, and one of the three repeats Guard(G) at each promise. Fixed(G) uses the whole cap at the same promise and is infeasible under the harm rule. Budgets scale as $k/4$ and the skip count is $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$, the largest integer with $(\kappa + 2k - 2)L/k \leq G$, so both differ with the cell's k : at $\rho = 0.5$ the four overlays at $k = 8$ run $\kappa = 26/66/146$ and the fifth, at $k = 7$, runs $23/58/128$; the column prints the second of the two. Excess and harm are the worst over the five overlays, the rest are means. All rows use original source-clock scores (optimistic reference).

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$; FCFS p99 _{dl} = 27.64 s, SPJF-E 14.68 s with max excess 1,241.4 s								
	Guard(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-fixed(300)	300	15.75	0.697 [0.673, 0.717]	43.2	143.0	132.1	2.12
	Fixed(300)	300	15.30	0.721 [0.696, 0.738]	44.7	193.0	181.8	0.77
	Skip(300), $\kappa = 23$	300	26.38	0.078 [0.052, 0.107]	4.8	62.1	59.6	45.41
	Guard(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-fixed(600)	600	15.09	0.733 [0.706, 0.752]	45.4	298.7	237.0	0.36
	Fixed(600)	600	14.91	0.742 [0.715, 0.763]	46.0	493.3	237.0	0.11
	Skip(600), $\kappa = 58$	600	24.64	0.184 [0.135, 0.233]	11.5	100.2	63.3	37.26
	Guard(1200)	1200	14.78	0.750 [0.723, 0.772]	46.4	996.7	106.0	0.09
	Guard-fixed(1200)	1200	14.83	0.747 [0.719, 0.768]	46.2	557.8	237.0	0.06
	Fixed(1200)	1200	14.70	0.753 [0.724, 0.775]	46.6	1,077.0	237.0	0.00
	Skip(1200), $\kappa = 128$	1200	21.62	0.367 [0.288, 0.432]	22.8	102.3	67.6	28.80
$\rho = 0.8, k = 5$; FCFS p99 _{dl} = 119.35 s, SPJF-E 43.33 s with max excess 3,044.5 s								
	Guard(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-fixed(300)	300	64.44	0.640 [0.543, 0.699]	45.9	171.6	151.1	20.62
	Fixed(300)	300	53.83	0.763 [0.721, 0.785]	54.8	210.9	195.4	10.75
	Skip(300), $\kappa = 17$	300	119.20	0.002 [-0.001, 0.009]	0.1	64.2	58.8	80.53
	Guard(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-fixed(600)	600	49.85	0.809 [0.786, 0.824]	58.1	315.4	285.7	4.72
	Fixed(600)	600	47.03	0.842 [0.824, 0.853]	60.5	502.6	447.1	1.16
	Skip(600), $\kappa = 42$	600	118.69	0.008 [-0.002, 0.026]	0.6	97.5	63.5	74.02
	Guard(1200)	1200	46.31	0.850 [0.829, 0.865]	61.1	1,096.1	117.7	0.41
	Guard-fixed(1200)	1200	46.76	0.845 [0.826, 0.857]	60.7	581.8	469.2	0.86
	Fixed(1200)	1200	45.32	0.862 [0.842, 0.874]	61.9	1,106.3	469.2	0.22
	Skip(1200), $\kappa = 92$	1200	117.48	0.022 [0.004, 0.074]	1.6	162.5	83.6	66.10
$\rho = 1.0, k = 4$; FCFS p99 _{dl} = 273.50 s, SPJF-E 62.79 s with max excess 5,904.5 s								
	Guard(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-fixed(300)	300	223.69	0.217 [0.151, 0.359]	18.2	165.1	158.4	41.50
	Fixed(300)	300	169.29	0.453 [0.369, 0.619]	38.1	222.6	208.2	28.33
	Skip(300), $\kappa = 14$	300	273.14	0.001 [-0.001, 0.003]	0.1	81.3	81.3	90.36
	Guard(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-fixed(600)	600	116.75	0.682 [0.601, 0.764]	57.3	325.4	292.9	17.50
	Fixed(600)	600	86.53	0.813 [0.778, 0.851]	68.3	515.8	465.5	7.23
	Skip(600), $\kappa = 34$	600	272.98	0.002 [-0.004, 0.006]	0.2	112.4	82.4	86.25
	Guard(1200)	1200	79.32	0.844 [0.817, 0.870]	70.9	1,110.4	123.4	1.40
	Guard-fixed(1200)	1200	83.32	0.827 [0.798, 0.860]	69.5	576.5	522.7	5.76
	Fixed(1200)	1200	72.94	0.872 [0.846, 0.893]	73.3	1,112.9	708.8	0.73
	Skip(1200), $\kappa = 74$	1200	273.08	0.002 [-0.009, 0.013]	0.1	169.7	105.0	80.69

Section 8.4 of the manuscript summarises where the three budget shapes differ, and Section S3.11 gives the paired differences. Table S7 gives the age-relative and queue-length rows themselves, in the same format and the same cells. One of the three shape winners coincides with Guard(G) at each promise, which is why some rows below repeat a row of Table S6.

Table S7. The age-relative and queue-length winners at three promises and three loads, in the format of Table 7 of the manuscript. Excess and harm are the worst over the five overlays, the rest are means; budgets scale as $k/4$.

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$								
	Guard-age(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-queue(300)	300	15.43	0.714 [0.688, 0.733]	44.2	189.4	75.1	0.92
	Guard-age(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-queue(600)	600	14.88	0.744 [0.718, 0.765]	46.1	504.5	106.0	0.16
	Guard-age(1200)	1200	14.73	0.751 [0.723, 0.773]	46.5	1,049.1	237.0	0.01
	Guard-queue(1200)	1200	14.78	0.750 [0.723, 0.772]	46.4	996.7	106.0	0.09
$\rho = 0.8, k = 5$								
	Guard-age(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-queue(300)	300	56.13	0.736 [0.694, 0.761]	52.8	210.7	109.5	10.95
	Guard-age(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-queue(600)	600	47.69	0.834 [0.814, 0.847]	59.9	508.5	117.7	1.35
	Guard-age(1200)	1200	45.16	0.864 [0.844, 0.876]	62.0	1,102.9	370.5	0.29
	Guard-queue(1200)	1200	46.31	0.850 [0.829, 0.865]	61.1	1,096.1	117.7	0.41
$\rho = 1.0, k = 4$								
	Guard-age(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-queue(300)	300	171.79	0.442 [0.357, 0.576]	37.2	220.0	102.2	28.14
	Guard-age(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-queue(600)	600	91.09	0.793 [0.758, 0.833]	66.6	515.8	123.4	7.08
	Guard-age(1200)	1200	72.52	0.874 [0.851, 0.893]	73.4	1,117.3	453.3	0.81
	Guard-queue(1200)	1200	79.32	0.844 [0.817, 0.870]	70.9	1,110.4	123.4	1.40

S3.4. Deliberately Corrupted Scores

Under a reversed score, a random score and a longest-called-shortest score, the guard still holds every job inside its promise, and the deadline-window percentile is not rescued. Table S8 is the measurement Section 8.4 of the manuscript reads.

Table S8. Deliberately corrupted scores, unguarded and under Guard(600). Waits and excesses in seconds; corruption of the original source-clock scores.

Load	Score	p99 _{dl}		Max excess		Harm guarded
		unguarded	guarded	unguarded	guarded	
$\rho = 0.5$	reversed	42.72	43.39	1,240.1	497.1	287.1
	random	16.82	17.12	1,307.7	489.9	324.0
	longest as shortest	38.24	38.71	1,305.8	497.1	273.4
$\rho = 0.8$	reversed	228.12	232.41	3,387.9	513.5	323.4
	random	78.52	88.39	3,319.4	504.5	352.3
	longest as shortest	212.54	222.52	3,292.0	516.1	306.0
$\rho = 1.0$	reversed	557.44	546.48	6,295.7	534.8	309.1
	random	194.78	261.93	6,235.5	520.7	332.9
	longest as shortest	523.55	543.63	6,107.8	534.7	324.6

S3.5. The Residual on All Nine Cells

Table S9 gives the residual Δ_i of Theorem 2 of the manuscript on all nine cells of overlay 0, and Section 8.5 of the manuscript quotes its largest ratio. Read across the cells, the residual is a property of the schedule and barely moves with the policy: inside each load the ratios are similar in scale, but are not equal to the third decimal. An R^2 rounded to unity is likewise not exact equality.

Table S9. The residual $\Delta_i = k(W_P[i] - W_{FCFS}[i]) - (In_i - Out_i)$ of Theorem 2 of the manuscript on overlay 0, 17,634,760 jobs per cell. Ratio is $\max |\Delta_i|$ over the bound $2(k-1)L$, which is $14L$, $8L$ and $6L$ at the three loads; R^2 and the worst error are for $(In_i - Out_i)/k$ against the realised excess over first-come first-served.

Load	Policy	$\max \Delta_i /L$	Ratio	$\Delta_i = 0$	R^2	worst err. (s)
$\rho = 0.5, k = 8$	SPJF-E	6.788	0.485	95.6%	0.985	50.9
	Guard(600)	6.788	0.485	95.6%	0.983	50.9
	Reversed score	6.788	0.485	95.6%	0.996	50.9
$\rho = 0.8, k = 5$	SPJF-E	3.922	0.490	87.2%	0.996	47.1
	Guard(600)	3.922	0.490	87.2%	0.995	47.1
	Reversed score	3.881	0.485	87.5%	0.999	46.6
$\rho = 1.0, k = 4$	SPJF-E	3.082	0.514	81.1%	0.998	46.2
	Guard(600)	3.082	0.514	81.1%	0.998	46.2
	Reversed score	2.994	0.499	81.7%	1.000	44.9

S3.6. A Single Server

At $k = 1$ the additive constant $(3 - 2/k)L$ is exactly L , and the identity of Theorem 2 of the manuscript holds with no error term. Table S10 gives the configuration in full. Guard(600) and Fixed(600) close 0.854 and 0.868 of the gap, with overlapping intervals, but differ by a factor of 3.41 in harm; the same comparison gives 3.01 at $G = 300$ s and 12.18 at $G = 1200$ s, so the looser the promise, the more the shape of the budget matters. The realised excess reaches 595.5 s against the promised 600 s, and 597.0 s under the worst corrupted score: at one server almost all of the promise is used. The parameters are the ones the selection rule chose on the 15 multiserver validation cells; $k = 1$ is not among them, the jointly selected shape transfers at the two tighter promises and not at $G = 1200$ s (Section S3.14), and we do not re-tune on it.

The firing rate of the three unguarded policies is printed ---: the column counts dispatch epochs at which the guard overrides the base policy, and these three runs have no guard. Scoring first-come first-served at 100% by the convention that every first-come first-served dispatch serves the queue head, as an earlier version of this table did, compares it with the guarded runs on a quantity it does not have.

Table S10. Single server, busy-hour $\rho = 0.80$, 3,209,347 jobs. Waits and excesses in seconds. The parameters are the ones the rule selected on the multiserver validation cells; this configuration is not among them. The skip count is $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$.

Policy	Promise	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
FCFS	—	221.27	0.000	0.0	0.0	—
SJF on true sizes	—	45.32	1.000	2,054.5	777.9	—
SPJF-E	—	60.91	0.911 [0.854, 0.933]	2,520.3	656.4	—
Guard(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-fixed(300)	300	125.68	0.543 [0.207, 0.718]	140.3	119.5	17.09
Guard-age(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-queue(300)	300	117.01	0.593 [0.376, 0.685]	286.9	81.8	8.79
Fixed(300)	300	83.18	0.785 [0.584, 0.865]	297.9	246.3	6.33
Skip(300), $\kappa = 5$	300	220.39	0.005 [-0.002, 0.019]	95.1	81.8	73.17
Guard(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-fixed(600)	600	82.10	0.791 [0.602, 0.868]	303.0	246.3	6.16
Guard-age(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-queue(600)	600	91.52	0.737 [0.640, 0.812]	582.0	81.8	3.43
Fixed(600)	600	68.48	0.868 [0.783, 0.906]	593.4	563.6	2.29
Skip(600), $\kappa = 10$	600	219.89	0.008 [-0.004, 0.037]	100.9	83.2	66.98
Guard(1200)	1200	89.73	0.748 [0.661, 0.819]	1,165.1	81.8	2.13
Guard-fixed(1200)	1200	69.03	0.865 [0.789, 0.905]	564.2	509.3	2.57
Guard-age(1200)	1200	63.80	0.895 [0.835, 0.920]	1,182.3	366.4	0.68
Guard-queue(1200)	1200	89.73	0.748 [0.661, 0.819]	1,165.1	81.8	2.13
Fixed(1200)	1200	63.19	0.898 [0.837, 0.923]	1,180.4	656.4	0.45
Skip(1200), $\kappa = 20$	1200	217.64	0.021 [-0.008, 0.084]	168.0	97.7	58.67

S3.7. Outcome Visibility: Exposure, Lag and the Variants

Four things stand behind the headline of Section 8.1 of the manuscript: the online simulation that produces it, the same-copy queueing exposure that the online and exact protocols correct for, an audit of the release lag of the conservative reference, and the references its table names. All four are defined here.

The conservative figure can be taken apart. At the busiest load, Guard(600) closes 0.429 of the gap under the conservative reference and unguarded SPJF-E 0.670. Applying the reference's three restrictions to the frozen original model gives 0.536, and refitting the models under them accounts for the rest of the fall to 0.429: of the fall from the original 0.801, the restrictions account for 0.265 and the refit for 0.107. Leaving out, in addition, the results of other classes in the same term moves the exact figure from 0.672 to 0.655.

Let H_i be the set of distinct outcome records used by score i , counting a record only once if it enters several entity aggregates. Separate records from the same class-term copy, histories from other classes or terms, and records with no submission job in the replayed pool. Same-copy queueing exposure is

$$V_i(P) = \{j \in H_i : \text{copy}(j) = \text{copy}(i), \quad a_j + W_P[j] + C_j > a_i\}.$$

An external past record is not unavailable merely because it has no replayed job. The earlier audit mapped histories through the same replication round while different classes were independently shifted. Its roughly 80% affected jobs and 95.39% with unreplayed history are mapping-exposure statistics, not estimates of same-copy queueing leakage. Section S5 specifies the external-history clock and the separate class-history sensitivity.

The conservative score restricts user, exercise and pair history to submission outcomes in the same class-term copy, and admits a result only when $\text{done}_j + 3600 \leq a_i$, with times in seconds. The static score uses code, course and clock information known at submission, without outcome history. Both use their own fitted models; original, conservative and static therefore differ in more than release delay. Their differences cannot be

attributed to queueing delay alone and are not mathematical upper or lower performance bounds. Guard parameters remain those selected under the original score. The online protocol below is the empirical headline and the exact protocol its stricter reference; the original-score results are optimistic references, and the conservative and static results are controls.

For a same-copy submission, $W_P[j] \leq D$ is a sufficient availability condition under a source release lag D . The largest development FCFS wait is 1544.602708 s. Adding the largest evaluated promise, 1200 s, still lies below the 3600 s lag. All conservative Guard and FCFS cells have 0 lag violations. Unguarded conservative SPJF-E has 336 job occurrences with waits beyond the lag across the evaluated cells; it lacks the guard's certificate, so its performance is a lagged-score comparator with residual availability violations. A new pool must pass the per-job lag audit; the development maximum does not certify it in advance.

The online protocol freezes the original model weights and simulates each policy once. When job i arrives, its score is computed from the history a deployed scheduler would hold at a_i : every record from another class or term at its recorded time, and a same-copy record j only if $a_j + W_P[j] + C_j \leq a_i$ in this simulation, that is, only if job j has already completed. Rolling windows over the same copy are ordered by these completion times. A score depends only on completions before its job arrives, and those depend only on the scores of jobs that arrived earlier, so one pass defines the simulation. We implement it by pausing the simulator at every arrival whose visible same-copy history differs from the one its current score was computed on, recomputing that job's history and score, and resuming. A second implementation reaches the same simulation by iteration: simulate, rescore every job whose visible history differs, and simulate again until none does. On the first overlay at the lightest load the two agree job for job for SPJF-E, Guard(600) and Timeout(600), in every wait and in every rescored score. About 255,000 of the 17.6 million arrivals pause, and the pausing implementation needs 4 to 5 minutes per policy against 15 to 26 for the iteration. In every development cell, 2,000 jobs drawn at random and up to 2,000 rescored ones are scored again directly from their visible history, and every one matches the simulation.

The exact protocol freezes the original model weights, replays each policy, leaves out same-copy outcomes still unavailable at the target arrival, recomputes affected histories and scores, and repeats until no used outcome violates availability. The set of left-out outcomes only grows, so this finite procedure terminates. Every development cell, five overlays by three loads by five policies, and every sensitivity cell ended its final pass with no used outcome unavailable. The procedure certifies availability in that final replay. Unlike the online protocol, it never returns a left-out result, even one its final replay completes in time, so it can use less history than a deployed scheduler would hold. The visibility table of Section 8.1 of the manuscript prints Guard(600) under all five protocols. The other policies under each protocol, the same-copy exposure statistics of each replay, and the attribution ladder that separates leaving results out from refitting are in the package output directory `outputs/dev_consistent_visibility/`, in `exact_comparison.csv`, `same_copy_exposure.csv` and `attribution_comparison.csv`; the emitted tables `tab_exact_visibility.tex` and `tab_same_copy_exposure.tex` under `outputs/consistent_paper_tables/` typeset the first two.

S3.8. Aging Outside the Budget Family

A linear aging credit, selected under the conservative score and the guard's harm limit, carries no proved per-job guarantee and closes far less of the gap. Section 8.4 of the manuscript states the comparison; here it is run.

Table S11 extends the policy definitions with linear aging. It uses the conservative score of Section S3.7, with a validation search over 11 credits, of which 4 pass the 300 s harm limit. The selected credit is $\omega = 0.03$; its worst validation gap is 0.0375 and harm 246.36 s. The label Aging(600) records the reference promise used to set the validation constraint, and carries no proved per-job guarantee. All evaluated loads and harmful outcomes are retained.

Table S11. Additional policy definition; the reference label is not a theorem.

Policy	Dispatch priority, smallest first	Guarantee
Aging(G)	$\hat{C}_i - \omega(t - a_i)$, equivalently $\hat{C}_i + \omega a_i$; ties by arrival rank. Select ω on validation cells under harm $\leq G/2$.	None proved

Table S12. Conservative-score comparison with the selected aging baseline. All waits, excesses and harms are in seconds; intervals are paired week-block intervals.

Load	Policy	p99 _{dl}	Gap closed	Max excess	Harm
0.5	Guard(600)	16.68	0.640 [0.604, 0.671]	499.9	235.2
0.5	Aging(600)	23.83	0.228 [0.180, 0.271]	306.8	155.8
0.8	Guard(600)	64.77	0.634 [0.560, 0.688]	509.4	327.7
0.8	Aging(600)	109.83	0.111 [0.082, 0.161]	673.0	179.1
1.0	Guard(600)	174.71	0.429 [0.357, 0.533]	527.8	303.5
1.0	Aging(600)	261.68	0.051 [0.035, 0.074]	855.6	184.7

S3.9. Package Predictor Metrics and Their Splits

The AUROC of Table 1 of the manuscript and the AUROC of the deployed expected-cost score are not the same measurement. The first belongs to the retrospective classifier comparison, the second to the score the scheduler uses. This is what separates them.

The AUROC 0.928 in the earlier predictor comparison is M4 under the protocol of Section 4.2 of the manuscript, with results released at their recorded times, using heavy-class classifier probabilities averaged over seeds, trained on the core terms and evaluated on 2022-2. It is not the AUROC of the package's deployed expected-cost score. The latter is 0.9360 pooled over rolling forward development predictions and 0.9372 on 2022-2; the conservative pooled score is 0.9116. Table S13 uses the package values and the split definitions that the sealed predictor results of Section S10 also use. The earlier value remains evidence for its own retrospective classifier comparison, whose predictions never enter scheduling; it is not a competing estimate of the same fitted score.

Table S13. Package development AUROC with user-block intervals. Pooled means pooling unrepliated forward-prediction rows, not averaging term AUROCs.

Target	Score variant	AUROC [interval]
pooled	Original	0.9360 [0.9289, 0.9424]
pooled	Conservative	0.9116 [0.9054, 0.9173]
pooled	Static	0.8371 [0.8249, 0.8492]
2022-2	Original	0.9372 [0.9218, 0.9491]
2022-2	Conservative	0.9196 [0.9060, 0.9324]
2022-2	Static	0.8806 [0.8544, 0.9022]

S3.10. Alternative Ranking Losses

Alternative ranking losses were fitted on the same features and run through the same simulator (Table S14). The fitted quantile scores and raw-second squared-error model lose

to the Tweedie fit at every load in this comparison. Raw squared error and the Tweedie mean loss have the same unrestricted population target, so this does not identify a change of estimand as the cause. Finite model capacity, error weighting, regularisation and optimisation can produce different rankings for the same target. A two-class rule that sends predicted-heavy jobs behind predicted-light ones and runs FCFS inside each class reaches a deadline-window p99 of 64.05 s at $\rho = 0.8$ on the first overlay, against 43.86 s for the full ordering on the same overlay, so most of the benefit comes from the fine ordering inside the light class and little of it from separating the two classes.

Table S14. Ranking scores fitted on one feature set and run through one simulator: fraction of the deadline-window gap closed, five overlays, and the paired difference against the log-scale score. Archived source-clock fits, distinct from the package refit in Table S16; differences between the tables do not compare identical fitted scores.

Score	$\rho = 0.5$	$\rho = 0.8$	$\rho = 1.0$	Difference at $\rho = 1.0$
SPJF-log	0.7528	0.8728	0.8967	—
SPJF-E	0.7515	0.8847	0.9159	+0.0191 [+0.0111, +0.0238]
Expected cost, gamma hurdle	0.7404	0.8826	0.9141	+0.0174 [+0.0082, +0.0223]
Probability of heavy	0.7476	0.8821	0.9122	+0.0155 [+0.0066, +0.0199]
Predicted 99th percentile	0.7245	0.8742	0.9005	+0.0038 [−0.0050, +0.0097]
Predicted 90th percentile	0.7024	0.8470	0.8755	−0.0212 [−0.0297, −0.0156]
Predicted median	0.7071	0.8196	0.8402	−0.0565 [−0.0684, −0.0463]
Squared error, raw seconds	0.7197	0.8585	0.8873	−0.0094 [−0.0194, −0.0043]

S3.11. What the Shape of the Budget Is Worth

The two shapes the main table no longer prints are read here through the paired differences and in full in Table S7, all three at one fixed promise. The shape matters where the promise is tight and the load is high, and hardly at all elsewhere. At $G = 600$ s the constant shape trails the joint winner by 0.007 [0.003, 0.012], 0.028 [0.020, 0.038] and 0.119 [0.074, 0.176] of the gap as the load rises through the three levels, each interval clear of zero; at $G = 300$ s the same three differences are 0.008 [−0.002, 0.015], 0.099 [0.056, 0.157] and 0.262 [0.202, 0.288]; and at $G = 1200$ s they are 0.003 [−0.000, 0.008], 0.005 [−0.001, 0.011] and 0.017 [0.002, 0.031]; only the last, at the busiest load, clears zero, and barely. At a loose promise and a light load the shape is not doing measurable work.

The other axis is harm. At $G = 600$ s the queue-length shape reaches within 0.009 of the joint winner's gap at every load while its worst-overlay harm is 106.0, 117.7 and 123.4 s against the winner's 237.0, 319.6 and 339.9 s, under half at every load. That is also why the rule picks the queue-length shape at $G = 1200$ s, where the three shapes are within 0.005 of one another on the validation objective and it has the smallest worst harm. The shape decides which jobs pay for the promise more than it decides how much of the benefit survives.

Fixed(G), which spends the whole cap, is an equal-promise candidate in the search, but is infeasible under the validation harm constraint at every promise. Its evaluation-overlay harm is 181.8, 195.4 and 208.2 s at $G = 300$ s against a 150 s limit, 237.0, 447.1 and 465.5 s at $G = 600$ s against 300 s, and 237.0, 469.2 and 708.8 s at $G = 1200$ s against 600 s. Thus the evaluation cells also exceed the corresponding limits at each promise. It is ahead of the joint winner on the gap closed in eight of the nine cells, by up to 0.028, so it buys its extra gap with harm above the declared limit in some cells; we make no claim that any shape beats it on the gap at equal promise.

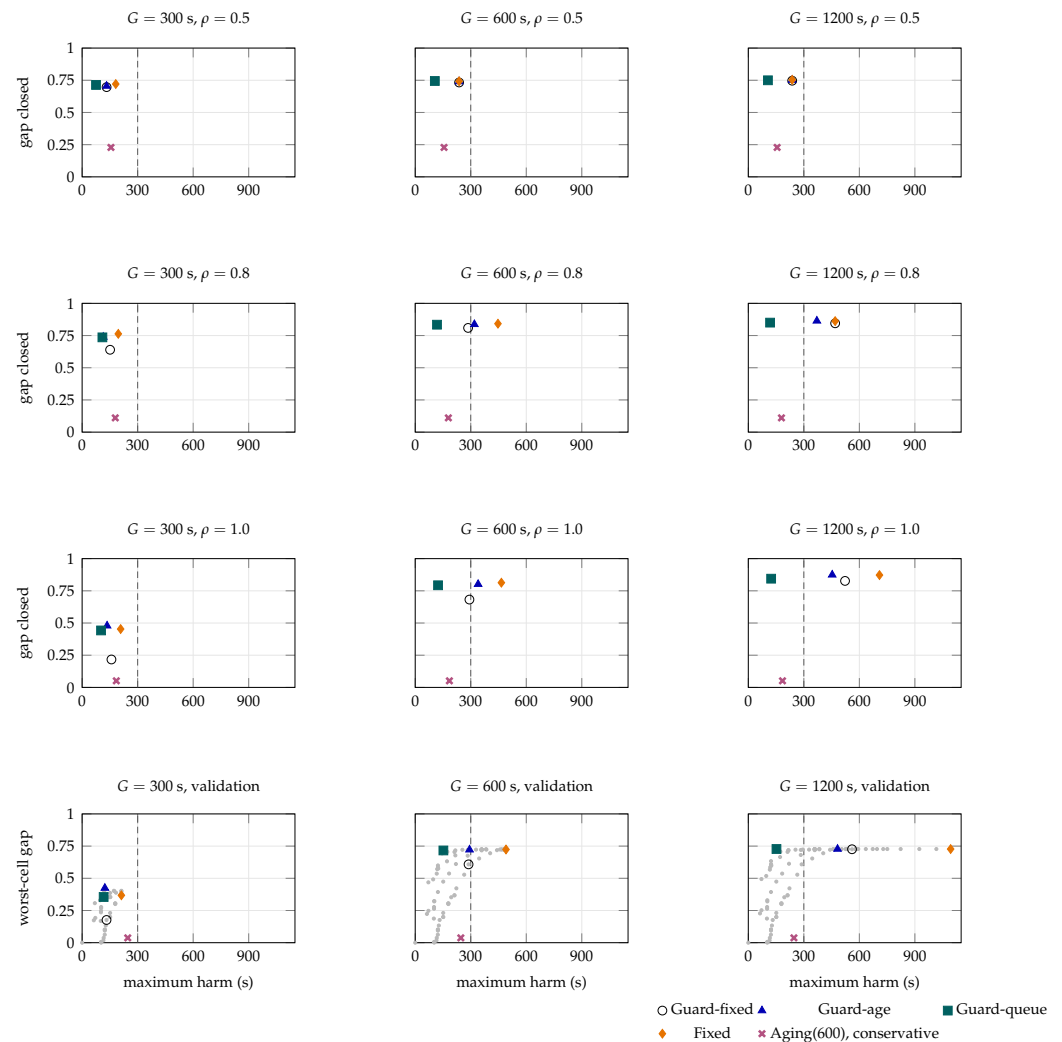


Figure S1. Gap-harm trade-offs for every selected budget shape and Fixed(G), at each promise and load (upper rows); validation candidates and per-family selections (bottom row). A joint winner coincides with its selected family point. The dashed line is the 300 s validation limit for $G = 600$ s; it is shown in every panel as a common reference, not as the constraint for every promise. The other promise-specific limits follow $\text{harm} \leq G/2$. All budget-family points use original scores. The magenta point is the only selected aging comparator, Aging(600), with conservative scores and no proved guarantee; it is repeated across promise panels for orientation, not returned at other promises. Cross-protocol points do not establish Pareto dominance. Maxima have no population interval; gap intervals for the selected rows are reported in the tables.

S3.12. The Gap Closed Against the Promise

The fraction of the gap each policy closes is plotted below against the promise, with the count-based baseline beside it. Section 8.4 of the manuscript cites this figure for the count-based baseline.

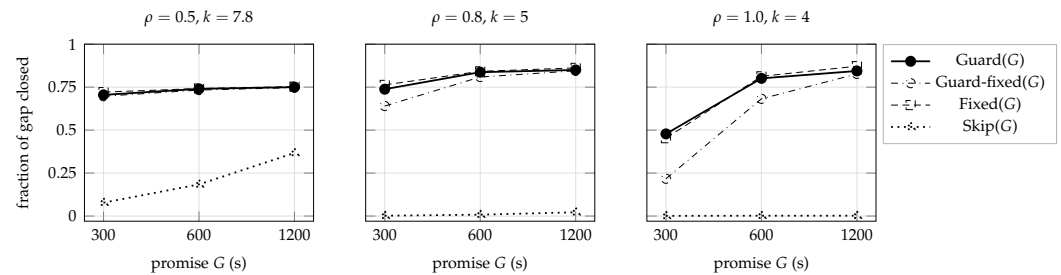


Figure S2. Original source-clock scores (optimistic reference): what each budget shape keeps of the FCFS-to-SJF gap as the promised excess G is relaxed. Every point is a “Gap closed” cell of Table S6; the intervals are in that table and are omitted here. All four policies carry the same promise at a given G , since the promise depends on the cap alone. The gap between Guard(G) and its constant-shape ablation opens where the promise is tight and the load is high and closes elsewhere; counting positions rather than work (Skip) collapses at the two higher loads; Fixed(G), which spends the whole cap, buys its gap with harm the selection rule rejects.

S3.13. The Identity, Measured Job by Job

Neither exhaustive search nor real input reaches the error term of Theorem 2 of the manuscript. Exhaustive and random search over millions of small instances never exceeds $(k - 1)L$ by more than one time unit, and on the 17.6-million-job trace of Section 8 of the manuscript the largest residual reaches 0.470 to 0.518 of $2(k - 1)L$, again about $(k - 1)L$. That is where a search which does not build the cascade of Theorem 4 of the manuscript stops, so the measurement says the workload does not attain the constant and not that the constant is loose. The constant is where Section 6 of the manuscript put it: the family that approaches it is a geometric cascade needing hundreds of jobs arranged against the wrapper, and nothing in a replayed workload arranges them.

The net-overtake identity had been tested on small synthetic instances and never measured on the trace this paper runs on, so we measured it. Table S9 reports the residual $\Delta_i = k(W_P[i] - W_{FCFS}[i]) - (In_i - Out_i)$ on overlay 0 at the three load levels under Guard(600), SPJF-E and the reversed score, with FCFS on the same input as the reference schedule in every cell. Recovering In_i and Out_i needs the dispatch sequence, which the guard kernel does not expose, so the kernel and the independent reference simulator were each copied verbatim and given recorders that add no branch and no state update; the two agree on the dispatch sequence of every job of 3,000 random instances, the recording kernel reproduces the production kernel’s waits element for element inside every cell, and In_i and Out_i are carried in exact integer microseconds, the currency the guard itself meters.

Theorem 2 of the manuscript is asserted on every job of every cell: 0 violations over 1,428,415,560 job-checks, the 27 policies of this overlay at the three loads. The residual stays far from its ceiling. The largest $|\Delta_i|$ over the nine cells of Table S9 is $6.788L$, at $k = 8$, where the bound $2(k - 1)L$ is $14L$, and over those nine cells the maximum sits between 0.485 and 0.514 of the bound, which is about $(k - 1)L$; over all 81 cells the largest is 0.5173 of the bound, so it never exceeds 0.518, for the reason Section S3.16 gives. Same-phase bookkeeping, the mechanism behind the degenerate extremal instances of Remark S1 above, accounts for at most 0.36% of the total In in any of those nine cells, so the executed-work convention would not change the picture for these policies. It is not negligible everywhere: under the randomised ranking, which ties many jobs into one dispatch instant,

the share reaches 14.95%. $(In_i - Out_i)/k$ explains $R^2 = 0.983$ to 0.9996 of the variance of $W_P[i] - W_{FCFS}[i]$, with a worst per-job error of 44.9 to 50.9 s where the theorem permits $(2 - 2/k)L$, that is 90 s at $k = 4$ and 105 s at $k = 8$. This is one overlay of development data, and the residual is a property of a schedule rather than a statistic, so no interval is attached to it.

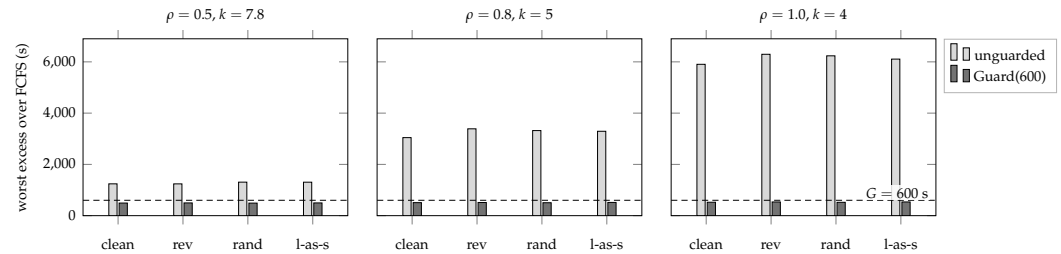


Figure S3. Original-score reference and its corrupted-score controls. The worst single job, with and without the guard, under the expected-cost score (“clean”) and the three corrupted scores of Table S8: reversed (rev), random (rand) and truly-longest-called-shortest (l-as-s). The dashed line is the promise $G = 600$ s. Sources: the clean pair is the block header and the Guard(600) row of Table 4 of the manuscript; the three corrupted pairs are the “Max excess” columns of Table S8. The unguarded bar grows with load and with corruption; the guarded bar does not, which is the content of Theorem 7 of the manuscript and not a property of the trace.

S3.14. Choosing a Shape on One Set of Cells

The single-server configuration also shows the cost of choosing a shape on one set of cells. The parameters were selected on the 15 multiserver validation cells, and $k = 1$ is not among them. At $G = 300$ and 600 s the joint winner transfers: it closes 0.746 and 0.854 of the gap, against 0.543 and 0.791 for the constant shape. At $G = 1200$ s it does not: the queue-length shape that wins the joint comparison on the multiserver cells closes 0.748 here, while the age-relative shape closes 0.895 and the constant shape 0.865. Section 8.5 of the manuscript says what we do with that and Section 9 there says what it means.

The promise itself is unaffected, since it holds for every budget under the cap; what the shape decides is how much of the base policy’s benefit survives. We report the number the pre-stated rule produces and do not re-tune on the configuration that exposes it. An operator whose pool is much smaller than the validation pool should re-run the selection rather than take our shape. The three shapes also differ in where the harm falls, so which is preferable depends on a judgement about the jobs first-come first-served would have started at once, and that judgement is the operator’s.

S3.15. Trace-Construction Sensitivities

Five trace-construction variants were run through an earlier source-clock pipeline (Table S15). These are sensitivity results for its stored M4 log-scale score and fixed completed-work budget, not the selected expected-cost Guard(G). Within that earlier configuration, the alternatives retain a positive gap; adding the term whose busiest hour is dominated by runs that outlived their container raises the gap, and the opposite reading of the timestamp, in which the stamp is receipt rather than completion, gives 0.926 against 0.895. Merging or dropping the groups of runs by one user that end in the same second moves the gap closed by 0.017 and 0.003 at full load, so that earlier result survives these two treatments of co-ending runs; delaying the visibility of a completed job’s outcome costs prediction quality (Section 4.2 of the manuscript and Section S1.3). The earlier fixed-budget guard is more sensitive to the construction, ranging from 0.442 to 0.843 of the gap at a fixed budget of 600 s at full load. What these variants show is that the earlier ordering experiment is sensitive to how the timestamps are read, to which source terms are included,

and to how co-ending runs are treated. They validate neither the final selected Guard(G), nor conservative history, nor the policy-specific exact protocol. A trace built from the last development term alone is not available this way: loading a pool of four from it would take 190 copies of its 8 class-terms, which stacks every deadline of one term on the same weekday and hour.

Table S15. Trace-construction sensitivities at busy-hour $\rho = 1.0$, three overlays, fraction of the deadline-window gap closed. Archived M4 log-scale score and fixed completed-work budget; the budget in the last column is not a promise of equal value.

Variant	k	FCFS p99 _{dl}	Unguarded	Guard, budget 600 s
Primary pool (overlays 0–2)	4.000	—	0.895 [0.864, 0.916]	0.442 [0.345, 0.631]
Earlier term added	4.667	95.41	0.945 [0.885, 0.965]	0.843 [0.715, 0.880]
Co-ending runs merged	4.000	228.19	0.912 [0.892, 0.924]	0.603 [0.445, 0.740]
Co-ending runs dropped	4.000	209.30	0.898 [0.879, 0.914]	0.608 [0.471, 0.753]
Opposite timestamp reading	4.000	268.75	0.926 [0.904, 0.939]	0.545 [0.422, 0.699]

Table S15 supports sensitivity of the earlier ordering experiment to timestamp interpretation, source-term inclusion and co-ending-run treatment. It does not validate robustness of the final selected Guard(G), conservative history or policy-specific exact visibility.

Two of the constructions were rerun with the final method: the expected-cost score, the selected budget shapes and the original protocol, on the five development overlays at busy-hour $\rho = 1.0$. With the earlier term added, the FCFS p99_{dl} is 129.69 s, SPJF-E closes 0.941 of the gap and Guard(600) closes 0.920 [0.886, 0.938]. With the stamp read as receipt, the FCFS p99_{dl} is 272.33 s, SPJF-E closes 0.941 and Guard(600) closes 0.861 [0.839, 0.884]. On the primary pool the three figures are 273.50 s, 0.916 and 0.801. The largest excess under Guard(600) is 510.9 s and 527.4 s, against 523.7 s on the primary pool, so both constructions keep the promise, and both leave the guard a larger share of the gap than the primary pool does. The co-ending-run treatments and the conservative and exact protocols were not rerun. Section S5 gives the configuration and release boundary explicitly.

S3.16. Further Measurements from the Guarded Runs

The worst wait among heavy jobs, over the five overlays, is 487, 1,023 and 1,543 s under FCFS at the three loads, 1,375, 3,313 and 6,286 s under SPJF-E, and 921, 1,467 and 1,993 s under Guard(600). The guard therefore leaves the worst heavy job within 434, 444 and 450 s of the FCFS figure, inside the promise, where the unguarded ranking multiplies it by 2.8, 3.2 and 4.1.

The ratio of the realised worst excess to the bound reaches 0.9950 at $k = 1$, 0.9311 at $k = 4$, 0.9219 at $k = 5$, 0.8975 at $k = 7$ and 0.8850 at $k = 8$. The finite-skip bound, by contrast, is attained exactly on small instances, at a ratio of 1.0000.

Every guarded run asserts the per-job bound for each job separately, inside the simulation rather than afterwards on summary statistics. Across the 315 policy-by-cell runs that is 5,554,949,400 checks, and the single-server configuration adds 21 runs and 67,396,287 checks, all with 0 violations. A second simulator, written independently and sharing no code with ours, reproduces the runs job for job over 17,634,760 jobs and 22 policies with 0 differing jobs, and asserts the same bounds on its own waiting times over 5.7 billion checks of its own. That cross-check was run against the earlier development pipeline, whose dispatch sequence the package reproduces.

Replacing the largest 1% of underestimates by the true cost moves the deadline-window p99 at $\rho = 1.0$ from 66.57 s to 45.01 s on the first overlay and from 65.02 s to 45.32 s on the second; repairing the largest 1% of overestimates instead moves it to 60.37

s and 59.72 s. Section 8.3 of the manuscript prints the four fractions of the distance to the SJF reference that these give, and the two at 0.1%.

The count baseline is charged at dispatch, which allows 34 rather than 30 passes at $k = 4$ and $G = 600$ s; charging at completion instead changes the gap closed by at most 0.015, and by at most 0.001 at the two higher loads.

S3.17. The Unguarded Policies at the Three Loads

Table S16 is the table Section 8.3 of the manuscript reads for the unguarded comparison: the four ranking policies and the two development rows, at the three loads, on the original source clock.

Table S16. Unguarded policies, five overlays. Waits in seconds; gap and reduction on the deadline-window p99, with 95% paired week-block intervals. Original source-clock scores; these are optimistic references.

Load	Policy	p99 _{dl}	Gap closed	Red. (%)	Mean
$\rho = 0.5, k = 7.8$	FCFS	27.64	0.000	0.0	0.719
	SJF	10.59	1.000	62.0 [58.7, 66.4]	0.228
	SPJF-log	14.71	0.754 [0.722, 0.777]	46.7 [43.9, 50.1]	0.296
	SPJF-log, refitted ^a	14.72	0.753 [0.722, 0.776]	46.7 [44.0, 50.0]	0.295
	SPJF-E	14.68	0.754 [0.725, 0.776]	46.7 [43.9, 49.9]	0.292
	SPJF-E, GRU state ^a	14.87	0.741 [0.707, 0.767]	45.9 [43.2, 48.8]	0.290
$\rho = 0.8, k = 5$	FCFS	119.35	0.000	0.0	4.160
	SJF	33.55	1.000	71.8 [67.0, 75.1]	0.959
	SPJF-log	44.36	0.873 [0.852, 0.887]	62.7 [57.5, 66.2]	1.252
	SPJF-log, refitted ^a	44.41	0.873 [0.851, 0.886]	62.7 [57.4, 66.2]	1.251
	SPJF-E	43.33	0.885 [0.862, 0.899]	63.6 [58.1, 67.4]	1.232
	SPJF-E, GRU state ^a	42.82	0.891 [0.870, 0.904]	64.0 [58.5, 67.7]	1.230
$\rho = 1.0, k = 4$	FCFS	273.50	0.000	0.0	10.270
	SJF	43.63	1.000	84.0 [79.6, 86.8]	1.696
	SPJF-log	67.17	0.897 [0.872, 0.914]	75.4 [70.2, 78.6]	2.354
	SPJF-log, refitted ^a	67.36	0.896 [0.871, 0.913]	75.3 [70.1, 78.5]	2.352
	SPJF-E	62.79	0.916 [0.892, 0.930]	77.0 [71.6, 80.3]	2.294
	SPJF-E, GRU state ^a	62.71	0.917 [0.895, 0.929]	77.0 [71.7, 80.3]	2.302

^a These two rows are not produced by the released package, which fits only the expected-cost and log-scale scores. They come from the earlier development study (evidence/main_v3), which used a floating-point clock and a stored copy of the predictions, and are printed at the values that study reported; the remaining rows differ from it in the second decimal of a wait.

S4. Deferred Proofs

This section gives the proofs that Section 6 of the manuscript states without them or only in outline. The manuscript proves in full the inequality of the workload comparison, the net-overtake identity, the guard bound and the necessity pair; they are repeated here with the strictness arguments the manuscript omits. The constructions showing that the constants cannot be lowered are in Section S8 of this file.

We use the manuscript's notation, collected in its Table 3. A job's *rank* is its position in the arrival order; $j < i$ means that j has the lower rank and $j \prec i$ that j is started before i . $U_Q(t)$ is the unfinished work at time t under policy Q , $N_Q(t)$ the number of jobs present, In_i the total cost of the higher-rank jobs started before i , and Out_i the total cost of the lower-rank jobs started after i . A *dispatch epoch* is an instant at which a server is free and a job waits, so that a decision is made. $E(t)$ is the set of jobs eligible at a decision at time t , the waiting jobs q with $\text{over}[q](t) \geq \text{budget}(q, t)$; when it is not empty the guard starts its minimum-rank member, and otherwise the base policy chooses. The consistency proof of Section S4.2 uses two further facts. A higher-rank job that has completed by t while q still

waits arrived after q and started before it, so it overtook q , and $\text{over}[q](t) \leq \text{In}_q$ in the run considered; and Corollary 6 of the manuscript bounds In_q under a guarantee.

S4.1. The Workload Comparison

Proof of Lemma 1 of the manuscript. Put $D = U_{Q_1} - U_{Q_2}$. Arrivals add the same jump to both terms, so D is continuous, and between arrivals it changes at rate $-(\min(N_{Q_1}, k) - \min(N_{Q_2}, k))$; it is therefore piecewise linear with breakpoints among the finitely many arrival and completion instants of the two runs.

If $U_{Q_1}(u) > (k-1)L$ then more than $k-1$ jobs are present under Q_1 , each carrying at most L , so every server of Q_1 is busy and $\min(N_{Q_1}, k) = k \geq \min(N_{Q_2}, k)$: D does not increase at u . Fix t and let $S = \{u \leq t : U_{Q_1}(u) \leq (k-1)L\}$, which is non-empty because U_{Q_1} vanishes before the first arrival, and put $t_0 = \sup S$. The supremum need not be attained, since U_{Q_1} jumps upwards at arrivals, so we use the left limit: there are $u_n \uparrow t_0$ with $U_{Q_1}(u_n) \leq (k-1)L$, hence $U_{Q_1}(t_0^-) \leq (k-1)L$ and, D being continuous, $D(t_0^-) \leq U_{Q_1}(t_0^-) \leq (k-1)L$ because $U_{Q_2} \geq 0$. On $(t_0, t]$ we have $U_{Q_1} > (k-1)L$, so D does not increase there, and $D(t) \leq (k-1)L$. Exchanging Q_1 and Q_2 gives the other direction. That the constant cannot be lowered is the cascade of Section S8.1 of this file, which reaches $(k-1)L(1 - ((k-1)/k)^m)$ after m rounds. At $k = 1$ the bound reads $|D| \leq 0$ and is attained everywhere: one server executes work at rate 1 whenever any job is present, so $U_{Q_1} \equiv U_{Q_2}$. $\square$

Proof of the strict inequality in Lemma 1, for $k \geq 2$. Keep the notation above, so $D = U_{Q_1} - U_{Q_2}$ is continuous and piecewise linear with finitely many pieces. Suppose $D(t) = (k-1)L$ for some t and let t_1 be the smallest such t ; it exists because $\{D = (k-1)L\}$ is closed and $D(0) = 0 < (k-1)L$, which uses $L > 0$ and $k \geq 2$.

Let I be the linear piece immediately to the left of t_1 , which has positive length. Its slope is strictly positive: a negative slope would put $D > (k-1)L$ somewhere on I and a zero slope would put $D = (k-1)L$ on all of I , and either contradicts the minimality of t_1 . The slope of D is $-(\min(N_{Q_1}, k) - \min(N_{Q_2}, k))$, so $\min(N_{Q_1}, k) < \min(N_{Q_2}, k)$ on I and therefore $N_{Q_1} \leq k-1$ there.

With at most $k-1$ jobs present and Q_1 work-conserving, none of Q_1 's jobs waits, so all of them are in service; hence U_{Q_1} falls at rate N_{Q_1} on I , and $U_{Q_1} \leq (k-1)L$ on I because each of the at most $k-1$ present jobs carries at most L . Taking left limits at t_1 and using the continuity of D ,

$$U_{Q_1}(t_1^-) - U_{Q_2}(t_1^-) = (k-1)L, \quad U_{Q_1}(t_1^-) \leq (k-1)L, \quad U_{Q_2} \geq 0,$$

which forces $U_{Q_2}(t_1^-) = 0$ and $U_{Q_1}(t_1^-) = (k-1)L$. Since $L > 0$ and $k \geq 2$, this gives $N_{Q_1} \geq 1$ on I —with $N_{Q_1} = 0$ we would have $U_{Q_1}(t_1^-) = 0 = (k-1)L$ —so U_{Q_1} is strictly decreasing on I and $U_{Q_1}(t_1^-) < \sup_I U_{Q_1} \leq (k-1)L$, a contradiction. Exchanging Q_1 and Q_2 gives the other direction. Both steps that invoke $(k-1)L > 0$ fail at $k = 1$, where the statement is false: there $U_{Q_1} \equiv U_{Q_2}$ and the bound 0 is attained at every instant. $\square$

S4.2. Consistency, and Why the Converse Fails

Section 6 of the manuscript states in prose that a guard which never fires leaves the base policy alone. Here is the statement it summarises, with its proof.

Proposition S1 (Consistency, one direction). *Run the base policy A alone. If $\text{over}[q](t) < \text{budget}(q, t)$ for every waiting q at every dispatch epoch of that run, then the wrapped run is identical to it, with the same dispatch sequence and the same waits. Consequently, for a constant*

budget B , each of the following implies $\text{guard}(A, B) = A$: $\max_i \text{In}_i^A < B$; or A itself satisfies $\text{excess}_A[i] \leq G$ on this input and $B > kG + (3k - 2)L$.

Proof. Compare the two runs decision by decision, with the same realisation of any randomness in A . Suppose they have made the same choices up to a decision at time t . Then the same jobs have completed by t in both runs, so $\text{over}[q](t)$ is the same in both for every waiting job q , and by hypothesis it is below $\text{budget}(q, t)$. No job is eligible, so the guard lets A choose, and A , seeing the same history, makes the choice it makes in its own run. The runs therefore agree at this decision too, and by induction at every decision.

For the first consequence, a higher-rank job that has completed by t while q still waits overtook q in A 's run, so $\text{over}[q](t) \leq \text{In}_q^A < B$ and the hypothesis holds. For the second, Corollary 6 of the manuscript gives $\text{In}_q^A \leq kG + (3k - 2)L < B$, and the first consequence applies. $\square$

Consistency is one-sided: a run on which no job ever reaches its budget is the base policy's own run, but a run the guard leaves unchanged need not be one. Take $k = 1$, two jobs of service 1 at $t = 0$, the base policy "higher rank first", and $B = 1$. Both runs dispatch in the order $(1, 0)$, yet at the second dispatch the waiting job has $\text{over} = 1 \geq B$, so the guard fires and happens to choose what the base would have chosen. Nor is there multiplicative consistency: with $k = 1$, one job of service L followed by m jobs of service 1 all at $t = 0$, the wrapper with $B = 0$ is first-come first-served while the base ordering by service time is not, and the ratio of mean waits is $(2L + m - 1)/(m + 1)$, unbounded in L .

S4.3. The Net-Overtake Identity, and Its Two Directions

The manuscript proves the net-overtake identity; the proof is repeated here with the strictness argument. A remark follows on why its two extremes do not have the same content, and on what the executed-work convention would change.

Proof of Theorem 2 of the manuscript. *Busy-period accounting.* Job i waits throughout $[a_i, s_i^P)$, so by work conservation every server is busy on that interval and exactly $k W_P[i]$ units of work are executed there, none of it on i . (If $s_i^P = a_i$ the interval is empty and both sides vanish.) Split that work by rank.

A job j of rank above i has $a_j \geq a_i$, so it executes inside the window only if it is dispatched inside it, that is only if $j \prec i$; conversely every such j is dispatched at or before s_i^P . The work executed on jobs of rank above i is therefore $\text{In}_i - \beta^{\text{new}}$, where β^{new} is the remaining work at s_i^P of those overtakers not finished by then; a same-phase overtaker contributes C_j to both terms and 0 to the executed work, as it must.

Every job of rank below i is present at a_i , with total remaining R_i^P . At s_i^P their total remaining is Out_i , the jobs not dispatched before i each carrying their full service time, plus β^{old} , the unfinished part of those dispatched strictly before i , so the work executed on them is $R_i^P - \text{Out}_i - \beta^{\text{old}}$. Adding the two parts gives equation (9) of the manuscript, $k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \beta_i^P$ with $\beta_i^P := \beta^{\text{old}} + \beta^{\text{new}}$.

Bounding β . The jobs counted in β_i^P occupy distinct servers, and none of them occupies the server that takes i , so there are at most $k - 1$ of them and each has remaining work at most L : $0 \leq \beta_i^P \leq (k - 1)L$.

The reference policy. Under first-come first-served, $\text{In}_i = 0$, since a job of rank above i dispatched before i would have to be chosen at an instant at which i waits, and $\text{Out}_i = 0$, since at s_i^{FCFS} every job of rank below i has already been dispatched. So (9) reads $k W_{\text{FCFS}}[i] = R_i^{\text{FCFS}} - \beta_i^{\text{FCFS}}$ with $0 \leq \beta_i^{\text{FCFS}} \leq (k - 1)L$.

Comparison. At a_i , after the arrivals there are processed, the two systems contain the same jobs, and those of rank i or above are untouched in both, so $U_P(a_i) - U_{\text{FCFS}}(a_i) =$

$R_i^P - R_i^{\text{FCFS}}$ and Lemma 1 of the manuscript gives $|R_i^P - R_i^{\text{FCFS}}| \leq (k-1)L$, strictly for $k \geq 2$. Subtracting the two instances of (9) gives the identity. Each direction uses only two of the three ranges: upward $R_i^P - R_i^{\text{FCFS}} \leq (k-1)L$ and $\beta_i^{\text{FCFS}} \leq (k-1)L$ with $\beta_i^P \geq 0$, and downward the mirror image, so the slack is $2(k-1)L$ and not $3(k-1)L$. At $k = 1$ all three terms vanish. $\square$

Remark S1 (The upward and downward extremes have different content). *Upward, the extremal value is a statement about delay: on the witnesses we have computed exactly, none of In_i is same-phase work and the victim's own excess is about $+3L$. Downward, half of the extremal value is an accounting convention. There $\text{In}_i = (k-1)L$ consists entirely of overtakers dispatched in the victim's own phase, which execute nothing while it waits; the same-phase share of $|\Delta_i|$ is 50% to 63% at $k = 2, 3, 4$ (Section S8.2), and the downward victim's excess is negative, since it starts earlier under P than under first-come first-served. If In_i is redefined as the work of higher-ranked jobs actually executed during $[a_i, s_i^P)$, Theorem 2 of the manuscript holds with the same constant and the same proof, the upward supremum is still $2(k-1)L$, and the best downward witness we have falls to $(k-1)L$. We keep the dispatch-sequence convention for two reasons. Its constant is exactly determined in both directions, whereas the downward constant for the executed-work variant is open, somewhere in $((k-1)L, 2(k-1)L]$; and its two sides are symmetric. Completed work lower-bounds both currencies, so the guard is implementable either way: a job j of rank above i that completes while i waits has $a_j \geq a_i$ and is dispatched at or after a_i , so all of C_j is executed inside $[a_i, s_i^P)$ and is counted in full by the executed-work variant as well.*

S4.4. What a Count of Overtakes Costs

A count bound on overtakes gives a per-job guarantee only through the worst-case charge of L per pass, and is not necessary for one. Section 6 of the manuscript states that in prose; here are the remark it summarises and the proofs of both halves.

Remark S2 (Counting overtakes is not necessary, and is sufficient only at the worst-case charge). *Bounded-overtaking mechanisms in the literature limit how many times a job may be passed [12–14]. Inside the model of Section 3 of the manuscript, where $0 < C_i \leq L$, a count bound does give a per-job guarantee; it is not necessary for one, and it gives the guarantee only at a factor $L/\bar{C}$, where $\bar{C}$ is the mean cost. The count rule, called Skip in the manuscript, keeps a pass counter for every waiting job, adds one to it each time a higher-rank job is started ahead of the job, and, once some counter has reached κ , starts the earliest-arrived such job next; we say the rule fires at that decision. Counted when an overtaker is started, which is how a position counter in the prior art counts, a bound of κ passes gives $\text{In}_i \leq \kappa L$, and Corollary 3 of the manuscript turns that into*

$$W_P[i] \leq W_{\text{FCFS}}[i] + (\kappa + 2k - 2)L/k, \quad (\text{S1})$$

which is the accounting and the formula used wherever that baseline appears in the manuscript; inverting it, a promise G admits $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$ passes, the largest integer with $(\kappa + 2k - 2)L/k \leq G$. Either accounting charges every overtaker the worst case L , so the rule is loose against a work budget by the ratio $L/\bar{C}$, 150 on the main trace ($L = 60$ s against a mean cost of 0.40 s).

When the count is taken decides which guarantee follows. Counted when an overtaker is started, a job's counter is the number of higher-rank jobs started while it waits, so an earlier-arrived waiting job has at least as many passes as a later one, and the job a firing decision starts is the earliest-arrived waiting job, whose rank is at most i 's while i waits. No overtaker of i is therefore started at a firing decision. At every other decision all counters, i 's included, are below κ , and each overtaker started adds one to i 's counter.

So at most κ overtakers are started while i waits, which gives $\text{In}_i \leq \kappa L$ and (S1). Charging the count at completion instead leaves up to $k - 1$ overtakers in service and the one being dispatched uncounted, which costs a further L on the right and gives the baseline fewer passes at the same promise.

The sufficiency half uses the bound on costs, and without that bound a single overtake makes the excess arbitrarily large. At $t = 0$ let $k - 1$ jobs of cost X , then a job of cost ϵ (the victim), then one more job of cost X arrive together. First-come first-served starts the victim immediately, a policy that takes the last job instead fills every server, and the victim waits X with an overtake count of 1. That instance steps outside Section 3 of the manuscript, which fixes $C_i \leq L$; inside the model the same accounting is what makes a count bound work, with L charged per pass. Necessity fails with no such caveat: normalise to $k = 1$, $L = 1$ and $G = 1/2$. Let a victim of service 1 arrive first together with M short jobs, each of service $1/(2M)$. Running the short jobs first gives every job excess at most $1/2$, uses $\text{In}_i = kG$ exactly, and lets the overtake count M exceed any fixed limit.

S4.5. Proofs of the Guard Bounds

The additive guarantee of the wrapper holds under every budget rule that respects the cap, and the age-relative shape carries a multiplicative form as well. Both proofs follow.

Proof of Theorem 7 of the manuscript. For (10) of the manuscript, let j be the overtaker of i dispatched last in the dispatch sequence and let t be its instant. Because dispatches are sequential and i is dispatched after j , job i is waiting at the moment j is dispatched; this includes the case $t = s_i^P$, in which j is dispatched in i 's own phase, just before i . Since j is dispatched, either $E(t)$ is empty or its minimum-rank member is j ; in both cases $i \notin E(t)$, because i has the smaller rank. Hence $\text{over}[i](t) < \text{budget}(i, t) \leq B_{\max}$.

The overtakers of i dispatched strictly before j split into those completed by t , whose total service is $\text{over}[i](t)$, and those still in service at t ; the latter occupy distinct servers, none of them the server taking j , so there are at most $k - 1$ of them, each of service at most L . Adding j itself gives $\text{In}_i < \text{budget}(i, t) + kL \leq B_{\max} + kL$.

For (11) of the manuscript, Theorem 2 of the manuscript with $\text{Out}_i \geq 0$ gives $k(W - W_{\text{FCFS}}[i]) \leq \text{In}_i + 2(k - 1)L$; substituting (10) of the manuscript and dividing by k gives the claim, strictly. If i has no overtaker then $\text{In}_i = 0$, the excess is at most $(2 - 2/k)L$, and the bound holds strictly because $L > 0$ in the model of Section 3 of the manuscript. $\square$

The age-relative shape is the one the manuscript names in prose. Its bound is stated here.

Proposition S2 (Multiplicative bound for the age-relative shape). *Suppose in addition that $\text{budget}(q, t) = \min(B_0 + \eta k(t - a_q), B_{\max})$ with one constant $B_0 \geq 0$ for every job and $\eta \in [0, 1)$. Then for every input and every job i ,*

$$\text{In}_i < B_0 + \eta kW + kL, \quad (\text{S2})$$

$$(1 - \eta)W < W_{\text{FCFS}}[i] + B_0/k + (3 - 2/k)L, \quad (\text{S3})$$

with (S2) asserted whenever i has at least one overtaker.

Proof. The argument for (10) of the manuscript gives $\text{In}_i < \text{budget}(i, t) + kL$ at the instant t of i 's last overtaker, and $t \leq s_i^P = a_i + W$ gives $\text{budget}(i, t) \leq B_0 + \eta kW$, which is (S2). Theorem 2 of the manuscript with $\text{Out}_i \geq 0$ then yields $kW - \eta kW < kW_{\text{FCFS}}[i] + B_0 + (3k - 2)L$; divide by k . Both inequalities are strict, by the same argument as Theorem 7 of

the manuscript: when i has an overtaker the strictness comes from (10) of the manuscript, and when it has none $\text{In}_i = 0$ and the excess is at most $(2 - 2/k)L < (3 - 2/k)L$. $\square$

S4.6. Why the Multiplicative Bound Is Claimed for One Shape Only

With the queue-length shape, the multiplicative bound stated with one common constant is false, which is why Section 5 of the manuscript restricts the multiplicative bound to the age-relative shape. The counterexample is below.

Remark S3 (Why the multiplicative bound is claimed for one shape only). *The proof of Proposition S2 goes through when B_0 is replaced by a quantity $B_{0,i}$ fixed at i 's own arrival, and then bounds W in terms of $B_{0,i}$. The queue-length shape of Section 5 of the manuscript sets $B_{0,q} = B_0 + \gamma n_q$ with n_q the number of jobs waiting when q arrives, so $B_{0,i}$ varies from job to job. Stated with the common B_0 the multiplicative bound is then false, by a margin that grows with γ . Take $k = 1$, $B_0 = 0$, $B_{\max} = \gamma$, any $\eta \in [0, 1)$, and the input: a job of service L at $t = 0$, which is dispatched at once; a second job of service L at $t = 1$, which waits with $n_q = 0$; the victim i , also of service L , at $t = 1$ and ranked after it, so that i arrives behind a single waiting job, has $n_i = 1$ and hence budget γ ; and γ jobs of service 1 at $t = 2L$, all ranked above i . First-come first-served runs the first two jobs on $[0, L]$ and $[L, 2L]$ and starts i at $2L$, so $W_{\text{FCFS}}[i] = 2L - 1$. Under the wrapper the second job is taken at $t = L$ as well, by the guard if $\eta = 0$ and by the base otherwise; at $t = 2L$ the victim has $\text{over}[i] = 0 < \gamma$, so the guard does not fire and the base spends the whole budget on the stream, giving $W_P[i] = 2L - 1 + \gamma$ against a right-hand side of $(3L - 1)/(1 - \eta)$. The bound is violated for every $\gamma > (3L - 1)/(1 - \eta) - (2L - 1)$, that is for $\gamma > L$ at $\eta = 0$, and the margin grows without bound in γ . The same happens for every per-job constant. For $\gamma > 0$ we therefore claim Theorem 7 of the manuscript and not Proposition S2.*

S4.7. Proofs of the Necessity Pair

A work budget is necessary for a per-job guarantee, up to an additive $O(kL)$. The two statements that make that argument, Lemma 5 and Corollary 6 of the manuscript, are proved here.

Proof of Lemma 5 of the manuscript. Every job j counted in Out_i has rank below i , so first-come first-served dispatches it before i and $s_j^P \leq s_j^{\text{FCFS}} + G \leq s_i^{\text{FCFS}} + G$; it is also dispatched at or after s_i^P . All of them therefore start inside a window of length $w = G - \text{excess}_P[i] \geq 0$. Of the jobs one server starts inside such a window, all but the last complete inside it, so their total service is at most w and the last adds at most L . $\square$

Proof of Corollary 6 of the manuscript. Theorem 2 of the manuscript gives $\text{In}_i - \text{Out}_i \leq k \text{excess}_P[i] + 2(k - 1)L$; add Lemma 5 of the manuscript, whose hypothesis holds because the guarantee is assumed for every job, and the terms in $\text{excess}_P[i]$ cancel. $\square$

S4.8. The Floor of Order L , and the Scope of Its Hypothesis

The floor of Proposition 13 of the manuscript is proved here, and its hypothesis is shown to be neither droppable nor tightenable, the remaining cases of the budget family being exhaustive.

Proof of Proposition 13 of the manuscript. Into an empty system, at $t = 0$, let $k + 1$ jobs arrive together, all of service L . The first k ranks are precisely the jobs FCFS starts at once. Nothing has completed, so every waiting job has $\text{over} = 0$; because its budget at the first decision is positive, no waiting job is eligible and the wrapper accepts the base policy's choices. At the first choice of a job beyond the first k ranks, that job displaces one of FCFS's first k equal-length jobs. The displaced job waits until a completion under the wrapper but starts at zero under FCFS, and hence has excess L . A prediction-ranked base

produces the same choice when one of the later ranks receives the preferred score. At $k = 1$ and $B > 0$, (10) of the manuscript gives excess $< B + L$ strictly, and a family approaching it is in Section S8.4; $B = 0$ is FCFS. $\square$

The hypothesis on the budget rule cannot be dropped. Without it the statement would claim a floor for every non-preemptive work-conserving policy that observes costs only at completion, and first-come first-served is such a policy with excess 0 on every input. It would also claim a floor for every base policy, and the guard run around first-come first-served is first-come first-served whatever the budget. The floor comes from a budget rule that keeps the guard from acting at the first decision, together with a base policy that uses the freedom this leaves it. The hypothesis is also not necessary. A rule that is 0 at the first dispatch epoch and positive later, such as the age-relative shape with $B_0 = 0$ and $\eta > 0$, falls to the same instance placed behind a saturating prefix: let k jobs of service L arrive at $t = 0$, the victim just after them, and k further jobs just before $t = L$. The victim's wait up to $t = L$ is what first-come first-served gives it, its budget at $t = L$ is positive because it has aged, and the k newcomers then take every server. The remaining case is degenerate: if the rule gives budget 0 to a job whenever no completed work has overtaken it, then at every dispatch epoch the lowest-ranked waiting job h has $\text{over}[h] = 0$, so h is eligible and, having the lowest rank, is the job the guard starts, and by induction the wrapper is first-come first-served itself, whatever the base policy. For the budget of Algorithm 1 of the manuscript these cases are exhaustive. If $B_{\max} = 0$ or $B_0 = \gamma = \eta = 0$ the budget is identically zero and the wrapper is first-come first-served. If $B_0 > 0$ the proposition applies, and if $B_0 = 0 < \eta$ the saturating prefix does. If $B_0 = \eta = 0 < \gamma$, let $k + 2$ jobs of service L arrive together at $t = 0$: the first has $n_q = 0$, hence budget 0, and is released at once, every other job has a positive budget and nothing has overtaken it, so a base policy that prefers the highest ranks passes over the second job at $t = 0$ when $k \geq 2$, and at $t = L$ when $k = 1$, and that job's excess is L . A budget rule of this family therefore either makes the wrapper first-come first-served around every base policy, or admits a base policy and an input with excess at least L .

S4.9. Why the Budget Is Measured in Work and Not in Positions

A count of passes is the currency of the finite-skip and Nudge- K rules [12–14]. Counting is not necessary: a job can be passed any number of times and stay inside its promise, provided the work that passes it is small. Counting is sufficient in this model, but only through the worst-case charge of L per overtake, since every overtaker is bounded by the time limit; a count of κ therefore buys a promise of $(\kappa + 2k - 2)L/k$ (Remark S2 and (S1)). That is the bound a count can give, and it is loose by the ratio $L/\bar{C}$, 150 on the main trace. Ten overtakers that each run for the full limit L and ten that run for ten milliseconds carry the same count and delays that differ by three orders of magnitude, so the charge is paid on every dispatch whether or not the work is there. Section 8 of the manuscript runs the count rule as a baseline and measures how much the looseness costs on a real load. That the excess is set by the work that passes a job, and not by the number of passes, is what the identity of Section 6 of the manuscript states; it is not a claim about what earlier mechanisms could not do.

S4.10. Which Assumptions Can Be Dropped

Four of the assumptions behind Theorem 2 and Theorem 7 of the manuscript can be dropped, and one cannot. The global bound on service times goes first, and it is the one whose replacement has to be read at the right instant.

Proposition S3 (The prefix maximum replaces the global bound). Write $\Lambda(t) = \max\{C_j : a_j \leq t\}$ for the largest service time among the jobs that have arrived by t . No global bound on service times is needed, provided Λ is read at the right instant, and the right instant is not the same for every statement:

1. the upward direction of Theorem 2 of the manuscript, and hence Corollary 3 of the manuscript, hold with L replaced by $\Lambda(a_i)$;
2. the wrapper bounds (10) and (11) of the manuscript, and likewise (S2) and (S3), hold in the sharp forms $\text{In}_i < B_{\max} + k \Lambda(s_i^P)$ and $\text{excess}_P[i] < B_{\max}/k + \Lambda(s_i^P) + (2 - 2/k) \Lambda(a_i)$, and since $\Lambda(a_i) \leq \Lambda(s_i^P)$ they give (11) with $\Lambda(s_i^P)$ throughout; with $\Lambda(a_i)$ in place of $\Lambda(s_i^P)$ both are false, and by an unbounded margin;
3. the downward direction of Theorem 2 of the manuscript holds with $\Lambda(\max(s_i^P, s_i^{\text{FCFS}}))$, and Corollary 6 of the manuscript holds with $\Lambda(s_i^{\text{FCFS}} + G)$; both are false with $\Lambda(a_i)$.

Item 1 holds because every job counted in R_i^P , R_i^{FCFS} or β_i^{FCFS} has rank below i and so arrived by a_i . Item 2 needs the later instant because the kL term of (10), which (11) inherits, charges the last overtaker and the at most $k - 1$ overtakers still in service beside it, all of them jobs of rank above i , which arrive after a_i and are bounded only by the prefix maximum at the instant i starts. The counterexample at the earlier instant is short. Take $k = 1$ with budget $B = 2$, let i and one job of service 1 arrive at $t = 0$ and a job of service M at $t = 1$, and let the base policy take the two others first. Then $\Lambda(a_i) = 1$ for every M , the guard never fires because $\text{over}[i]$ stops at $1 < 2$, and $\text{In}_i = M + 1$ against a claimed $B + k\Lambda(a_i) = 3$. In item 3 the horizon $\Lambda(\max(s_i^P, s_i^{\text{FCFS}}))$ is not claimed to be the smallest that works, and $\Lambda(s_i^{\text{FCFS}} + G)$ is what Corollary 6 needs because Lemma 5 of the manuscript bounds jobs that start as late as that.

The remaining four are shorter, and the last of them is the one that fails.

Proposition S4 (Relaxations). In the setting of Theorem 2 and Theorem 7 of the manuscript:

1. Servers may have different speeds. With speeds $v_1, \dots, v_k$ and $V = \sum_{\ell} v_{\ell}$, Theorem 2 of the manuscript holds with k replaced by V on the left and the same $2(k - 1)L$ on the right, and the guard satisfies $W_P[i] \leq W_{\text{FCFS}}[i] + B/V + (3k - 2)L/V$: the budget is divided by total capacity, not by the server count. Here L bounds a job's work in a common unit, not its wall-clock duration: a common wall-clock limit does not produce policy-independent executed work when speeds differ. The reference assigns each dispatched job to the fastest free server, ties by server index.
2. The timeout is already in the model. Section 3 of the manuscript defines C_j as executed work, $\min(C_j, L)$ for a job stopped at the limit, so every currency above $(\text{In}_i, \text{Out}_i, \text{over}[q](t))$ and L is in executed work and no assumption on raw service demands is needed. Read in raw work the statements are false, a stopped job contributing more to In_i than is ever executed.
3. Setup times survive. If a setup of the same length is paid before every service under both policies, add it to every job's cost; the model is unchanged and every result holds with L increased by the setup length.
4. Work conservation cannot be dropped. If servers may idle while jobs wait, let Γ_i^P be the server-idle time accumulated during $[a_i, s_i^P]$ while at least one job waited. Then equation (9) of the manuscript becomes $kW_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \beta_i^P + \Gamma_i^P$, and nothing survives without a bound on Γ : with $k = 1$, pausing the server for T while i waits leaves $\text{In}_i = \text{Out}_i = 0$ and makes the excess T . Release constraints are the same phenomenon: if rank is keyed on arrival while eligibility starts later, the identity fails by exactly the forced idle time, and ranking on release repairs it.

Only the last of the four changes what can be promised. The other three change the constant, the unit or nothing at all.

S4.11. A Clock in Place of the Counter, and an Absolute Bound

This section proves Propositions 10 and 11 of the manuscript and the combined rule stated after Proposition 10. We keep the conventions of Section 6.1 of the manuscript: a phase is the ordered list of dispatches at one instant, and remaining work at a_i is read after the arrivals at a_i .

Lemma S5 (Fluid comparison). *Let $V(t)$ be the unfinished work at t of a single server of rate k fed the same jobs at the same instants, serving whenever it holds work. For every non-preemptive work-conserving k -server policy Q and every t , $V(t) \leq U_Q(t) \leq V(t) + (k - 1)L$.*

Proof. Both quantities rise by C_j at the arrival of j , so arrivals leave $D = U_Q - V$ unchanged. Between arrivals U_Q falls at a rate equal to the number of busy servers, and V falls at rate k while it is positive. If $V > 0$, the slope of D is k minus the number of busy servers, which is non-negative; if $V = 0$, then $D = U_Q \geq 0$. Since $D = 0$ before the first arrival, $D \geq 0$ throughout. If $D > (k - 1)L$, then $U_Q > (k - 1)L$. All k servers are then busy, since otherwise the queue is empty by work conservation and the at most $k - 1$ jobs in service hold at most $(k - 1)L$. So U_Q falls at rate k and D does not rise. Hence $D \leq (k - 1)L$ throughout. $\square$

Write V_i^- for the fluid backlog at a_i of the jobs of rank below i , as defined in Section 6.7 of the manuscript. No dispatch happens between two arrivals at the same instant, and an arrival moves U_Q and V by the same amount. Reading D at a_i after the arrivals of rank below i and before i 's own gives $U_Q = R_i^Q$ and $V = V_i^-$, so Lemma S5 yields $R_i^Q \leq V_i^- + (k - 1)L$ for every policy Q . A clairvoyant policy is included, since the lemma is a statement about the schedule that was actually run.

Proof of Proposition 11 of the manuscript. Equation (9) of the manuscript gives $k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \beta_i^P \leq R_i^P + \text{In}_i$, and $R_i^P \leq V_i^- + (k - 1)L$. Under first-come first-served $\text{In}_i = 0$. Under Algorithm 1, $\text{In}_i < B_{\max} + kL$ when i has an overtaker, by Theorem 7 of the manuscript, and $\text{In}_i = 0$ otherwise. Dividing by k gives the first bound; $V_i^- \leq \sigma - C_i < \sigma$ gives the second. That σ is the least depth of a conforming rate- k bucket is the usual reading of the Lindley recursion [15]: $V_i^- + C_i$ is the largest amount by which the work arriving in a closed window ending at a_i exceeds k times the window's length. $\square$

Proof of Proposition 10 of the manuscript. Let i have an overtaker. At every decision at a time $t \geq a_i + \theta$ at which i waits, the minimum-rank waiting job has rank at most i and has waited at least $t - a_i \geq \theta$, so the rule starts it; no job of rank above i is dispatched at or after $a_i + \theta$ while i waits. Every overtaker therefore starts in $[a_i, a_i + \theta)$ and ends no later than L after its start. The overtakers on one server run one after another inside $[a_i, a_i + \theta + L)$, the last starting strictly before $a_i + \theta$, so their total service is below $\theta + L$. Summing over the k servers, $\text{In}_i < k(\theta + L)$. Theorem 2 of the manuscript with $\text{Out}_i \geq 0$ gives $k(W - W_{\text{FCFS}}[i]) \leq \text{In}_i + 2(k - 1)L < k\theta + (3k - 2)L$, and the first inequality of Proposition 11 gives the absolute form $W < V_i^-/k + \theta + (2 - 1/k)L$. A job without an overtaker has $\text{In}_i = 0$ and excess at most $(2 - 2/k)L$. $\square$

The reading of "oldest" as minimum rank is needed. If jobs that arrive at the same instant are treated as equally old and the base policy chooses among them, the step "serves a job of rank below i " fails. At $k = 1$, $L = 3$ and $\theta = 0$, six jobs arrive at $t = 0$ with sizes 1, 3, 3, 3, 3, 3. A base policy that serves the five long jobs first makes job 1 wait 15 s with $W_{\text{FCFS}} = 0$, against the bound $\theta + L = 3$.

Proof of the combined rule. The rule serves the minimum-rank waiting job if it has waited θ , and otherwise runs Algorithm 1. Suppose j is served while i , of lower rank, waits. Either the base policy chose j , in which case $E(t)$ is empty, or j is the minimum-rank member of $E(t)$, in which case $i \notin E(t)$. In both cases $\text{over}[i](t) < \text{budget}(i, t)$, which is the only property of the dispatch the proof of Theorem 7 uses; so Theorem 7 holds. When the minimum-rank job has waited θ it is served, which is the only property the proof of Proposition 10 uses; so Proposition 10 holds. The second argument reads no completion, so it holds for charges that arrive late or never. $\square$

The order of the two tests matters whenever budgets differ between jobs. Serving the minimum-rank member of $E(t)$ first can pass over an aged head, and it breaks Proposition 10. At $k = 1$, $L = 3$, $\theta = 3.5$ and $\text{budget}(q) = \min(3n_q, 5)$, jobs arrive at 0, 0, 0, 2, 2, 2 with sizes 1, 1, 1, 3, 1, 3, so their budgets are 0, 3, 5, 3, 5, 5. The base policy serves jobs 1, 2, 5 and 6 in that order, and at $t = 6$ both job 3 and job 4 have $\text{over} = 4$. Job 3 has waited $6 \geq \theta$ but is below its budget, and job 4 has passed its budget. Serving job 4 first makes job 3 start at 9, with $W_{\text{FCFS}} = 2$ and excess $7 > \theta + L = 6.5$; serving the aged head first leaves every excess at most 4. The age-relative shape gives a second instance, at $k = 1$, $\theta = 0$, $\eta = 1/4$, arrivals 0, 1, 2 and sizes 2, 3, 3. With a constant budget $E(t)$ is a prefix of the queue and the two orders coincide.

We checked every statement of this section by exhaustive enumeration and by random search, with two simulators written independently of each other and of the package. The enumeration covered $k = 1, 2, 3$, up to six jobs of sizes in $\{1, 2, 3\}$ with $L = 3$, arrivals in $\{0, \dots, 4\}$, and every choice a base policy can make at every dispatch, so clairvoyant and randomised base policies are included. It ran 53 to 56 million schedules per k . The random search covered 320,000 bursty instances with $k \leq 4$ and up to 40 jobs. It found no violation of Lemma S5, of Proposition 11 for any policy, of Proposition 10 under the rank reading, or of the combined rule under late or lost reports; every failure found came from the two readings the text excludes, ties broken by the base policy and serving $E(t)$ before an aged head, and the instances above are the smallest found. The worst observed ratio of the excess to $\theta + (3 - 2/k)L$ was 0.996 at $k = 1$ and between 0.756 and 0.829 for $k = 2, 3, 4$; for $k \geq 2$ the constant may be loose, and we do not claim it is exact.

S4.12. Completion Reports That Arrive Late

This section proves Proposition 12 of the manuscript. The charge of a job c that completes at f_c reaches the shared counter at $f_c + d_c$, with $0 \leq d_c \leq \delta$, and is added to every job of rank below c still waiting then. At one instant the order is completions, charges due by then, arrivals, dispatches, so $\delta = 0$ is Algorithm 1 exactly. Servers still pull the instant they free.

Proof of the first part. Fix i with an overtaker, let j be its last overtaker, t the instant of j 's dispatch and s^* the server taking j . As in Theorem 7 of the manuscript, $i \notin E(t)$, so $\text{over}[i](t) < B_{\max}$. Every charge added to $\text{over}[i]$ comes from an overtaker, since a job of higher rank that completes while i waits was dispatched before i . Split the overtakers dispatched before j into (a) those in service at t , (b) those completed and charged by t , and (c) those completed by t whose charge has not arrived, so $f_c \in (t - \delta, t]$; we call the jobs in (c) *stale*. Then $\text{In}_i = (a) + (b) + (c) + C_j$ with $(b) < B_{\max}$. The jobs in (a) sit on distinct servers other than s^* , so $(a) \leq (k - 1)L$. On one server s , the jobs in (c) do not overlap in time. Let c_1 be the one that finishes first. Then $C_{c_1} \leq L$, and every other job of (c) on s starts at or after $f_{c_1} > t - \delta$ and finishes by t , so together they hold less than δ . Hence (c) holds less than $L + \delta$ per server, and $\text{In}_i < B_{\max} + (k - 1)L + k(L + \delta) + L$. Theorem 2 of the manuscript with $\text{Out}_i \geq 0$ gives $k(W - W_{\text{FCFS}}[i]) < B_{\max} + (3k - 2)L + k(L + \delta)$. $\square$

The extra $L + \delta$ cannot be charged against (a): one server can hold a stale job of size L and an overtaker in service at the same instant. At $k = 1$ the following instance attains $B + 2L + \delta - 2$, which is the bound when budgets, sizes and delays are integers, since each of the two strict inequalities in the proof then loses one unit. All jobs arrive at 0. The victim has rank 1, size L and the worst score. Then come $B - 1$ unit jobs charged at once, one job of size L and $\delta - 1$ unit jobs all charged after δ , and a last job of size L , scored in that order. When the last unit finishes, over $= B - 1 < B$, the last job starts, and the victim starts $B + 2L + \delta - 2$ after its FCFS start. For $k \geq 2$ we start from the cascade of Section S8.1 with $L = k^m$, delay the charges of its final jobs by δ , let each server run $\delta - 1$ unit jobs charged late, add a second batch of k jobs of size L , and set $B = f + L + 1$. At $k = 2$ the victim's excess is then $f + 3L + \delta - 1$, two units below the bound at every depth m , so the bound is attained in the limit. At $m = 12$ the ratio is at least 0.99988 for $\delta = 1, 2048, 4096$, and at $m = 8$ the excess exceeds Theorem 7's bound by $L + \delta - 2$. On the same family at $k = 3$ and $k = 4$, every instance we built falls short of the bound by $(1 - 1/k)L((k - 1)/k)^m + 1 + 1/k$, a vanishing fraction of L as m grows; we have no proof of that formula. At $\delta = 0$ the rule is Algorithm 1 and Theorem 7 gives the smaller bound, so attainment needs $\delta > 0$.

Proof of the second part. Suppose every server applies its own completion report in the same step as its next pull. Take t, j and s^* as above, and let c be a stale overtaker on server s . Every earlier pull of s applied the report of the job s had just finished, so c is the last job s finished and s has not pulled since f_c . If $f_c < t$, then at the instant f_c the server s was free while i waited, so by work conservation it pulled at f_c and applied c 's report, a contradiction. So $f_c = t$, and s has not pulled yet in the phase at t . Then $s \neq s^*$, because s^* applied its own report before pulling j , and s holds no job in service. Per server other than s^* , (a) and (c) together are at most one job of size at most L , and on s^* both are empty. So $\text{In}_i < B_{\max} + kL$, which is inequality (10) of the manuscript, and Theorem 7 follows as before. Nothing in the argument reads another server's report, so it holds for every δ , and for reports that never arrive. $\square$

A third arrangement keeps one view of the counter per server, in which a server sees its own completions at once and other servers' within δ . The same argument then gives excess $< B_{\max}/k + (3 - 2/k)L + (1 - 1/k)(L + \delta)$. At $k = 2$ a variant of the cascade falls short of it by $3/2$ at every depth, so this bound too is attained in the limit.

We checked the three statements in two ways, with no violation in either. The first enumeration fixed static priority keys, constant budgets $B \in \{1, 2, 3\}$ and $\delta \in \{0, 1, 2\}$ at $k = 1, 2$, and took every key order and delay vector for up to four jobs and a sample of them for five and six: 720,894,276 schedules, plus 200,000 random instances with $k \leq 4$. The second, written independently, lets the adversary take any dispatch the budget allows, which covers dynamic and clairvoyant base policies. It also chooses when each charge arrives within δ , the order in which servers pull at one instant, and, under report-before-pull, whether a report arrives at all. It enumerated every such schedule for $k = 1, 2, 3$ and up to six jobs, about 1.4 billion in all, and a hill-climbing search over up to 40 jobs and $k \leq 4$ added 15.4 million instances. The combined rule of Section S4.11, run with lost reports, had no violation either.

S5. Reproducibility and Provenance of the Empirical Results

S5.1. Feature Definitions, Targets and Rolling Splits

The executable feature dictionary is `src/spjf_guard/features/sweep.py`: `HIST_COLS`, `CTX_COLS`, `GROUPS` and `feature_frame` specify the entity aggregates, missing values and event order; `src/spjf_guard/data/events.py` specifies code columns

and clock inputs. M4 combines static code, assessment/calendar context, exercise history, user history, user-exercise history and previous-event history. It excludes the relational extensions. Outcome aggregates contain log-cost moments and quantiles, error/heavy rates and recent-result summaries; counts and elapsed time since arrival are known at arrival and are distinguished from outcome fields. The rolling-history window, tie order and update formulas are defined in that dictionary. Static uses only the code and context blocks. The independent source-clock audit in Section S1 verifies that dictionary under its original clock, not under an arbitrary replayed policy.

Package scores are fitted by `src/spjf_guard/predict/forward.py`. For each target term, training includes only terms whose first arrival precedes that target, and only records whose result is readable strictly before its first arrival. History thresholds and user-slowness cutpoints are refitted from those training rows; a model is then frozen for the target term. The targets are 2020-ERE, 2020-2, 2021-1, 2021-2, 2022-1 and 2022-2. Starting at 2018-1, their training periods end respectively at 2019-2, 2020-1, 2020-2, 2021-1, 2021-2 and 2022-1; availability filtering still applies at each cutoff. Validation overlays exclude the final development term from selection, but that term and the earlier family exploration are development data, not sealed testing.

The score objectives are in `src/spjf_guard/predict/scores.py`, and the deterministic fit settings in `lgb_parameters` and the configuration identified by the output manifest. `spjf_e` is a Tweedie regression of capped seconds; `spjf_log` is squared-error regression of log-transformed cost. In contrast, the earlier M4 AUROC 0.9283 [0.9118, 0.9415] is the heavy-class classifier probability, averaged over 3 seeds, trained on 2018-1–2019-2 and tested only on 2022-2. Those retrospective classifier predictions never enter scheduling. The package's pooled `spjf_e` AUROC 0.9360 combines 402,358 unreplicated forward-prediction rows. Its evaluation heavy threshold stays at the core training p95, 1.559043 s; it is not the threshold refitted inside each model's history features. Predictor intervals resample users, whereas scheduling intervals resample weeks. The two AUROCs compare different fits, targets and pooling and must not be read as a measured improvement on a common split.

S5.2. Exact Selection Grids and Counts

The full expanded grid, its definitions, counts and deduplication totals are in `outputs/selection_v4/selection_protocol.json`; per-cell values, worst-cell values and selections are in the adjacent `selection_grid.csv`, `selection_worst.csv`, `selection_frontier.csv` and `selected_parameters.csv`. All budget and queue-work bases below scale by $k/4$ before dispatch arithmetic. The cap is determined by the promise.

The constant grid has the following 39 base budgets (seconds), shared across promises:

15, 17.99, 21.576, 25.877, 30, 31.035, 37.222, 44.641, 53.54, 60, 64.212, 77.012, 92.363, 110.774, 120, 132.855, 159.338, 191.099, 229.192, 240, 274.878, 329.671, 360, 395.386, 474.201, 480, 568.726, 600, 682.093, 818.058, 981.126, 1176.7, 1411.26, 1692.57, 2029.96, 2434.6, 2919.9, 3501.94, 4200.

It is the union of 32 logarithmically spaced values between 15 and 4200 s, rounded as recorded, and the anchors 15, 30, 60, 120, 240, 360, 480, 600, with duplicate values removed. A full-cap constant point is added at each of $G = 300, 600, 1200$ s. This makes 42 expanded constant entries.

For each promise the age-relative grid is the Cartesian product

$$B_{0,\text{base}} \in \{0, 15, 30, 60, 120, 240, 360, 480, 600\}, \quad \eta \in \{0, 0.25, 0.5, 0.75, 0.9, 0.95\}, \quad \gamma = 0.$$

The queue-length grid is

$$B_{0,\text{base}} \in \{0, 30, 120\}, \quad \eta \in \{0, 0.9\}, \quad \gamma_{\text{base}} \in \{1, 4, 16\}.$$

Thus the family counts over all promises are $42 + 162 + 54 = 258$ expanded points. Deduplicating identical dispatch rules after scaling and saturation leaves 243 distinct schedules at each evaluated server count. This is a computational deduplication, not a reduction of the declared search space.

Candidate counts per promise answer a different question. The constant entries shared across promises qualify for a given G only if their worst-cell certified promise is at most G ; an age-relative or queue-length entry belongs to its own declared promise. Feasibility then additionally requires $\text{harm} \leq G/2$ in every validation cell. Table S17 separates these filters, explaining why the per-promise totals differ from the expanded search size. The objective is the minimum deadline gap over all validation cells, with ties resolved by smaller worst harm, then smaller B_0 , η and γ .

Table S17. Candidates / feasible points after promise assignment and the harm filter.

G (s)	Constant	Age-relative	Queue-length	Joint
300	27 / 22	54 / 15	18 / 7	99 / 44
600	35 / 31	54 / 30	18 / 9	107 / 70
1200	39 / 35	54 / 43	18 / 17	111 / 95

The aging comparator has the credit grid 0.00001, 0.00003, 0.0001, 0.0003, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3 and 1. It uses the same validation pool, harm threshold and worst-cell objective, with ties to less harm then larger credit. It uses conservative scores; the guard grids used the original package score. The selected guard parameters are subsequently held fixed across visibility controls, so this is not a claim of reselection under conservative or exact histories. The aging label carries no proved guarantee.

S5.3. Replication, History and Aggregation Order

Each overlay is built from the same 40 class-terms replicated 44 times with independent whole-week shifts; each of the five overlays therefore reuses the same jobs. Original scores are computed on the source calendar and copied with their source jobs. Conservative histories are namespaced within a class-term, use only replayed submissions and share the copy's shift. These copies are not independently trained predictors or independent workload samples.

For the exact protocol, a copy means the same class-term, replication round and shift. Its own submission outcomes follow policy-specific completions. Other classes and terms are exogenous history in that copy's original relative clock: when the target copy shifts by s_i , an external result is interpreted at $\text{done}_j + s_i$, not at another copy's independent shift. This preserves earlier completed history without claiming a single platform whose copied users share dynamically changing state. A separate sensitivity removes entire records from other replayed classes in the same term, including arrival counts and elapsed time since arrival; it then repeats same-copy refinement. Ordinary outcome withholding preserves already-observed arrivals. The event order releases positive duration completions before reads at the same instant, and zero-duration outcomes after those reads. These conventions are specified in docs/policy_consistent_visibility.md and implemented by src/spjf_guard/features/refine.py.

In the package main and visibility tables, each overlay's deadline p99 is first computed over its eligible jobs; its gap is then computed against that overlay's FCFS and SJF. Reported p99 and gap are separate arithmetic means over overlays. Consequently the

displayed mean waits need not algebraically recover the displayed mean gap. Harm and maximum excess are maxima over overlays. Firing is queue-weighted within a cell, then averaged across overlays. Each bootstrap draw uses the same week multiplicities for every policy and overlay, recomputes weighted job quantiles and cell-level ratios, and averages the ratios. No table uses an average of weekly p99 values. The capacity-sweep gap instead uses the ratio of averaged overlay quantiles, as explicitly identified in Section S6.

The week-block intervals of the main tables hold the 40 class-terms fixed and resample only which weeks the queue sees. Table S18 resamples the class-terms themselves. Each of 100 draws takes the development class-terms with replacement, reruns the copy-count probe and the week set on the draw, builds five overlays with fresh week shifts, and simulates FCFS, SJF, SPJF-E and the three selected guards at the three loads on the original clock. The predictor and the guard parameters stay fixed. A draw of the development pool itself reproduces the development tables cell for cell. The class-term intervals are several times as wide as the week-block ones. At the busiest load Guard(600) closes 0.801 of the gap with interval [0.328, 0.913], against [0.764, 0.842] from week blocks, and its paired difference to unguarded SPJF-E, -0.115 , has interval $[-0.320, -0.046]$. At the two higher loads the difference excludes zero for every G. At the lightest load the difference of Guard(1200) has interval $[-0.015, 0.001]$, which contains zero. With 100 draws each end of an interval rests on two or three draws, so the class-term intervals are coarse.

Table S18. Two sampling units for the development comparison, original clock: whole weeks (the intervals of the main tables) and whole class-terms, redrawn with replacement and rebuilt into five overlays each time. Levels 0, 1 and 2 are the loads $\rho = 0.5, 0.8$ and 1.0 . Width is the length of the interval.

Level	Gap closed of	Point	Week blocks: width [95%]	Class-terms: width [95%]
0	SPJF-E	0.754	0.052 [0.725, 0.776]	0.210 [0.698, 0.908]
0	Guard(300)	0.705	0.046 [0.678, 0.724]	0.280 [0.577, 0.857]
0	Guard(600)	0.740	0.047 [0.713, 0.760]	0.201 [0.693, 0.894]
0	Guard(1200)	0.750	0.050 [0.723, 0.772]	0.207 [0.695, 0.902]
0	Guard(300) – SPJF-E	-0.049	0.026 [-0.061, -0.036]	0.262 [-0.289, -0.027]
0	Guard(600) – SPJF-E	-0.014	0.012 [-0.020, -0.008]	0.032 [-0.032, -0.000]
0	Guard(1200) – SPJF-E	-0.004	0.006 [-0.007, -0.000]	0.016 [-0.015, 0.001]
1	SPJF-E	0.885	0.037 [0.862, 0.899]	0.191 [0.762, 0.953]
1	Guard(300)	0.738	0.064 [0.697, 0.761]	0.733 [0.107, 0.840]
1	Guard(600)	0.837	0.036 [0.815, 0.851]	0.281 [0.634, 0.915]
1	Guard(1200)	0.850	0.035 [0.829, 0.865]	0.245 [0.687, 0.932]
1	Guard(300) – SPJF-E	-0.147	0.067 [-0.189, -0.122]	0.607 [-0.675, -0.069]
1	Guard(600) – SPJF-E	-0.049	0.020 [-0.058, -0.038]	0.129 [-0.141, -0.012]
1	Guard(1200) – SPJF-E	-0.035	0.013 [-0.041, -0.028]	0.058 [-0.072, -0.014]
2	SPJF-E	0.916	0.037 [0.892, 0.930]	0.267 [0.704, 0.971]
2	Guard(300)	0.478	0.214 [0.387, 0.602]	0.708 [0.066, 0.774]
2	Guard(600)	0.801	0.078 [0.764, 0.842]	0.585 [0.328, 0.913]
2	Guard(1200)	0.844	0.053 [0.817, 0.870]	0.393 [0.545, 0.938]
2	Guard(300) – SPJF-E	-0.438	0.213 [-0.526, -0.314]	0.538 [-0.714, -0.176]
2	Guard(600) – SPJF-E	-0.115	0.065 [-0.142, -0.077]	0.274 [-0.320, -0.046]
2	Guard(1200) – SPJF-E	-0.072	0.031 [-0.083, -0.052]	0.118 [-0.150, -0.032]

S5.4. Which Results the Package Reproduces

The package produces the main CodeBench ranking, guard, adversarial and residual tables from `outputs/dev_tables/`, including its separate single-server directory; predictor metrics from `outputs/dev_predictor/`; and visibility comparisons from `outputs/dev_visibility/`. Guard selection and aging selection are in `outputs/selection_v4/` and `outputs/selection_aging/`. Their manifests identify inputs, settings and output hashes.

The preserved original recipe is `configs/main_original_84932d9.yaml`; the original outputs are not claimed to have been regenerated under an unfinished visibility revision.

Earlier predictor-family comparisons and source-clock feature sensitivities are archived under `evidence/codebench_service_v2/` and `evidence/predictor_neural/`. The refit-control and GRU rows in the main ranking table come from `evidence/main_v3/`; the ranking-loss table uses its own fits in `evidence/ranking_score/`. Cross-domain rows combine the package CodeBench row with `evidence/cross_domain/`, `evidence/accoding_v2/`, `evidence/firefox_ci/` and the LPC-EGEE study below. CI chronological-refit validation is in `evidence/firefox_ci_holdout/`. The capacity and physical experiments are separate analyses in `evidence/weakness1_attack/`.

The trace-construction sensitivity table (Table S15) uses the earlier source-clock M4 log-scale score, overlays 0–2, and a fixed completed-work budget $B = 600$ s. This is neither a 600-second promise nor the selected age-relative Guard(600). It supports the earlier ordering result’s sensitivity to term inclusion, the alternate timestamp interpretation, and merging or dropping co-ending runs. It does not test the final expected-cost guard, conservative history or policy-specific refinement under those altered inputs. Numerical agreement of the exact-arithmetic package and its independent reference is also distinct from the archived earlier-pipeline validation.

The sealed procedure is specified by `docs/sealed_run_procedure.md`. It produces separate multiserver, single-server, predictor and visibility outputs, including the policy-specific exact stage, with unchanged declared selection rules and parameters. It does not extend the earlier predictor-family or cross-domain studies. Its results are in Section S10.

S5.5. The Selection Rule and the Cost per Event

The selection rule is stated here in full, with the data structures that make the guard’s firing test cost $O(\log n)$ per event, which is the figure Section 5.2 of the manuscript quotes.

The rule is fixed before the grids are run: three pre-stated grids, one per shape, 258 points in all; a point is feasible when its harm is at most $G/2$ in every one of the 15 validation cells; the objective is the deadline-window gap closed in the worst cell, with ties going to smaller worst harm and then to smaller B_0, η, γ . Guard(G) is the winner over the three grids together; the winner inside each grid is reported as an ablation row.

Adding each completed cost to every waiting job’s account would cost time linear in the queue at every completion. We keep $\text{over}[q] = TC - F(q)$ instead, where TC is the total completed work and $F(q)$ the prefix sum over ranks up to q of completed work, maintained in a Fenwick tree, so that a completion is one point update and a query one prefix sum. The firing test is a segment tree over the window of waiting ranks supporting suffix range-add and the query “leftmost leaf with value at least T ”, after rewriting $\text{over}[q](t) \geq B_{0,q} + \eta k(t - a_q)$, where $B_{0,q} = B_0 + \gamma n_q$ is fixed at q ’s arrival, with $\eta k = e_n/e_d$ in lowest terms as $e_n a_q - e_d B_{0,q} - e_d F(q) \geq e_n t - e_d TC$, whose left side is a per-job value changed only by range-add at completions and whose right side depends on t alone. The capped channel needs only the head, since at a fixed threshold over is non-increasing in rank. All budget arithmetic is carried in integer microseconds, so the test is exact and no floating-point clock enters the schedule. Each event costs $O(\log n)$ in the guard’s own maintenance, exclusive of what the base policy spends producing a proposal, and n counts waiting jobs, with arrivals sharing a timestamp processed in rank order.

S5.6. What the Selection Was Run On

What the released package regenerates and what remains archived development evidence are recorded here, behind the statements on selection in Section 7.3 of the manuscript.

The grids, the feasibility rule, the objective and the tie-break were fixed before the grids were run, at comparable density across the three shapes, and the package comparisons use exact integer microseconds for arrivals, service and budget arithmetic. Earlier predictor comparisons, cross-domain analyses and trace-construction sensitivities remain archived development evidence, with their different fits and configurations identified in the captions and this section. They are not all regenerated by the package. Two properties of the outcome belong here. The winning shape is not the same at every promise: the age-relative shape wins the joint comparison at $G = 300$ and 600 s and the queue-length shape at $G = 1200$ s, where it reaches worst-cell gap 0.7275 at worst-cell harm 152.2 s against 482.2 s for the best age-relative point. And a worst-cell rule can be decided by a single cell: dropping the one validation cell that decides the $G = 600$ choice moves that choice to $B_0 = 15k/4$ s, $\eta = 0.90$, with worst-cell gap 0.7237. The sealed terms were opened once after the protocol freeze, with the parameters unchanged, and Section S10 gives their tables. The frozen-run procedure also produced a separate single-server trace and its tables, per-term and pooled predictor metrics, conservative/static visibility tables and exposure/lag audits, the policy-specific exact-visibility stage and the online replay. It does not produce the refit-control and GRU rows of Table S16, the other predictor-family comparisons, cross-domain sealed results, or the earlier trace-construction sensitivities.

S5.7. The Compared Policies in Full

Table 2 of the manuscript gives each compared policy a one-line role. What each one is there for, at length:

- SJF sorts waiting jobs by their true service times. It is an undeployable reference and not an optimum, and it is the denominator of the fraction of the gap closed.
- SPJF-E carries prediction quality through to waiting time, unguarded. SPJF-log is the same pipeline fitted on the log scale, and the other candidate scores of Section 5.1 of the manuscript run through the same simulator.
- Guard(G) takes the budget shape the selection rule of Table S5 picks over all three grids together. Guard-fixed(G), Guard-age(G) and Guard-queue(G) take the shape the same rule picks inside the constant, age-relative and queue-length grid respectively; they isolate what the shape changes at a fixed cap, and one of the three coincides with Guard(G) at each promise.
- Fixed(G) carries the promise G but not the harm constraint, so the selection rule rejects it as infeasible. It is reported as the comparator that uses the whole cap.
- Skip(G) counts positions rather than work, in the finite-skip family, at $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$, the largest count that still promises G under dispatch-time charging (Section S4.4 here).
- The corrupted scores are a reversed order, a random order and a score that calls the truly longest 1% the shortest; they test the guard where the base policy is worst. The counterexample traces are retained from two audits and adversarial review, and every job in them must satisfy the bounds of Section 6 of the manuscript.

S6. Capacity, Physical Queueing and Feedback

These development experiments address distinct questions. The capacity sweep changes the size of the constructed pool while preserving its complete offered stream. The physical experiment checks an executing timed-work service on a selected window. Neither identifies a historical queue on CodeBench or a causal deployment effect. The scores are the archived embedded Tweedie predictions, not the later package refit, conservative score or exact protocol.

S6.1. Capacity Sensitivity on Complete Traces

The declared common grid uses every integer $k = 1, \dots, 20$ on all five development overlays; overlay 0 alone continues through $k = 76$. This gives 156 capacity cells. Each replay starts at the beginning of its complete trace, keeps queue state across weeks and drains all jobs; there is no time compression, window reset or thinning. The same source jobs occur in all overlays, so these are alternative constructions rather than independent deployments. The grid and comparison family were declared after a pilot on the first overlay, before the full sweep, not before inspecting the development demand.

Table S19 reports the average of the overlay-specific deadline p99 values, with point-wise intervals. Gap closure in this study is the ratio of differences of those averaged quantiles. This differs from the package main tables, which average overlay-specific gap ratios. Paired calendar-week resampling uses 2,000 common draws, recomputing quantiles from the weighted job distribution, not averaging weekly quantiles. The declared simultaneous band covers 80 absolute FCFS-minus-policy differences: each capacity and each non-oracle policy. It uses a centred coordinate-standardised maximum statistic. These intervals resample realised outcomes, not queue trajectories; they omit source-term selection, predictor fitting, shared source jobs, model mismatch, feedback and cross-week state dependence. Maxima below are trace maxima, not population confidence bounds.

Table S19. Capacity sweep with archived scores: mean overlay deadline p99 in seconds, with point-wise paired calendar-week intervals. Negative effects are retained.

k	FCFS	SPJF-E	Guard(600)
1	108,345.31 [92,267.38, 114,125.36]	130,231.12 [65,565.87, 155,632.22]	108,839.10 [92,739.31, 114,582.34]
2	10,362.62 [7,565.14, 11,503.45]	1,844.21 [766.70, 3,990.37]	10,618.06 [7,820.32, 11,755.23]
3	1,056.40 [688.91, 1,273.01]	138.86 [105.79, 169.05]	934.68 [452.24, 1,253.73]
4	273.50 [197.09, 352.10]	62.92 [53.69, 72.13]	89.61 [65.94, 114.14]
5	119.35 [92.91, 143.43]	43.39 [37.94, 47.54]	47.44 [40.46, 53.39]
8	24.43 [18.46, 29.58]	13.21 [9.58, 15.80]	13.36 [9.67, 16.05]
14	0.84 [0.45, 1.50]	0.47 [0.30, 0.73]	0.47 [0.30, 0.73]
20	0.00 [0.00, 0.04]	0.00 [0.00, 0.03]	0.00 [0.00, 0.03]

The negative cells are material. At $k = 1$, FCFS-minus-Guard p99 differences are -208.61, -493.79 and -1,071.72 s for the 300, 600 and 1200 s promises; all simultaneous bands are wholly negative. At $k = 2$ all guard point estimates remain negative. Positive simultaneous lower bounds occur for SPJF-E at $k = 2$ –12, Guard(300) and Guard(600) at $k = 4$ –12, and Guard(1200) at $k = 3$ –12. At $k = 14$ the guard point improvements are about 0.37 s but their simultaneous bands cross zero; at $k = 20$ the point differences are zero. A correct per-job bound does not imply positive tail benefit across queueing capacities.

For Guard(600) at $k = 4$ the maximum excess is 515.84 s and harm is 339.06 s, although it closes 0.8000 [0.7620, 0.8388] of the reference gap. At $k = 20$ that ratio is undefined because the reference gap is not positive, while maximum excess and harm remain 124.38 and 37.37 s. On overlay 0, FCFS still has one positive wait at $k = 75$; at $k = 76$ every policy has zero wait. The latter is the exact maximum infinite-server occupancy, hence a zero-wait endpoint for this overlay, not a capacity recommendation for a deployment.

S6.2. An Executing Timed-Work Service

The physical selection rule chooses the complete calendar-aligned 300-second bin with the greatest offered capped work in development overlay 0, breaking ties by earliest time. Its 2,151 jobs carry 4,118.215007 s of requested work, 6.864 times the two-worker release-window capacity. Each policy starts empty and drains its own queue. Inter-arrival spacing, requested costs and scores are retained; workers hold their slots with timed sleeps.

This tests queueing and the control rule, not the portability of submitted-program execution time. It also discards inherited full-trace backlog, so its percentile is not the capacity sweep's percentile.

The nominal payload cap is 60 s; the measured-service premise $L = 61$ s was fixed before execution. Guard parameters are $B_0 = 30$ s, $\eta = 0.5$, $\gamma = 0$ and $B_{\max} = 356$ s, corresponding to the nominal 300-second ideal-model promise. A measured holding time above L would fail the premise, not be clipped or used to raise L retrospectively. None did. Actual enqueue, choice, dispatch, worker start/finish and completion receipt are recorded separately; service becomes known only at completion receipt.

Table S20. Physical timed-work runs. The ideal comparator uses each run's own measured arrivals and holding costs; all entries are seconds.

Policy	Mean wait	p99 wait	Max absolute per-job ideal discrepancy
FCFS	1,337.05	1,763.30	0.49
SPJF-E	142.79	1,687.47	392.84
Guard(300)	480.39	1,783.65	1,354.10

All 6,453 observed-prefix decisions passed an independent choice audit. Physical and ideal measured-input Guard firing counts were 322 and 372. The small mean discrepancies do not establish job-level fidelity: the maximum absolute per-job error is 392.84 s for SPJF-E and 1,354.10 s for Guard. Guard's physical p99 exceeds the separate physical FCFS run by 20.34 s. Thus the physical window confirms an executing queue and rule conformance but does not show a guarded p99 gain. Actual arrivals and holding costs differ between physical runs; their comparison is descriptive. One selected window supplies no across-window confidence interval.

The guard's theorem reference is shadow FCFS on that guard run's measured arrivals and service, not the separately timed physical FCFS run. An observed-trace certificate augments the ideal bound by cumulative idle worker-time and dispatch-to-start delay. Its largest correction is 1.127 s; all 2,151 jobs pass, and the largest uncorrected excess over shadow FCFS is 222.21 s. This is an ex post certificate conditional on the measured trace and the 61 s service premise. It does not bound future operating-system stalls and does not restore an advance uncorrected real-time promise.

S6.3. Why Open-Loop Benefit Has No Known Bias Direction

The replay compares policies conditional on fixed offered work. Feitelson states the objection at its strongest, that "logged workloads actually contain a 'signature' of the logged system" [16]: a trace-driven simulation issues requests irrespective of system state [17], so faster replies cannot induce more submissions, change their costs or alter which jobs enter a deadline-window quantile. Shmueli and Feitelson also distinguish increased productivity from improvement in conventional waiting metrics [17]. The two constructions below show that the bias from the missing feedback has no known direction.

In an exact single-server feedback construction, jobs of length 10 and 1 s arrive together, the long job first, and the short job's user submits a second unit job 0.5 s after completion. Replaying the FCFS-induced arrivals gives SJF a mean-wait advantage of 3.00 s; letting each policy generate its own successor arrival changes the advantage to -0.167 s. The corresponding linearly interpolated p99 advantages are 8.82 and 0.47 s. The job cohort is identical, so this example does not depend on comparing different populations.

The opposite bias direction is possible too. An initial FCFS order with services 20, 1, 3 and 10 s, whose unit-job user returns after 2 s with a 0.1 s successor, gives open- and closed-loop mean advantages of 9.20 and 11.16 s and p99 advantages of 10.00 and 10.18 s. These exact recursions falsify a general lower-bound interpretation; they are not fitted be-

havioural models. The pathwise theorem still compares a guarded schedule with shadow FCFS on that schedule's realised arrivals and services, not independently evolving closed-loop systems [17].

S6.4. LPC-EGEE: Recorded Queueing with Weak Counterfactual Validation

The public LPC-EGEE trace supplies recorded waits and concentrated execution costs together [18]. After outage exclusion and queue-limit capping, the longest 1% of 206,222 retained jobs carries 40.7% of work. On the chronological evaluation part the top share is 35.1%, which is the value Table S24 uses. The physical pools stay disjoint. Effective capacities 83 and 56 are fitted only on the earlier part, with no test-driven refit. Service is the minimum of recorded run time and its documented queue limit; the common bound uses the largest pool limit, 259,200 s.

The fitted FCFS approximation underestimates evaluation-part mean waits by 24% and 39%, with hourly correlations of 0.44 and 0.40. The archive describes plain FCFS serial-job ordering during the logged period, but queue concurrency ceilings, polling and middleware are absent from the work-conserving replay. Recorded waits therefore cannot be equated with the theoretical FCFS counterfactual. This is weak simulator validation, not evidence that the original scheduler was an arbitrary non-FCFS policy.

In the conditional replay, SPJF-E reduces mean wait by 21.4% [13.2%, 38.2%], but its busy-hour p99 reduction of 4.2% [-11.2%, 46.0%] remains unresolved. Busy hours use frozen arrival-count thresholds, not deadlines. All six constant/age work-budget guards at the tested multiples of L fire zero times; a small fraction of the numerical bound being spent is therefore not evidence of effective protection. Histories use recorded completion times, and 2,906 evaluation arrivals (2.5%) have an outcome unavailable under replayed SPJF-E. This row is a recorded-feature conditional counterfactual, not a closed-loop deployment result.

S6.5. Closed-Loop Replay with the Recorded Think Times

Section S6.3 leaves the direction of the missing feedback unidentified. On the main judging trace one component of that feedback is recorded, and this subsection replays the development overlays a second time with it.

The logs identify each submission's author and its position in that student's sequence, so for two consecutive submissions of the same student-term the offset $\delta_i = a_i - (a_{\text{pred}} + C_{\text{pred}})$ between the previous result and the next submission is in the data. It is non-negative for 98.38% of the 17,566,120 consecutive pairs, and for 97.17% of the 4,217,092 pairs inside deadline windows, so on the header reading of Section 7 of the manuscript the next submission almost always follows the previous result. Its median is 123.5 s over all pairs and 109.0 s inside windows, with 11.7 s at the tenth percentile and 35.0 s at the twenty-fifth, and it carries no measurable dependence on the previous job's predicted cost: the Spearman correlation is 0.015 over 2,000,000 sampled result-gated pairs. The chains are 68,640 in number, 1,560 real student-terms replicated 44 times across 17,634,760 jobs.

The closed rule releases a submission at its predecessor's completion in the replay plus that pair's recorded δ_i when $\delta_i \geq 0$, and preserves the recorded inter-submission gap otherwise; the open rule is the recorded instants the rest of the paper uses. With gating switched off, the closed kernel reproduces the package simulator job for job, at a maximum absolute wait difference of 0 microseconds over four policies at three load levels, and its open-loop rows reproduce the main development cells to every printed digit.

The ordering of the policies survives in all nine cells and under both edge treatments, and the scale does not: the deadline-window p99 falls by a factor of 1.3 to 3.8 and the

Table S21. Open-loop against think-time-preserving replay on the development overlays: deadline-window p99 in seconds, mean over overlays 0–2, and the fraction of the FCFS-to-SJF gap each policy closes. “Late kept” and “late dropped” are the two treatments of submissions that gating pushes past their own assignment’s end. SPJF-E here is the stored expected-cost score read on the original result clock, the variant the main development cells use, and not the online headline of Section 8.1 of the manuscript; these rows are therefore a sensitivity and not a restatement of that headline.

ρ	k	Policy	Open p99	Closed p99, kept	Closed p99, dropped	Gap, open	Gap, closed
0.5	8	FCFS	23.405	8.991	9.006	—	—
0.5	8	SJF	8.788	4.897	4.914	1.000	1.000
0.5	8	SPJF-E	12.542	5.588	5.564	0.743	0.831
0.5	8	Guard(600)	12.669	5.609	5.595	0.734	0.825
0.8	5	FCFS	113.770	42.149	42.221	—	—
0.8	5	SJF	33.251	23.187	23.342	1.000	1.000
0.8	5	SPJF-E	43.007	26.474	26.326	0.879	0.827
0.8	5	Guard(600)	47.153	26.810	26.285	0.827	0.809
1.0	4	FCFS	268.542	71.400	71.674	—	—
1.0	4	SJF	43.590	32.884	32.496	1.000	1.000
1.0	4	SPJF-E	62.475	38.775	38.563	0.916	0.847
1.0	4	Guard(600)	88.486	40.200	39.787	0.800	0.810

mean wait by 1.5 to 3.9. A queue that builds up holds back the submissions that would have deepened it, so the realised busiest-hour utilisation under FCFS drops from 0.505, 0.807 and 1.009 to 0.450, 0.745 and 0.942, and the heaviest level is no longer overloaded. The baseline gains most from this, because its own longer waits delay its users more: at the heaviest level the mean arrival drift is 440.0 s under FCFS against 171.5 s under SPJF-E and 142.3 s under SJF. The absolute gap therefore narrows while the fraction of it a policy closes does not, which is why the closed columns of Table S21 move in both directions and stay inside 0.81–0.85.

What changes qualitatively is the price of the promise, not its value. Gating shortens the queues, so the guard fires far less often: the queue-weighted firing rate of Guard(600) falls from 0.113% to 0.0025% at the lightest level, from 1.314% to 0.134% at the middle one and from 7.521% to 0.272% at the heaviest. Its deadline-window p99 penalty over unguarded SPJF-E falls with it, from 41.6% to 3.7% at the heaviest level and from 9.6% to 1.3% at the middle one. The worst wait it admits is unchanged in kind: 1,074.3 s under gating, where SPJF-E reaches 5,783.2 s and SJF 4,234.3 s, so starvation of long jobs remains a property of the ranking and not of the arrival process. The per-job guarantee is unaffected. It is checked against FCFS run open-loop on the guarded policy’s own realised arrival sequence, which is the counterfactual Section 3 of the manuscript defines, and it holds on every job of every run: zero violations in 18 guarded runs of 17,634,760 jobs each, with the worst job using 80 to 90 % of its bound under gating against 87 to 95 % without it. Refusing the submissions that gating pushes past their own assignment’s end removes 0.27 to 1.93 % of the offered work and moves the p99 by at most 1.3 %.

A paired week-block bootstrap on overlay 0, 400 resamples of whole weeks with replacement, puts the closed-minus-open difference in the gap closed at

ρ	SPJF-E	Guard(600)
0.5	+0.092 [0.000, 0.174]	+0.123 [0.043, 0.193]
0.8	−0.071 [−0.108, −0.025]	−0.025 [−0.077, 0.036]
1.0	−0.055 [−0.089, −0.007]	+0.010 [−0.037, 0.063]

so the movement is resolved in four of the six comparisons and is small in every one. This is a reduction of the scheme Section S5.3 describes, which uses 2,000 resamples over five overlays, and it is a cost decision rather than a methodological one.

Four limits stand. The think times were recorded on a platform that gave each student a private container, so every δ in the trace was produced by a student who never waited, and the replay assumes δ is invariant to the wait; nothing in this trace separates impatience from disengagement. The model has no abandonment and no content feedback, so it does not address the productivity distinction Shmueli and Feitelson draw [17], which Section S6.3 states. The chains are 44 copies of 1,560 real student-terms, so the effective sample for anything driven by user behaviour is 1,560 and the week-block bootstrap, which resamples time, does not cover that. And the comparison stops being pathwise: each policy generates its own arrival sequence, so the closed columns compare different sample paths over the same job population, although the per-job theorem, stated against FCFS on the realised sequence, still applies job by job.

S6.6. LPC-EGEE per Walltime Class: A Limit Class Where the Guard Fires

Section S6.4 pools the six documented walltime classes of the LPC-EGEE log and uses the largest limit, 259,200 s, as L . Reading each class as its own pool, with that class's own limit, changes the promise by two and a half orders of magnitude: on the 900-second class the work-budget guard fires and comes within 0.7% of its bound, where the pooled reading of the same log never fires and spends 0.046 of it. It does not make the first applicability condition robust on this trace.

The class pool is a construction of ours, and measurably so. The SWF header names three machines, not the two the pooled reading used: separating `clrglop195`, which owns no held-out arrival, leaves CE1 with 111,236 jobs and CE2 with 60,519, and CE2 alone then maxes out at exactly 56 simultaneous jobs, which is its documented processor count and resolves the earlier audit's unexplained 58. Within each partition the per-class maxima of simultaneously running jobs sum to 337 against 84 processors on CE1 and to 203 against 56 on CE2, so the classes demonstrably shared their partition's machines and a per-class pool is a constructed pool on the same footing as the others in this paper.

Table S22. LPC-EGEE by walltime class, both partitions pooled. Recorded p99 is over all retained arrivals, as is the pooled row's tail share, which is the figure Section S6.4 reports; the per-class tail shares and the FCFS counterfactual are on the held-out part, at the capacity fitted on the earlier part. The batch class has no arrival that both submitted and completed before the cutoff, so no capacity can be fitted for it; the day class has fewer fitting rows than the pre-set minimum and its simulation diverges, so its counterfactual is an artefact and is marked.

Class	L (s)	Jobs	Held-out	Recorded p99/ L	FCFS p99/ L	Tail share	Cond. 1?
test	900	41,143	34,707	23.885	13.763	0.213	yes
batch	3,600	1,506	1,506	0.795	—	—	—
short	7,200	69,446	19,353	0.653	0.000	0.240	no
long	86,400	30,602	21,235	0.164	0.244	0.274	no
day	129,600	18,594	18,236	0.065	30.970	0.161	diverged
infinite	259,200	44,931	21,773	0.063	0.139	0.180	no
pooled	259,200	206,222	116,810	0.046	0.042	0.407	no

The 900-second test class is the one cell that behaves differently. Its recorded p99 queue wait is 23.885 times its own limit over the whole log, and 7.573 and 18.675 times on CE1 and CE2 over the held-out part; no other class approaches it, and pooled over all six the same ratio is 0.046. Cost concentration inside a capped class is correspondingly weaker than pooled: the longest 1% of the class carry 0.213 of its held-out executed work,

0.167 on CE1 and 0.350 on CE2, against 0.407 when the six classes are pooled, so part of the heavy tail of a mixed pool is the mixing of limits itself. The predictor, refitted per class on the earlier part with the package defaults, reaches a held-out AUROC of 0.746 on the test class, 0.846 on short, 0.792 on infinite and 0.737 on long.

Table S23. Replay of the 900-second test class at its own limit and its fitted capacity $k = 1$ on each partition, held-out arrivals only. Mean waits are in seconds; intervals are 2,000 paired resamples of 17 arrival-week blocks. The busy-hour p99 gap is not resolved for any policy.

Policy	Mean	Mean gap closed	Busy p99	Busy p99 gap	Max excess (s)	Firing
FCFS	410.2	—	13,412.76	—	—	—
SJF (oracle)	245.7	1.000	8,340.64	1.000	112,805	—
SPJF-E	298.7	0.678 [0.622, 0.757]	9,967.52	0.679 [−3.109, 0.920]	77,467	—
Constant-3.5L	316.6	0.569 [0.534, 0.630]	13,933.44	−0.103 [−0.542, 0.560]	3,129	1.42%
Age-3.5L	314.9	0.579 [0.548, 0.641]	13,933.44	−0.103 [−0.542, 0.572]	3,082	1.64%
Constant-5L	310.3	0.607 [0.577, 0.664]	13,478.56	−0.013 [−0.593, 0.550]	4,444	1.04%
Age-5L	311.1	0.602 [0.575, 0.647]	13,478.56	−0.013 [−0.584, 0.571]	4,338	1.46%
Constant-10L	304.6	0.642 [0.591, 0.710]	13,965.24	−0.109 [−2.812, 0.620]	8,726	0.67%
Age-10L	304.1	0.645 [0.598, 0.711]	13,940.20	−0.104 [−2.507, 0.622]	8,765	0.77%

The guard fires at 1.42% of dispatch epochs at the tightest promise and its per-job bound is nearly attained: the largest realised excess over the FCFS wait is 3,129 s against an allowance of 3,150 s, a ratio of 0.9933, where the pooled reading of Section S6.4 reached 0.046. Over 1,021,494 job-policy checks there is no violation. The trade-off is visible in one line: the guard gives up 0.11 of the mean gap the unguarded score closes and cuts the worst excess from 77,467 s to 3,129 s, with harm of 25 s under both. The busy-hour p99 is not improved by any guarded policy, every interval spanning zero.

Four things keep this from being evidence of deployment benefit. The capacity that makes the first condition pass is the boundary value $k = 1$ of the search grid, chosen on 7.6 days of pre-cutoff data on CE1 and 14.2 on CE2, and at the class's own observed concurrency ceiling the held-out FCFS p99 is exactly zero on both partitions, so the condition fails there; on CE2 it also fails the rule-of-thumb 3L form at the fitted capacity, the FCFS p99 being 2.448L. The held-out wait profile is not reproduced: the per-class simulator is 2.9 times high on CE1 and 7.3 times low on CE2 on the mean, with hourly correlations indistinguishable from zero, and for the 7,200-second class it produces identically zero waits against a recorded p99 of 14,288.96 s. The mechanism is the one the concurrency sums show: a short-limit job's recorded wait at this site is largely time spent behind other classes' long jobs on shared machines, which a per-class pool deletes by construction. And the search that found this cell covered eleven cells, two capacity readings, three promises and two budget shapes, which the intervals do not cover.

S6.7. Footnotes to the Cross-Trace Table

Table 9 of the manuscript points here for the fitted capacities of its two continuous-integration rows and for the LPC-EGEE grid log [18], whose two rows Table S24 gives in the same format. LPC-EGEE records both waits and concentrated costs, but its fitted FCFS approximation underestimates the mean waits of the chronological test, and the one reading on which a guard fires is a per-class pool we impose ourselves. Both rows are therefore conditional counterfactuals and not estimates of deployment benefit; Sections S6.4 and S6.6 give the two runs. Each paragraph below says what a row does and does not support.

Table S24. The LPC-EGEE rows of the cross-trace comparison, in the format of Table 9 of the manuscript. Tail share: fraction of executed work carried by the longest 1% of jobs. Screen 1: whether the FCFS p99 exceeds $(3 - 2/k)L$. Excess in units of each row's L . The guard entry of the second row is its mean-wait gap at $G = 3.5L$ (see below).

LPC-EGEE reading	Tail share	AUROC	FCFS p99/ L	Screen 1	Gap closed, SPJF-E	Unguarded max excess	Guard at $G \approx 5L$ gap closed / max excess
Chronological test	0.351	0.892	0.042	no	0.193	0.162 L	0.193 / 0.162 L
test class, per-class pool	0.213	0.746	13.76	yes	0.679	86.07 L	0.569 / 3.48 L

The two continuous-integration capacities.

The CI capacities are fitted, not recorded, and the two scheduling columns and the FCFS p99 of those rows exist only at the whole-window fit ($k_{\text{eff}} = 156$ of 173 and 96 of 105), so they are printed at it. What supports them is the held-out refit of Section 7.2 of the manuscript, which recovers the same capacity to within a few machines on chronological splits of each window and reproduces the recorded waits on the untouched part, with the p90 the one quantile that does not survive. The tail share and the AUROC do not depend on the capacity.

The LPC-EGEE chronological test.

LPC-EGEE [18] uses disjoint pools with 83 and 56 servers fitted on the earlier part, and full chronological test wait-profile validation is weak. L is the largest documented queue limit. The p99 and the gaps of that row use frozen-threshold busy hours, not deadlines, and history follows recorded completions. The displayed age-relative guard at $G = 5L$ never fires. This conditional FCFS counterfactual is not evidence of real-scheduler improvement; Section S6.4 here gives the run.

The imposed test-class pool.

The test class is a pool we impose on machines the six queues demonstrably shared, at the boundary capacity $k = 1$ of the search grid on each partition, and its held-out wait profile is not reproduced. Condition 1 holds on both partitions' recorded waits and on the pooled FCFS counterfactual, and fails at the $3L$ form on one partition and at the observed concurrency ceiling on both. The 0.679 is the unguarded score's busy-hour p99 gap, whose interval spans zero, and the guard's own is -0.103 , so the last column of that row reports its resolved mean gap, 0.569 [0.534, 0.630], at $G = 3.5L$ rather than the $5L$ of the other rows, where its largest excess is 3,129 s of an allowed 3,150 s (Section S6.6).

S6.8. What Failing an Applicability Condition Does Not Establish

Section 8.6 of the manuscript names two screens, which this file calls applicability conditions, and gives the numbers trace by trace. What a failure does and does not support is here.

Under the first condition, the four failing workloads have a first-come first-served p99 wait between 0.021 and 1.78 multiples of L , against the additive term of about $3L$ in the certificate. The Intel Netbatch compute farm [3,19] is the extreme. Its service cap is 5.7 days against a worst-window p99 wait of 18.7 hours, so the uniform additive term is about 17 days, the guard never fires at any positive budget tested, and predicted-cost ordering makes the p99 wait worse than first-come first-served there under sustained overload. That does not establish impossibility below the term for a restricted base policy or workload, and the guard results the manuscript reports on those four workloads measure the ordering mechanism there.

The second condition is an empirical opportunity screen. It is not a theorem assumption and not a necessary condition for a benefit. On the two Firefox CI hardware pools

[4,5] the longest 1% of jobs carry 3.4% and 5.2% of the work, at coefficients of variation 0.61 and 0.87, and there even true-size ordering makes the p99 wait worse than first-come first-served on three of the four windows we ran.

S6.9. The Guard on a Course Platform's Own Two-Worker Back End

The timed-work experiment of Section S6.2 ran the guard on a service whose jobs were sleeps of known length. This run puts it on a real one. The service is the back end of an online course platform whose jobs are calls to a language model: a short call that writes a learner report and a long one that writes a study plan. Two worker slots serve one queue. The server stops a job at an enforced limit, 25 s for a report and 170 s for a plan; a stopped job holds its slot until the model call has returned, so the executed work of a stopped job is the limit itself and not a number read off afterwards. Dispatch, the guard and the limit were added to the platform's own code and run locally on the authors' machine; nothing was deployed, and no human learner was in the loop. The demand came from 20 scripted accounts. Each account submitted in the class pattern of the platform's own users, six reports for every two plans, in a burst shaped like the deadline hours of the main trace, and waited for each result before submitting again; the think time was drawn per seat from one seed, so it was the same in every cell. The promise was $G = 400$ s with $k = 2$, budget shape $B_0 = 30$ s and $\eta = 0.5$, the shape of the timed-work experiment and not a new selection. Each cell ran 100 jobs.

Table S25. Six cells on the platform back end. ρ is measured executed work over k times the span. Excess is measured wait minus the wait in an offline first-come first-served replay of that cell's own arrivals and measured service times. The last column is the largest per-job difference between the measured wait and the package simulator fed the same inputs; the reservation cell is not comparable because the simulator has no reservation-and-refund wrapper.

Cell	Policy, demand rule	ρ	Mean wait	p90 wait	Max excess	Excess/ G	Fired	Simulator
1	FCFS, closed	1.000	308.8	479.4	0.17	0.0004	0	167.8 ms
2	SPJF, closed	0.971	227.4	1083.3	1869.9	4.67	0	125.3 ms
3	Guard, completed-work charging, closed	0.994	235.2	680.2	151.7	0.379	24	138.8 ms
4	Guard, reservation charging, closed	0.976	206.7	896.8	205.5	0.514	20	—
5	Guard, completed-work charging, open	0.695	70.3	177.3	184.0	0.460	5	115.5 ms
6	Guard, completed-work charging, closed with slower resubmission	0.993	205.7	433.8	184.8	0.462	21	132.8 ms

The promise held.

Jobs beyond the promise: 0 in cells 1 and 3 to 6, 7 in cell 2. Cell 2 is the unguarded base policy, which promises nothing; its row measures what the guard is for. On the same k and the same promise it let the worst job wait 1869.9 s longer than first-come first-served, 4.67 times G . The same base policy under the guard (cell 3) brought that to 151.7 s, 0.379 G . Against its own replayed first-come first-served run, unguarded SPJF cut the mean wait from 725.2 s to 227.4 s and the guarded policy from 483.7 s to 235.2 s; the cost moved from 20 jobs each waiting 460.2 s longer on average to 32 jobs each waiting 57.8 s longer.

The limit is enforced.

Of the 600 jobs, 29 reached their own limit, 22 plans and 7 reports. The largest amount by which a measured service time exceeded its limit was 16.1 ms. The engine log records 26 of the stops itself; in the other 3 the caller's abort and the engine's deadline fell on the same instant and the abort landed first. Within each class the service times are not heavy-

tailed: the coefficient of variation is 0.20 to 0.69 per class and per cell. The mixture of the two classes has a coefficient of variation above 1, and that is a property of the mixture, not of the platform's jobs.

Demand reacted to waiting.

This is what the timed-work experiment could not measure. Because every account waited for its result before submitting again, the arrival sequence was a function of the policy, and the first-come first-served replay of each cell's own arrivals gave a different load in every cell: a mean wait from 141.9 s in the open-loop cell to 725.2 s in the unguarded cell, whose fast short jobs made the accounts resubmit sooner. Measured ρ was 0.97 to 1.00 in the four closed-loop cells and 0.695 in the open-loop cell, where arrivals followed a fixed timetable. Measured waits therefore cannot be compared across cells; the only clean comparison is each cell against its own replay. Cell 6 made the accounts think longer the longer they had waited, and the mean wait fell from 235.2 s in cell 3 to 205.7 s.

Two ways of charging the budget.

Cell 4 charges the budget by the reservation-and-refund rule of Section S7.6, whose additive constant is lower by L , so under the same promise its cap was 460 s against 120 s for completed-work charging. It let more overtaking through, kept more of the base policy's benefit (mean wait 206.7 s; reports waited 26.9 s on average) and cost the jobs it did delay more (102.6 s each on average against 57.8 s). Neither cell exceeded the promise. The two cells are not a controlled comparison, since their caps differ by rule and their demand differed under the closed loop; they show that both rules run on this stack and held the promise.

The simulator agrees with the service.

Fed each cell's measured arrivals, service times, scores and k , the package simulator reproduced the measured wait of every one of the 500 jobs in the five comparable cells within 167.8 ms. Cell 4 is excluded: the simulator has no reservation-and-refund wrapper, and its 982 s difference on that cell measures the difference between the two mechanisms, not the simulator's accuracy. The dispatcher itself took 10 to 12 μ s per dispatch epoch on average and at most 75 μ s, at measured queue depths of 11 to 18, against service times of seconds to minutes. The six cells took 3 h 27 min of wall clock and at least 832,109 model tokens; jobs stopped at the limit were cut before their usage record arrived, so the true figure is a few percent higher.

What this run does not support.

No human learner took part; the demand was scripted on a local deployment, and the platform's own log holds no learner demand to replay. One hundred jobs per cell give no p99, and the burst shape was borrowed from the main trace, since the platform has no assignment deadlines of its own. The budget shape was not reselected. The size proxy the scores came from is uncalibrated, which the guarantee tolerates and the benefit figures do not. The run is one process on one machine and says nothing about distributed dispatch. And the simulator's agreement here does not undo the timed-work result of Section S6.2, where the guard's per-job difference from its ideal replay reached 1354.10 s on 2151 jobs; one hundred jobs and about twenty firings per cell do not reach that regime.

S7. Additional Theory: Scope, Accounting and Instance-Dependent Bounds

The results below delimit the main-text claims rather than enlarge them. The first example rules out instance optimality for the uniform guard. The exact identity that follows

is a useful job-level accounting decomposition; its core is standard sample-path workload balance, and we do not treat it as a new conservation law. The final bounds replace the global timeout by realised order statistics, but in their actual-size form they are retrospective certificates under completion-only observation.

S7.1. Comparison with the Nearest Guarantee Disciplines

Table S26. Scope of the nearest per-job guarantees and wrapper precedent.

Method	Reference discipline	Arrival assumptions	as-fi	Preemption	Information needed	Server model	Guarantee
This work	Same-input FCFS	Arbitrary finite path		No	Ranks, completions, public opaque base proposal	k identical servers	Deterministic per-job additive waiting excess; wrapper around any base
PFCFS [20]	Flow time with-out later submissions	Online leases	re-	Gang pre-emption	Processor demand; service time need not be known; pre-emption penalty	Rigid jobs	λ -fairness; PFCFS is $(2 + 25)$ -fair with competitive efficiency bounds
Conservative backfilling [21]	Submission-time reservation	Online missions	sub-	No	Requested time and resources	Rigid jobs	Later backfills do not delay any reserved start
PV-EASY [22, 23]	Priority reservation	Online missions; prediction error allowed	sub-	Kill and restart	Runtime estimates and resources	Rigid jobs	Zero lower-priority delay to the protected reservation
Nudge [12]	FCFS	Stationary Poisson		No	Size class at arrival	One server	Response-time stochastic dominance for its model; at most one swap per job
Load ancing Guardrails [24]	Bal-Unconstrained dispatcher	Stochastic heavy-traffic regime		SRPT servers	Size class and patch state	Parallel pools	Any dispatcher becomes mean-response-time optimal in heavy traffic

S7.2. No Uniform Instance-Optimality Ratio

Proposition S6 (An unbounded instance-optimality ratio). *At $k = 1$, let three jobs arrive together at $t = 0$ with costs L , 1 and 1 in arrival order, with $L \geq 2$, and set $G = L$. The cap of the parameter rule of the manuscript (Corollary 8 there, $B_{\max} = k(G - (3 - 2/k)L)$) is $B = 0$, so the guard is FCFS and has total waiting time $2L + 1$. The order $(1, 1, L)$ has total waiting time 3 and satisfies the per-job promise G . Consequently the ratio between the guard's total wait and that of a feasible policy is at least $(2L + 1)/3$, which is unbounded as $L \rightarrow \infty$.*

Proof. FCFS produces waits $0, L, L + 1$. Under the competing order the two short jobs wait 0 and 1, and the long job waits 2. The long job's excess is therefore 2 and both short jobs have nonpositive excess, so $L \geq 2$ makes the order G -feasible. The comparison allows a clairvoyant feasible policy; restricting the comparator to completion-only policies would be a different question. $\square$

The pointwise-maximal budget $B^* = G - L + 1$ of Proposition S19 admits one short job here, giving total wait $L + 2$ and ratio $(L + 2)/3$; the conclusion is unchanged.

S7.3. Idle-Capacity Accounting

Let t_0 be the first arrival time, let $b_Q(u)$ be the number of busy servers under policy Q at time u , and define the cumulative unused capacity

$$\Gamma_Q(t) = \int_{t_0}^t (k - b_Q(u)) du.$$

This includes idle capacity caused by having fewer than k jobs in the system; work conservation excludes only deliberate idling while a job waits.

Lemma S7 (Exact accounting with idle-capacity correction). *For every finite input, every non-preemptive work-conserving policy P , and every job i ,*

$$k(W_P[i] - W_{\text{FCFS}}[i]) = \text{In}_i - \text{Out}_i + \Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i) - \beta_i^P + \beta_i^{\text{FCFS}}.$$

The identity was checked on 214,735 job-instances, including tied events, non-batch arrivals, and same-phase overtakers.

Proof. Let $\text{Arr}(t)$ be the total cost of the jobs that have arrived by t and $\text{Exe}_Q(t)$ the work executed by t under Q . A busy server executes one unit of work per unit of time, so $\text{Exe}_Q(t) = k(t - t_0) - \Gamma_Q(t)$ and

$$U_Q(t) = \text{Arr}(t) - \text{Exe}_Q(t) = \text{Arr}(t) - k(t - t_0) + \Gamma_Q(t).$$

At a_i , after the arrivals and before the dispatches at that instant, the jobs of rank i or higher are untouched in both runs, so R_i^Q differs from $U_Q(a_i)$ by the same amount under both policies. Hence

$$R_i^P - R_i^{\text{FCFS}} = U_P(a_i) - U_{\text{FCFS}}(a_i) = \Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i).$$

Subtracting equation (9) of the manuscript for FCFS, where $\text{In}_i = \text{Out}_i = 0$, from the same equation for P and substituting this difference proves the formula. A same-phase overtaker enters both In_i and β_i^P with its full size and therefore cancels. $\square$

Writing $D_i = \Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i)$, the arrival-frozen budget

$$b_i = \max\{0, k(G - L) - (k - 1)L - D_i\}$$

has the mathematical promise G in the standard range $G \geq (3 - 2/k)L$: combine $\text{In}_i < b_i + kL$ with Lemma S7 and $\beta_i^{\text{FCFS}} \leq (k - 1)L$. This is not a deployable rule under the paper's completion-only observation model. In general $\Gamma_{\text{FCFS}}(a_i)$ depends on when an FCFS-shadow job completes before its size has been revealed in the realised run; simulating the shadow does not supply that missing completion time.

S7.4. Instance-Dependent Order-Statistic Bounds

For a set of jobs S , let $\Lambda_r(S)$ be the sum of its largest at most r original service times, with missing terms replaced by zero. We abbreviate

$$\Lambda_{k-1}(t) = \Lambda_{k-1}(\{j : a_j \leq t\}), \quad \Lambda_{k-1}^{<i}(t) = \Lambda_{k-1}(\{j : j < i\}),$$

and

$$\Lambda_k^{>i}(t) = \Lambda_k(\{j : j > i, a_j \leq t\}).$$

Proposition S8 (Instance-dependent workload gap). *For any two non-preemptive work-conserving policies Q_1, Q_2 ,*

$$|U_{Q_1}(t) - U_{Q_2}(t)| \leq \Lambda_{k-1}(t) \quad \text{for every } t.$$

Proof. Let $D = U_{Q_1} - U_{Q_2}$. An arrival adds its cost to both workloads, so D changes only while servers work, and continuously; $\Lambda_{k-1}(t)$ never decreases. If Q_1 has fewer than k jobs present at t , all of them are in service by work conservation, each has at most its full cost left, and they arrived by t , so $U_{Q_1}(t) \leq \Lambda_{k-1}(t)$. Hence whenever $D(t) > \Lambda_{k-1}(t)$, which forces $U_{Q_1}(t) > \Lambda_{k-1}(t)$, all k servers of Q_1 are busy, U_{Q_1} falls at rate k , and D does

not increase. Since $D = 0$ before the first arrival, D can never rise above Λ_{k-1} : at the last instant at which $D \leq \Lambda_{k-1}$ held, D would have to increase while above it. Exchanging Q_1 and Q_2 gives the other direction. $\square$

Proposition S9 (Instance-dependent excess accounting). *For every policy P and job i ,*

$$k \text{ excess}_P[i] \leq \text{In}_i - \text{Out}_i + \Lambda_{k-1}(a_i) + \Lambda_{k-1}^{<i}.$$

Proof. In Lemma S7 the idle-capacity difference $\Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i)$ equals the workload difference $U_P(a_i) - U_{\text{FCFS}}(a_i)$, which Proposition S8 bounds by $\Lambda_{k-1}(a_i)$. Drop $-\beta_i^P \leq 0$. The at most $k-1$ FCFS jobs still running when i starts all have rank below i , so $\beta_i^{\text{FCFS}} \leq \Lambda_{k-1}^{<i}$. $\square$

Proposition S10 (Instance-dependent overtaking under the guard). *If job i has an overtaker and τ is the dispatch time of its last overtaker, then*

$$\text{In}_i < \text{budget}(i, \tau) + \Lambda_k^{>i}(s_i^P).$$

If the budget is nondecreasing in time, the first term may be replaced by $\text{budget}(i, s_i^P)$; with only a uniform cap it may be replaced by $B_{\max}$.

Proof. As in the proof of Theorem 7 of the manuscript, i is waiting and not eligible when its last overtaker is started, so the overtakers completed by τ have total cost over $[i](\tau) < \text{budget}(i, \tau)$. The other overtakers are running at τ : the one started at τ and at most one on each other server. These at most k jobs have rank above i and arrived by $\tau \leq s_i^P$, so their total cost is at most $\Lambda_k^{>i}(s_i^P)$. $\square$

Proposition S11 (Combined instance-dependent guard bound). *Under a cap $B_{\max}$, every job satisfies*

$$k \text{ excess}_P[i] \leq B_{\max} + \Lambda_k^{>i}(s_i^P) + \Lambda_{k-1}(a_i) + \Lambda_{k-1}^{<i},$$

and the inequality is strict when i has an overtaker. When it has none, the bound holds without the first two terms.

Proof. Combine Propositions S9 and S10, use $\text{Out}_i \geq 0$, and apply the cap. If no job overtakes i , then $\text{In}_i = 0$ and Proposition S9 gives the bound without them. $\square$

These Λ quantities use actual sizes, including sizes of unfinished jobs, and hence are offline instance certificates in the completion-only model. Replacing each unknown size by a public class-specific upper bound gives a deployable but generally looser inequality. The homogeneous substitution recovers the main text's aggregate worst-case constant; it does not prove that each order-statistic term is separately minimal.

As an illustration, consider independently sampled truncated-Pareto sizes with $\alpha = 1.5$, $x_{\min} = 1$, hard cap 1000, sample size $N = 200$, and 5,000 samples drawn with seed 73621. The first ratio below sets L after the fact to the maximum of each finite sample; the second uses the fixed hard cap as the denominator.

k	$\mathbb{E}[\Lambda_{k-1}/((k-1)L_{\text{sample}})]$	$\mathbb{E}[\Lambda_{k-1}/((k-1)L_{\text{cap}})]$
4	0.6825	0.0444
8	0.4648	0.0263
16	0.3139	0.0161

Thus the larger ratios use $L = L_{\text{sample}} = \max_j C_j$, not the published timeout $L_{\text{cap}} = 1000$.

S7.5. Why Kill-and-Restart Tiering Does Not Escape the Floor

Section 9 of the manuscript states that the obvious escape from the $\Omega(L)$ floor fails. Kill-and-restart tiering runs every job for a short first quantum τ and restarts the survivors, discarding the killed attempt; on n jobs of length L arriving together it leaves some job with an excess of $n\tau/k - L$ over first-come first-served, and that excess is wasted work rather than overtaking, so no budget removes it. The light-job constant does not improve either, a second-tier piece being still a non-preemptive block of length L : holding τ fixed and varying L , the worst light-job additive constant we measure grows as about $1.1L$ and is flat in τ . The mechanism is TAGS [25] with the dedicated hosts removed, and removing them is what breaks it: in TAGS the kill is a routing device and the work is redone on the next host, whereas in a judging queue the kill is terminal.

S7.6. Reservation-and-Refund Charging under Enforced Run-Time Caps

Everything below assumes an information model the manuscript does not. Each job j carries a cap ℓ_j with $C_j \leq \ell_j \leq L$ that is known when the job arrives and that the service enforces, whereas Sections 5 and 6 of the manuscript assume a service that learns a job's cost only at its completion. That is why this is a supplementary subsection and not a theorem of Section 6: admitting it into the main line would put two information models side by side.

The mechanism itself is older than the guard, and we claim none of it. Conservative backfilling gives every queued job a reservation when it is submitted and lets a job move ahead only if it does not delay any job in the queue [26], and enforced caps are standard there, a job that exceeds its declared run time being killed. With one server-slot per job the head's reservation is whichever server is free at that instant, so in our model both conservative and aggressive (EASY) backfilling reduce to first-come first-served and neither is a comparator here. Lindsay et al. [21] keep the same tentative profile, reserve on the enforced estimate, compress the profile when a job ends early, and guarantee every job an absolute start time fixed at its arrival. Reservation with refund is therefore their mechanism; what differs here is the quantity guaranteed, a per-job bound on the delay relative to first-come first-served, proved pathwise, with a maximal permission rule at one server, and the exact constants at $k = 1$. Load-balancing guardrails [24] charge dispatched work per size rank and per server and balance those counters across servers, with the size known at dispatch, no reference policy and no budget parameter; their Definition 2.1 does adjust a counter, lowering it to the minimum across servers when a server empties, so it is not a rule without refund, only one that never refunds at an individual job's completion.

Give every job q an allowance $Z_q \geq 0$, and write $\text{over}_R[q](t)$ for the charge a waiting job q carries: the sum, over the jobs of rank above q already started, of the job's cap while it is in service and its true cost once it has completed.

Proposition S12 (Reservation charging bounds the overtaken work). *Accept the base policy's proposal j only when j is the queue head or when $\text{over}_R[q] + \ell_j \leq Z_q$ for every waiting q of rank below j , and dispatch the head otherwise. Then $\text{In}_i \leq Z_i$ for every input, every base policy and every job i .*

Proof. Rank is decided by (a_j, index) , so every job counted in In_i is dispatched after i arrives and before i starts, at an epoch where i is waiting and is not the head; the head exemption therefore does not apply and the test is run with $q = i$. The charge carried for an already dispatched job is its cap while it runs and its true service time afterwards, and both are at least its service time, so $\text{over}_R[i]$ at the dispatch of the s -th overtaker is at least $\sum_{u < s} C_{j_u}$; the test then gives $\sum_{u \leq s} C_{j_u} \leq \text{over}_R[i] + \ell_{j_s} \leq Z_i$. A rejected epoch dispatches

the head, whose rank is not above i , so it never enters In_i . The third step is where the cap has to be enforced: a job that may exceed ℓ_j breaks it at once. $\square$

Proposition S13 (The promise under a constant allowance). *With $Z \equiv B$, the rule gives excess $\leq B$ at $k = 1$ and $k \text{ excess} < B + 2(k - 1)L$ for $k \geq 2$. Setting $B = k(G - (2 - 2/k)L)$ therefore offers the promise G .*

Proof. Proposition S12 gives $\text{In}_i \leq B$. At $k = 1$ Theorem 2 of the manuscript is the equality $\text{excess}_P[i] = \text{In}_i - \text{Out}_i \leq B$. For $k \geq 2$ the same theorem, with its strict inequality and $\text{Out}_i \geq 0$, gives $k \text{ excess}_P[i] < B + 2(k - 1)L$, and dividing by k at the stated B gives an excess below G . $\square$

The additive constant of the promise falls from $(3 - 2/k)L$ to $(2 - 2/k)L$, a drop of exactly L at every k : the kL overshoot that completed-work charging pays in the guard's overtaken-work bound disappears, while the $2(k - 1)L$ slack of the identity remains. That residual is still approached and not attained. On the upward witness family of Section S8.2 run at $B = f + L + 1$, the ratio $(k \text{ excess} - B)/L$ is exactly $2 - 2^{1-m}$ after m rounds at $k = 2$, reaching 255/128, and reaches 907/243 at $k = 3$ and 323/64 at $k = 4$, against the limits $2k - 2$ of 2, 4 and 6; a closed-form witness that attains the constant is not known.

Two consequences are worth separating, because they are easy to confuse. The *entry threshold* does not drop by L . An admitted overtaker always has $\ell_j \leq Z_q$, so at $k = 1$ an overtaker of cap L becomes admissible as soon as $G \geq L$, which meets the L -scale floor of Proposition 13 of the manuscript with equality, and for general k the condition is $G \geq (2 - 1/k)L$ against $G > (3 - 2/k)L$ for Algorithm 1 of the manuscript, a difference of $(1 - 1/k)L$, which is zero at one server. It is the additive constant of the promise, not the threshold, that falls by exactly L .

At one server the reservation is inert. A dispatch epoch there has an idle server, so no job of rank above the head is in service and over_R coincides with the completed-work charge: over 255,861 readings at $k = 1$ the two never differ, against 8,277 differences in 110,503 readings at $k \geq 2$. The single-server gain comes entirely from the forward-looking ℓ_j term in the admission test. For a constant allowance only the queue head has to be tested, because $\text{over}_R[q]$ is non-increasing in rank, so the implementation cost of Section 5 of the manuscript is unchanged.

Proposition S14 (One server: closed form and maximal permission). *Take the family of Theorem 14 of the manuscript at $k = 1$, m jobs of cost L followed by n jobs of cost c , all arriving at $t = 0$, with integer costs and G , every long job carrying the cap L and every short job the cap ℓ , $c \leq \ell \leq L$. Around an exact shortest-job-first proposal sequence, the rule with $Z \equiv G$ closes $\min(1, \max(0, \lfloor (G - \ell)/c \rfloor + 1)/n)$ of the FCFS-to-SJF total-wait gap. This is $\lfloor G/c \rfloor/n$ at $\ell = c$, the size-aware optimum of Proposition S17, and $(\lfloor (G - L)/c \rfloor + 1)/n$ at $\ell = L$, the size-oblivious frontier of Proposition S20, so at $\ell = L$ it matches, and does not beat, the sharp completed-work guard. Moreover, at any reachable history with queue head h , a wrapper that reads the caps and promises G must reject a later-ranked proposal j whenever $\text{over}_R[h] + \ell_j > G$; the largest constant allowance with promise G is therefore exactly $B^* = G$, against $B^* = G - L + 1$ for completed-work charging, and the statement needs no integer lattice.*

The maximality claim is proved as Proposition S19 is, with the cap in place of L . Suppose a wrapper starts a later-ranked j while $\text{over}_R[h] + \ell_j > G$. At $k = 1$ no job is in service at a decision, so $\text{over}_R[h]$ is the total cost of the overtakers of h already completed. Change the input only in j 's cost, setting it to ℓ_j . Before j completes, the wrapper sees the same arrivals, caps, completions and proposals in both inputs, and so starts j in both. In

the changed input $\text{In}_h \geq \text{over}_R[h] + \ell_j > G$, and $\text{Out}_h = 0$ because every job of lower rank than the head has already started, so the head's excess exceeds G . Two conditions are used. $W_{\text{FCFS}}[h]$ does not change, because it depends only on the jobs of rank below h . And the promise must hold for every sequence of proposals the base policy can make, since a base that reads true costs might not propose j again after the change.

At $k \geq 2$ the rule is not pointwise maximal, and this is settled rather than open. Take two servers, four unit jobs arriving together and $G = 0$. At the first epoch the rule refuses every later-ranked proposal, because $\text{over}_R[0] + \ell_1 = 1 > 0$, yet dispatching job 1 is safe: the second server takes the head in the same instant, the remaining two start at $t = 1$ exactly as under first-come first-served, and every job has excess 0 under any continuation. The scale of the gap is not marginal. Of 452,810 refused actions at $k = 2$, 416,368 leave the victim inside the promise after relabelling, and 481,964 of 491,340 do so at $k = 3$. This is what one expects once In and excess stop corresponding one to one on several servers.

Finally, the residual the reservation cannot remove is not an artefact of charging. There are inputs on which a job has $\text{In}_i = \text{Out}_i = 0$ and excess $L(1 - ((k-1)/k)^m)$, so no charging rule of any kind removes an $\Omega(L)$ term. Its supremum is L , approached and not attained, which is the whole of $(2 - 2/k)L$ at $k = 2$ and only the part of it up to L for $k \geq 3$.

The claims above are the ones an independent check re-derived from the written rule and re-ran against its own implementation: 1,983,467 exhaustive runs and 10,230,898 per-job checks give $\max(\text{In} - Z) = 0$ with no violation, the instance form k excess $\leq B + \Lambda_{k-1}(a_i) + \Lambda_{k-1}^{<i}$ holds with zero margin over 1,488,984 checks, the closed form of Proposition S14 matches on all 848 family rows including the corner cases $\ell = C$ and $\ell = L$, and the $k = 1$ maximality step survives 573,252 relabelling tests. Two statements of the earlier study did not survive that check and are not made here: that the entry threshold drops by L , and that maximality at $k \geq 2$ is open. A third needs a qualifier: the largest excess attainable at budget B equals B when the cap is tight, $\ell = C$, and is 0 while $B < L$ when $\ell = L$, so in general only $\leq B$ can be claimed.

S8. Extremal Constructions and the Remaining Proofs

Every constant the theory section calls exact is witnessed by a construction, and every such construction is given here in full, beside the proofs whose ideas Section 6 of the manuscript states. All of them are exact in integer arithmetic; every one was simulated and every intermediate quantity of every proof checked job by job, with an independent audit of work conservation and server capacity re-derived from the produced schedule. A cross-reference without an S points into the main manuscript.

The six subsections below run in the order the theory section raises them, and each is its own idea in one line. The cascade drives the workload gap of Lemma 1 of the manuscript to its ceiling geometrically: in m identical rounds, each opening with $k-1$ long jobs of service L and then L short jobs of service 1, a static-priority policy that prefers the short ones carries $k-1$ part-served jobs into the next round, and the carried workload obeys $\rho_{j+1} = (k-1)(\rho_j + L)/k$, so that $\rho_m = (k-1)L(1 - ((k-1)/k)^m) \rightarrow (k-1)L$; the rescaling that fixes an arbitrary service bound is part of the lemma. Both witnesses for the identity run that cascade and then inject a finale at T_m : upward, $k-1$ long jobs, the victim and a stream of more than $2kL$ unit jobs arrive together, first-come first-served starts the victim at once with $\beta_i^{\text{FCFS}} = (k-1)L$ while the base policy holds it until the stream runs out, leaving $\Delta_i \geq 2(k-1)L - (k-1)L((k-1)/k)^m - (k-1)$; downward, the mirrored cascade is followed by a filler job of service f , the victim and $k-1$ jobs of service L , giving $\Delta_i = -\rho_m - (k-1)L \rightarrow -2(k-1)L$, and non-attainment follows from Lemma 1 of the manuscript and the *Size of β* step of the proof of Theorem 2. For the

guard bound, part (i) is a queue of k jobs of service L , the victim and a stream of unit jobs, where the constant budget B makes the guard fire at $L + \lceil B/k \rceil$, so the term B/k is attained; part (ii) bounds every finite instance by Theorem 7 of the manuscript strictly and approaches $3k - 2$ from below by running the cascade for m rounds and then injecting a finale timed so that $\text{over}[i]$ sits below $B = f + L + 1$ at three successive dispatch epochs, at 0, at f and at $f + L = B - 1$, the last by a single unit, which lets the base policy fill every server three times before the guard fires. The single-server family places the victim ahead of $B/g - 1$ jobs of service g and one job of service L , with g dividing B : the guard does not fire while $\text{over}[i] = B - g < B$, so the long job is taken and $\text{excess}_P[i] - B = L - g$, which approaches L and never reaches it. The price of the guarantee counts inverted pairs in two ways, which makes $\sum_i \text{In}_i$ and $\sum_i \text{Out}_i$ the two sums of service times over the inversions exactly—the pairwise-interchange argument behind Smith’s rule [27] stated on an arbitrary arrival sequence—and then bounds the numerator through Corollary 6 of the manuscript and the denominator through the mn inversions of shortest job first. Achievability counts the short jobs exact shortest job first places before the first long one, which is $r_g = \min(n, k \lceil B/(kc) \rceil)$ and at least qn , while maximality turns on one relabelling: if a wrapper permits a later-ranked job while the head has $\text{over}[h] > G - L$, giving that job the unrevealed size L leaves the transcript the wrapper sees unchanged and pushes the head past G , after which the frontier counts $r = \lfloor (G - L)/c \rfloor + 1$ permitted short-before-long moves.

S8.1. The Cascade

The cascade drives the workload gap of Lemma 1 of the manuscript to its ceiling, and everything that shows the constant $2(k - 1)L$ cannot be lowered starts from it.

Lemma S15. Fix $k \geq 2$ and $m \geq 1$ and set $L = k^m$. There is an input and a work-conserving policy P such that, at time $T_m = mL$, first-come first-served has completed every job and idles all k servers, while P holds exactly $k - 1$ jobs in service, each with remaining service

$$f = \frac{\rho_m}{k - 1} = L \left(1 - \left(\frac{k-1}{k} \right)^m \right), \quad \rho_m = (k - 1)L \left(1 - \left(\frac{k-1}{k} \right)^m \right),$$

so that $U_P(T_m) - U_{\text{FCFS}}(T_m) = \rho_m \rightarrow (k - 1)L$ as $m \rightarrow \infty$. Every ρ_j and every intermediate time is an integer. For any prescribed service bound $L_0 > 0$, multiplying every arrival time and service time in the construction by L_0/k^m gives the same ratios with bound L_0 ; integrality is used only before this final rescaling.

Proof. The input consists of m identical rounds; round j occupies $[T_j, T_{j+1}) = [jL, (j + 1)L)$. At time T_j inject, in rank order, $k - 1$ long jobs of service L and then L short jobs of service 1, so that the long jobs carry the lower ranks. Let P be the static-priority policy that dispatches a waiting short job whenever one exists and a long job otherwise; it is work-conserving.

The empty-system base round must be treated separately. At $T_0 = 0$ both systems are empty. First-come first-served dispatches the $k - 1$ long jobs and one short job; its remaining server completes all L short jobs by $T_1 = L$, when the long jobs also complete. Policy P first processes the L short jobs across all k servers, finishing them at L/k , and then dispatches the $k - 1$ long jobs. At T_1 each has L/k remaining, so $\rho_1 = (k - 1)L/k = (k - 1)(\rho_0 + L)/k$ with $\rho_0 = 0$.

For the induction step, fix $1 \leq j < m$ and suppose that at T_j FCFS is empty, whereas P has exactly $k - 1$ carried-over jobs with equal remaining service and total ρ_j . FCFS again dispatches the new $k - 1$ long jobs and one short job; one server processes the L short jobs and all work finishes at T_{j+1} . Under P , the one free server takes a short job. The carried-

over jobs complete at $T_j + \rho_j / (k - 1)$ and their servers then take short jobs. All k servers are busy until the short work runs out at $T_j + \tau_j$, where $k\tau_j = \rho_j + L$. Indeed, $\rho_j / (k - 1) \leq \tau_j$ is equivalent to $\rho_j \leq (k - 1)L$, which follows from the induction hypothesis. At that instant P dispatches the round's $k - 1$ long jobs and leaves one server idle.

On $(T_j + \tau_j, T_{j+1}]$, FCFS has k busy servers and P has $k - 1$. At T_{j+1} each of those long jobs has τ_j left, reproducing the hypothesis with $\rho_{j+1} = (k - 1)\tau_j = (k - 1)(\rho_j + L)/k$. Together with the base round this solves to $\rho_j = (k - 1)L(1 - ((k - 1)/k)^j)$. With $L = k^m$, every raw ρ_j and τ_j for $j \leq m$ is an integer; the rescaling in the statement is then applied to the whole input, including its arrival times. $\square$

S8.2. Tightness of the Net-Overtake Identity

In both directions $\sup |\Delta_i| = 2(k - 1)L$, which is Theorem 4 of the manuscript. Two witness families prove it, one for each direction.

Proof of Theorem 4 of the manuscript. Upward. Run the cascade of Lemma S15 for m rounds and, at T_m , inject in rank order $k - 1$ jobs of service L , then the victim i of service L , then $n_{\text{sh}} > 2kL$ jobs of service 1. Let P continue as the static priority “the $k - 1$ new long jobs first, then the short jobs, then i ”.

At T_m all k servers of the reference policy are free and its waiting set in rank order is the $k - 1$ new long jobs, then i , then the short ones. It dispatches the long jobs and then i , all in the phase at T_m . Hence $W_{\text{FCFS}}[i] = 0$; the $k - 1$ new long jobs are dispatched strictly before i in its sequence and are in service at $s_i^{\text{FCFS}} = T_m$ with their full service, so $\beta_i^{\text{FCFS}} = (k - 1)L$ and $R_i^{\text{FCFS}} = (k - 1)L$.

Under P , one server is free at T_m and takes a new long job; when the $k - 1$ carried-over jobs finish, all at $T_m + f$ with $f < L$, their servers take the remaining new long jobs and then short jobs. The victim waits while any short job waits. Short jobs are executed at rate at most k , so they are not exhausted before $T_m + n_{\text{sh}}/k$, and $n_{\text{sh}} > 2kL > k(f + L)$ makes that later than $T_m + f + L$, by which time every new long job has completed. So i is dispatched after all $k - 1$ new long jobs have finished, and at s_i^P the other $k - 1$ servers hold short jobs, giving $0 \leq \beta_i^P \leq k - 1$. Also $R_i^P = \rho_m + (k - 1)L$, since the carried-over jobs and the new long jobs all have rank below i and the new long jobs are untouched at a_i . Therefore $R_i^P - R_i^{\text{FCFS}} = \rho_m$ and

$$\begin{aligned} \Delta_i &= (R_i^P - R_i^{\text{FCFS}}) - \beta_i^P + \beta_i^{\text{FCFS}} \geq \rho_m + (k - 1)L - (k - 1) \\ &= 2(k - 1)L - (k - 1)L\left(\frac{k-1}{k}\right)^m - (k - 1), \end{aligned}$$

and both error terms are $o(L)$ as $m \rightarrow \infty$ with $L = k^m$.

Downward. Run the mirrored cascade: inject the L short jobs first in rank order and the $k - 1$ long jobs after, and let P be “long jobs before short ones”. The induction of Lemma S15 runs with the roles of the two policies exchanged, so at T_m the policy P is empty and idle while the reference holds $k - 1$ jobs with f of service left each. At T_m inject, in rank order, one filler job of service f , the victim i of service $L - 1$, and $k - 1$ jobs of service L ; let P be “the $k - 1$ new long jobs first, then i , then the filler”.

At T_m the reference policy has $k - 1$ servers busy with carried-over jobs, each with exactly f left, and one free server, which takes the filler—the lowest-ranked waiting job—which also has service f . So all k of its servers complete simultaneously at $T_m + f$, and it then dispatches i , with nothing else dispatched in that phase before it. Hence $W_{\text{FCFS}}[i] = f$ and $\beta_i^{\text{FCFS}} = 0$.

Under P the system is empty at T_m with k free servers, and it dispatches the $k - 1$ new long jobs and then i , all at T_m . So $W_P[i] = 0$, $\text{In}_i = (k - 1)L$, $\beta_i^P = (k - 1)L$, and the filler, of rank below i , is dispatched after i , so $\text{Out}_i = f$. Therefore

$$\begin{aligned}\Delta_i &= k(0 - f) - ((k - 1)L - f) = -(k - 1)f - (k - 1)L \\ &= -\rho_m - (k - 1)L \longrightarrow -2(k - 1)L.\end{aligned}$$

These calculations use the raw normalisation $L = k^m$. Given a fixed target bound L_0 and tolerance ε , multiply the entire upward or downward instance by L_0/k^m . The service bound is then exactly L_0 , all waiting times and every term in Δ_i scale by the same factor, and the upward error becomes $(k - 1)L_0((k - 1)/k)^m + (k - 1)L_0/k^m$, while the downward error tends to zero. Choosing m large proves the statement with L_0 fixed.

Non-attainment. For $k \geq 2$, Lemma 1 of the manuscript gives $|R_i^P - R_i^{\text{FCFS}}| < (k - 1)L$ strictly, and the *Size of β* step of the proof of Theorem 2 of the manuscript gives $0 \leq \beta_i^P, \beta_i^{\text{FCFS}} \leq (k - 1)L$; combining them in each direction gives $|\Delta_i| < 2(k - 1)L$. $\square$

For the record, the exact values of Δ_i/L produced by these two families, against the ceiling $2(k - 1)$:

k	L	m	upward Δ_i/L	downward Δ_i/L	$2(k - 1)$
2	64	6	$127/64 = 1.984$	$-127/64 = -1.984$	2
3	81	4	$290/81 = 3.580$	$-292/81 = -3.605$	4
4	64	3	$303/64 = 4.734$	$-303/64 = -4.734$	6

In the executed-work convention of Remark S1 of this file the upward column is unchanged to within $o(L)$, while the downward column becomes $-63/64$, $-130/81$ and $-111/64$, values that tend to $-(k - 1)L$ as m grows rather than to $-2(k - 1)L$. The same-phase share of $|\Delta_i|$ is 0.00%, 0.69% and 0.00% upward and 50.39%, 55.48% and 63.37% downward, and the victim's own excess is $+3.00L$, $+3.20L$, $+3.19L$ upward and $-0.98L$, $-0.80L$, $-0.58L$ downward.

S8.3. Tightness of the Guard Bound

Section 6 of the manuscript states that search does not see the growth of the additive constant with k . A hill-climb over instances of at most 20 jobs suggests a flat value near $1.75L$, while the witness below needs 396 jobs at $k = 2$.

Both constants of the guard bound are exact, the first attained and the second approached. Section 6 of the manuscript states that in prose; here is the statement it summarises, followed by the two constructions behind it.

Proposition S16 (Tightness of Theorem 7 of the manuscript). *Take the constant budget $\text{budget}(q, t) \equiv B = B_{\max}$, and write $c(k) = \sup(k \text{ excess}_P[i] - B)/L$ over all inputs, base policies and budgets; by Theorem 7 of the manuscript the supremum over all budget rules that respect the cap B is the same number. Then*

- the term B/k is attained: on an explicit family the excess is exactly $\lceil B/k \rceil$, hence B/k whenever k divides B ;
- every finite instance satisfies $(k \text{ excess}_P[i] - B)/L < 3k - 2$, while its supremum is exactly $3k - 2$ for every $k \geq 1$. Equivalently, at every k the additive constant $(3 - 2/k)L$ of (11) of the manuscript cannot be lowered.

Proof of Proposition S16(i). At $t = 0$ let there arrive, in rank order, k jobs of service L , the victim i , and at least $B + k$ jobs of service 1; let the base policy be the static priority “the k long jobs, then the short ones in rank order, then i ”, and take the constant budget B .

First-come first-served dispatches the k long jobs in the phase at $t = 0$, filling every server, so $W_{\text{FCFS}}[i] = L$.

If $B = 0$ then $E(t)$ contains every waiting job at every dispatch epoch and the wrapper is first-come first-served, so the excess is $0 = \lceil B/k \rceil$. If $B > 0$ then at $t = 0$ no job has completed, $E(0)$ is empty, and the base fills the servers with the k long jobs. They complete at $t = L$, and $\text{over}[i](L) = 0$ still, every completed job having rank below i . From $t = L$ on, all k servers are free at each integer instant; while $\text{over}[i] < B$ the set E is empty—the waiting jobs of rank above i have no completed job of still higher rank ahead of them—so the base dispatches k short jobs, which complete one time unit later. Hence $\text{over}[i](L + r) = rk$, the first instant at which $\text{over}[i] \geq B$ is $L + \lceil B/k \rceil$, and there i is the minimum-rank member of E and is dispatched. So $\text{excess}_P[i] = \lceil B/k \rceil$. $\square$

Proof of Proposition S16(ii). For every finite instance, Theorem 7 of the manuscript gives $(k \text{excess}_P[i] - B)/L < 3k - 2$: for a constant budget, $k \text{excess}_P[i] \leq \text{In}_i - \text{Out}_i + 2(k - 1)L \leq \text{In}_i + 2(k - 1)L < B + kL + 2(k - 1)L = B + (3k - 2)L$, the last step being the strict inequality (10) of the manuscript. When i has no overtaker the excess is at most $(2 - 2/k)L$ and the inequality is strict as well.

For the lower bound at $k = 1$, the family of Section S8.4 gives $\text{excess}_P[i] - B = L - g \rightarrow L$, so $(k \text{excess}_P[i] - B)/L \rightarrow 1 = 3k - 2$. For $k \geq 2$, fix $m \geq 1$, $L = k^m$, and let f and ρ_m be as in Lemma S15. Run the cascade for m rounds. At T_m inject, all arriving at T_m and in rank order,

- $k - 1$ new long jobs of service L , of rank below the victim;
- the victim i , of service L ;
- one sync job of service f ;
- one pre job of service L ;
- k final jobs of service L ;

set the constant budget to $B = f + L + 1$, and let the base policy be the static priority “short jobs of the cascade, then sync, then the new long jobs and pre, then the finals, and the victim last”.

The reference run. At T_m it is empty with all k servers free; it dispatches the $k - 1$ new long jobs and then i in the phase at T_m , so $W_{\text{FCFS}}[i] = 0$.

The guard does not fire during the cascade. Inside round j the only waiting jobs with completed higher-ranked work are that round’s long jobs, whose higher-ranked jobs are the L short jobs of the round and everything in later rounds, which have not arrived. Over $[T_j, T_j + \tau_j]$ the k servers execute $k\tau_j = \rho_j + L$ units of work, of which ρ_j belongs to the carried-over jobs, so at most L units of short-job work can have completed. Hence $\text{over}[q](t) \leq L < B$ for every waiting q at every dispatch epoch of the cascade, $E(t)$ is empty throughout, and the wrapped run reproduces Lemma S15.

The finale. At T_m one server is free and $\text{over}[q](T_m) = 0$ for every waiting q , so the base chooses and takes the sync job. The $k - 1$ carried-over jobs have exactly f left and the sync job has service f , so all k servers free simultaneously at $T_m + f$. There $\text{over}[i] = f$, because only the sync job has rank above i among the completed jobs, and $f < B$; no other waiting job has a completed higher-ranked job either, so E is empty and the base fills all k servers with the $k - 1$ new long jobs and pre. These all have service L and complete simultaneously at $T_m + f + L$, where

$$\text{over}[i] = f + L = B - 1 < B,$$

since only sync and pre have rank above i among the completed jobs. So E is still empty, the base fills all k servers with the k final jobs, and they complete at $T_m + f + 2L$, at which

instant $\text{over}[i] = B - 1 + kL \geq B$. The guard fires, i is the minimum-rank member of E , and it is dispatched with every server free, so $\beta_i^P = 0$.

The value. $W_P[i] = f + 2L$, $\text{In}_i = B - 1 + kL$, $\text{Out}_i = 0$, and

$$\frac{k \text{excess}_P[i] - B}{L} = \frac{k(f + 2L) - (f + L + 1)}{L} = \frac{(k - 1)f + (2k - 1)L - 1}{L}.$$

As $m \rightarrow \infty$ with $L = k^m$ we have $f/L \rightarrow 1$ and $1/L \rightarrow 0$, so the ratio tends to $(k - 1) + (2k - 1) = 3k - 2$. Hence the supremum, although no finite instance attains it, equals $3k - 2$.

The same instance also has $\Delta_i = \rho_m + (k - 1)L \rightarrow 2(k - 1)L$, so the two sources of slack in Theorem 7 of the manuscript are saturated together. $\square$

The values produced, against the ceiling $3k - 2$:

k	L	m	jobs	B	excess	$(k \text{excess} - B)/L$
2	64	6	396	128	191	$127/32 = 3.969$
2	128	7	909	256	383	$255/64 = 3.984$
3	81	4	340	147	227	$178/27 = 6.593$
3	243	5	1233	455	697	$1636/243 = 6.733$
4	64	3	211	102	165	$279/32 = 8.719$
4	256	4	1046	432	687	$579/64 = 9.047$
5	125	3	399	187	311	$1368/125 = 10.944$
6	216	3	677	308	523	$1415/108 = 13.102$

S8.4. Approaching the Single-Server Bound

At one server the bound $B + L$ of Theorem 7 of the manuscript is the exact supremum of the excess, which is what the manuscript states in prose after Proposition 13. This is the family that approaches it.

Proof of the single-server supremum stated after Proposition 13 of the manuscript. Fix L , a budget $B > 0$, and a granularity g dividing B with $0 < g \leq L$. Take $k = 1$ and, at $t = 0$, the victim i of rank 0, then $B/g - 1$ jobs of service g , then one job of service L ; let the base policy dispatch the jobs of service g first, then the one of service L , and never the victim. First-come first-served starts i at once, so $W_{\text{FCFS}}[i] = 0$.

Under the wrapper, after r of the jobs of service g have completed, $\text{over}[i] = rg$, which stays below B for $r \leq B/g - 1$. At the last such instant $\text{over}[i] = B - g < B$, the guard does not fire, and the base dispatches the job of service L . When it completes, $\text{over}[i] = B - g + L \geq B$ and the guard releases i . Hence $\text{excess}_P[i] = B - g + L$, so $\text{excess}_P[i] - B = L - g$, which approaches L as $g \rightarrow 0$ and never reaches it. For example $(L, B, g) = (100, 40, 1)$ gives $\text{excess}_P[i] - B = 99$, and $(64, 32, 1)$ gives 63. $\square$

S8.5. The Price of the Guarantee

A promise caps how much reordering can buy at all, which is Theorem 14 of the manuscript, and at one server the achievable fraction is known exactly. At $k = 1$ the error term of Theorem 2 of the manuscript vanishes identically and the sum identity is exact, so that fraction can be computed rather than bounded.

Proposition S17 (The achievable fraction at one server). *In the family of Theorem 14 of the manuscript with $k = 1$, the largest value of $(\overline{W}_{\text{FCFS}} - \overline{W}_P)/(\overline{W}_{\text{FCFS}} - \overline{W}_{\text{SJF}})$ over all non-preemptive work-conserving P with $\text{excess}_P[j] \leq G$ for every job is exactly $\min(1, \lfloor G/c \rfloor / n)$.*

Proof of Theorem 14 of the manuscript. Let $\mathcal{I}(Q)$ be the set of inverted pairs of a policy Q , that is the pairs (u, v) in which u arrived before v and v was dispatched first, written $v \prec u$. Counting the same pairs in two ways,

$$\sum_i \text{In}_i = \sum_{(u,v) \in \mathcal{I}(Q)} C_v, \quad \sum_i \text{Out}_i = \sum_{(u,v) \in \mathcal{I}(Q)} C_u,$$

exactly, for every k . This is the standard exchange argument: at $k = 1$, where Theorem 2 of the manuscript is an equality, it says that the only way to beat first-come first-served in mean wait is to invert pairs whose later-arrived job is the shorter one, and that the saving is exactly the sum of the service differences over the inversions. That is the classical pairwise-interchange argument behind Smith's rule [27], stated here on an arbitrary arrival sequence rather than on a single batch; we use it as an observation and claim nothing new in it. Write $S(Q) = \sum_i (W_{\text{FCFS}}[i] - W_Q[i])$ for the total saving against the reference and $H = 2(1 - 1/k)L(m + n)$. Summing Theorem 2 of the manuscript over the $m + n$ jobs gives

$$\left| S(Q) - \frac{1}{k} \sum_{(u,v) \in \mathcal{I}(Q)} (C_u - C_v) \right| \leq H \quad \text{for every } Q. \quad (\text{S4})$$

Numerator. A pair contributes positively only when u is a long job and v a short one, contributing $L - c$. For a fixed long job u , the number of short jobs dispatched before it is at most In_u/c , and Corollary 6 of the manuscript gives $\text{In}_u \leq kG + (3k - 2)L$. Summing over the m long jobs and applying (S4),

$$S(P) \leq \frac{1}{k} m (L - c) \frac{kG + (3k - 2)L}{c} + H.$$

Denominator. Sorting by service time inverts every one of the mn long-short pairs, so its pair sum is exactly $mn(L - c)$, and (S4) applied to it gives $S(\text{SJF}) \geq \frac{1}{k} mn(L - c) - H$. At $k > 1$ the denominator carries the same error as the numerator, so treating the reference saving as exact would understate ε_+ .

Division. Writing $\mathcal{N}$ and D_0 for the two leading terms $\frac{1}{k} m(L - c)(kG + (3k - 2)L)/c$ and $\frac{1}{k} mn(L - c)$, both bounds are positive once $D_0 > H$, and then

$$\frac{S(P)}{S(\text{SJF})} \leq \frac{\mathcal{N} + H}{D_0 - H} = \alpha + \varepsilon_+, \quad \varepsilon_+ = \frac{H(\mathcal{N} + D_0)}{D_0(D_0 - H)} = \frac{H(1 + \alpha)}{D_0 - H},$$

The leading terms are $\Theta(mn)$ and $H = \Theta(m + n)$, so $\varepsilon_+ = O((m + n)/(mn)) \rightarrow 0$; since $H/D_0 = 2(k - 1)L(m + n)/(mn(L - c))$ and $\alpha = \mathcal{N}/D_0 = (kG + (3k - 2)L)/(nc)$, the product $\varepsilon_+ mn/(m + n)$ equals

$$\frac{2(k - 1)L}{L - c} \left(1 + \frac{kG + (3k - 2)L}{nc} \right) / \left(1 - \frac{H}{D_0} \right),$$

which exceeds the same expression without the last factor and approaches it as m, n grow. With k, L, c, G fixed it tends to $2(k - 1)L/(L - c)$, and it vanishes at $k = 1$. Dividing numerator and denominator by $m + n$ turns the two total savings into the two mean savings and leaves the ratio unchanged. $\square$

Proof of Proposition S17. Write r_u for the number of short jobs dispatched before a long job u . Every short job arrived after every long one, so Out_u contains only long jobs, and a long-long inversion (p, q) puts L into In_p and the same L into Out_q ; summing over the

long jobs those terms cancel, and since $k = 1$ makes Theorem 2 of the manuscript an equality,

$$\sum_{u \text{ long}} \text{excess}_P[u] = \sum_{u \text{ long}} (\text{In}_u - \text{Out}_u) = c \sum_{u \text{ long}} r_u.$$

Let u^* be the long job dispatched last among the long ones. No job that arrived before it is dispatched after it, so $\text{Out}_{u^*} = 0$ and $\text{excess}_P[u^*] \geq cr_{u^*}$; and r is non-decreasing along the dispatch order, so $r_{u^*} = \max_u r_u$. The guarantee therefore forces $\max_u r_u \leq \lfloor G/c \rfloor$ and $\sum_u r_u \leq m \lfloor G/c \rfloor$. Only long–short inversions contribute to the saving, each contributing $L - c$, so $S(P) = (L - c) \sum_u r_u$ against $S(\text{SJF}) = (L - c)mn$ and the fraction is at most $\min(1, \lfloor G/c \rfloor / n)$.

For achievability, put $r = \min(n, \lfloor G/c \rfloor)$ and dispatch the r earliest-arrived short jobs, then the m long jobs in arrival order, then the remaining short jobs in arrival order. Every long job then has excess exactly $rc \leq G$, every short job has excess at most 0, and the fraction is r/n . $\square$

So the shape of Theorem 14 of the manuscript is right at $k = 1$ and its additive $(3k - 2)L$ is slack: the guarantee is spent on the long jobs together, not one at a time. Exhaustive enumeration of every work-conserving schedule of the family, over all 1161 parameter settings with $m \leq 3$, $n \leq 4$ and $G \leq 3L$, agrees with Proposition S17 in every case.

S8.6. Achievability and Maximal Permission

The price bound of Theorem 14 of the manuscript is an upper bound on what any policy with the promise can close. Two results say that the bound is nearly met, and that the wrapper concedes nothing it could safely permit. Section 6.10 of the manuscript states both in words; here they are in full, with their proofs. Neither gives instance optimality on arbitrary inputs, and for $k \geq 2$ the exact value of $F^*(G)$ below remains open.

Proposition S18 (Achievability on the two-size batch family). *In the family of Theorem 14 of the manuscript, m jobs of cost L followed by n jobs of cost $c < L$, all arriving at $t = 0$, define*

$$S(Q) = \sum_i (W_{\text{FCFS}}[i] - W_Q[i]), \quad F(Q) = \frac{S(Q)}{S(\text{SJF})},$$

and let $F^*(G)$ be the maximum of $F(P)$ over all non-preemptive, work-conserving policies satisfying $\text{excess}_P[i] \leq G$ for every job. Let the guard use exact SJF as its base and the constant budget $B = kG - (3k - 2)L \geq 0$. Put

$$D_0 = \frac{mn(L - c)}{k}, \quad H = 2(1 - 1/k)L(m + n), \quad q = \min\left(1, \frac{B}{nc}\right), \quad \alpha = \frac{kG + (3k - 2)L}{nc}.$$

If $D_0 > H$, then

$$q - \varepsilon_- \leq F(\text{guard}) \leq F^*(G) \leq \min(1, \alpha + \varepsilon_+),$$

where

$$\varepsilon_- = \frac{H(1 + q)}{D_0 - H}, \quad \varepsilon_+ = \frac{H(1 + \alpha)}{D_0 - H}.$$

Thus the leading terms are separated by at most $2(3k - 2)L/(nc)$, in addition to the two displayed finite-sample errors. For $k \geq 2$, this sandwich does not determine the exact $F^*(G)$.

Proof of Proposition S18. For a policy Q , with the inverted pairs $\mathcal{I}(Q)$ of Section S8.5, let

$$I(Q) = \sum_{(u,v) \in \mathcal{I}(Q)} (C_u - C_v).$$

The summed identity (S4) gives $|S(Q) - I(Q)/k| \leq H$ and, because SJF inverts all mn long–short pairs, $D_0 - H \leq S(\text{SJF}) \leq D_0 + H$.

If $B = 0$, the guard is FCFS and the lower bound is immediate because $q = 0$. Suppose $B > 0$. Until the first long job is dispatched, every completed overtaker is short and exact SJF continues to propose short jobs. The long jobs' accounts grow only as short jobs complete, at most k at a time, and the first long job becomes eligible once $\lceil B/(kc) \rceil$ such rounds have completed, so the number of short jobs started before that first long job is

$$r_g = \min\left(n, k \left\lceil \frac{B}{kc} \right\rceil\right), \quad \frac{r_g}{n} \geq q.$$

Those r_g short jobs precede every long job, and hence contribute at least $mr_g(L - c) \geq qmn(L - c) = qkD_0$ to $I(\text{guard})$; no pair contributes a negative amount, since a short job never arrives before a long one. With (S4) it follows that $S(\text{guard}) \geq qD_0 - H$. Therefore

$$S(\text{guard}) - qS(\text{SJF}) \geq -H(1 + q),$$

and division by the positive lower bound $D_0 - H$ gives $F(\text{guard}) \geq q - \varepsilon_-$.

For any feasible P , Corollary 6 of the manuscript bounds the number of short jobs inverted ahead of each long job by $(kG + (3k - 2)L)/c$, since $\text{In}_u \leq kG + (3k - 2)L$ and each such short job adds c to In_u . Each such pair adds $L - c$ to $I(P)$ and no pair adds a positive amount otherwise, so $I(P) \leq m(L - c)(kG + (3k - 2)L)/c = \alpha kD_0$, and (S4) gives $S(P) \leq \alpha D_0 + H$. Dividing by $S(\text{SJF}) \geq D_0 - H$ yields

$$F(P) \leq \frac{\alpha D_0 + H}{D_0 - H} = \alpha + \frac{H(1 + \alpha)}{D_0 - H} = \alpha + \varepsilon_+.$$

SJF minimises total waiting time for a simultaneous batch on identical servers, so $F(P) \leq 1$. The guarded policy is itself feasible by Corollary 8 of the manuscript, which places it below $F^*(G)$ and completes the sandwich. $\square$

The guard therefore leaves at most $2(3k - 2)L/(nc)$ of the closable gap on the table here, and at one server it leaves none of it. Saying so precisely needs the class of wrappers the claim is quantified over.

Definition S1 (Opaque-base, size-oblivious wrapper). *At a dispatch epoch, the wrapper may use arrivals and ranks, its own dispatch and completion history, the revealed sizes of completed jobs, the public bound L , fixed random coins, and the sequence of proposals made so far by an opaque base policy. It may not inspect unfinished job sizes or the base policy's internal state. A deterministic pathwise promise G means that every job has excess at most G for every finite input and every possible base-proposal transcript.*

Inside that class, and on an integer time grid, the guard's trigger is not one admissible rule among several: it is the largest permission any wrapper with the promise can grant.

Proposition S19 (Pointwise maximality at one server). *Assume $k = 1$, sizes and budgets are nonnegative integer time units bounded by the positive integer L , and $G \geq L$ is an integer. Within Definition S1:*

1. *for constant budget $B \geq 1$, the guard's worst pathwise excess is exactly $B + L - 1$; at $B = 0$ it is FCFS and the excess is zero;*
2. *the largest positive integer constant budget with promise G is $B^* = G - L + 1$;*
3. *at every reachable history, if the queue head h satisfies $\text{over}[h] > G - L$, every wrapper with promise G must dispatch h . If $\text{over}[h] \leq G - L$, permitting one currently waiting*

later-ranked job for this dispatch is safe against the worst unrevealed size L . Consequently the guard triggered by $\text{over} \geq B^*$ is pointwise maximal in its permitted action set.

Proof of Proposition S19. At one server Theorem 2 of the manuscript is the exact relation $\text{excess}_P[i] = \text{In}_i - \text{Out}_i$. If $B = 0$, the queue head is always eligible and the guard is FCFS. Let $B \geq 1$. Immediately before the last overtaker of a job is dispatched, completed overtaking work is an integer strictly below B , hence at most $B - 1$; the last unrevealed job has size at most L . Thus every excess is at most $B + L - 1$. Equality is attained by putting a queue-head victim before $B - 1$ unit overtakers and then one overtaker of size L , and having the opaque base propose them in that order. This proves the first claim and shows that a positive constant budget has promise G exactly when $B \leq G - L + 1$.

For pointwise maximality, consider the minimum-rank waiting job h at a reachable dispatch history and write $w = \text{over}[h]$. All lower-ranked jobs have already been dispatched, so eventually $\text{Out}_h = 0$. If a wrapper permits a later-ranked job j when $w > G - L$, relabel the still-unfinished j to have size L and replay the same opaque proposal transcript. The wrapper has seen the same arrivals, dispatches, completions, revealed sizes and proposals, so determinism makes it take the same action. The FCFS start of h is unchanged because j has later rank, whereas $\text{excess}[h] = \text{In}_h \geq w + L > G$, contradicting the promise. Conversely, if $w \leq G - L$, permitting one later-ranked waiting job can add at most L to the head's overtaking work, so dispatching the head next remains within G . On the integer grid, $w > G - L$ is equivalent to $w \geq B^* = G - L + 1$, exactly the guard's trigger. $\square$

Read on the two-size family, that maximality fixes the exact fraction of the gap a size-oblivious wrapper can close.

Proposition S20 (The single-server consistency–robustness frontier). *On the integer two-size batch family, for $G \geq L$ the pointwise-maximal guard around the exact-SJF proposal sequence closes, of the FCFS-to-SJF total-wait gap, the fraction*

$$\min\left(1, \frac{\lfloor (G - L)/c \rfloor + 1}{n}\right),$$

and no opaque-base wrapper with the pathwise promise can permit a larger fraction of those short-before-long moves.

Exact SJF itself reads unfinished true sizes, so the composed exact-SJF-plus-guard scheduler is not size-oblivious; the frontier concerns the wrapper's permissions under an externally supplied proposal transcript.

Proof of Proposition S20. With $B^* = G - L + 1$, exact SJF proposes short jobs until the guard fires. The number that can pass every long job is

$$r = \min\left(n, \left\lceil \frac{B^*}{c} \right\rceil\right) = \min\left(n, \left\lfloor \frac{G - L}{c} \right\rfloor + 1\right).$$

Each of the m long jobs is inverted with exactly these r short jobs, so the closed fraction is r/n . Once their completed work exceeds $G - L$, the next long job is the queue head and Proposition S19 forces every opaque-base wrapper with promise G to dispatch it; the same applies successively to the remaining long jobs. No such wrapper can therefore permit more of the short-before-long moves on this transcript. $\square$

S9. Extended Related Work

Section 2 of the manuscript keeps one paragraph per theme and a compact citation list. This section holds the per-paper detail that paragraph compresses, in the same order; every work named here is cited in the manuscript as well. Nothing in it is a result of ours.

S9.1. Fairness Referenced to First-Come First-Served

Definition 2 of Schwiegelshohn and Yahyapour [20] calls a schedule λ -fair when jobs submitted after job i cannot increase i 's flow time by more than a factor λ . Their Lemma 4 bounds fairness and efficiency jointly: the preemptive PFCFS policy is $(2 + 2\bar{s})$ -fair, with competitive bounds for weighted completion time and makespan, where $\bar{s}$ is the normalised preemption penalty, and the proof bounds the delay caused to i by a later wide job by $(1 + 2\bar{s})p_i$. That is tighter for a short job than an L -denominated bound, and it is obtained by gang preemption, which our model does not have.

In the fair-start-time line, Sabin, Kochhar and Sadayappan [28] compare a job with a re-simulation excluding later arrivals, and Sabin and Sadayappan [29] and Rajbhandary, Bunde and Leung [30] use the resulting excess as a job-level metric. The fair start time is defined in two variants; the reference used in the manuscript, first-come first-served with no backfilling, is the relaxed one.

S9.2. Overtaking Restrictions in Queueing Theory

Nudge [12], Nudge-K [13] and Nudge*(M) [31] are analysed in steady state under Poisson arrivals with light-tailed or phase-type sizes, and yield, for suitable parameters, stochastic dominance over first-come first-served. Because a job is passed at most once, Nudge implies on every sample path a per-job additive bound of one small-job length; that implication is our own reading and not a claim of [12]. Theorem 2 of the WCFS finite-skip work [14] brackets the mean response time within additive constants of the M/G/1 first-come first-served expression at every load. Yu and Scully [32] define the *crossing work* of later arrivals that precede a tagged job under γ -Boost and derive stationary response-time bounds, and Yu, Harlev, Adakroy and Scully [33] extend the line to k servers.

S9.3. The Additive Shape, Stated as Its Sources State It

Worst-case fair weighted fair queueing bounds a session's service in both directions against the ideal fluid reference, falling behind it by at most one maximum packet and running ahead of it by less than one of the session's own [34]. The error term in the packet scale rate guarantee for expedited forwarding is the transmission time of one maximum-size lower-priority packet, and exists because that packet cannot be preempted [35]; additive per-individual bounds against a reference discipline are older still [36,37]. EEVDF, a preemptive proportional-share scheduler, bounds a client's lag against the fluid ideal by $-r_{\max} < \text{lag} < \max(r_{\max}, q)$, with $r_{\max}$ the largest request and q the quantum, and by $\pm q$ when no request exceeds a quantum [38]. In Graham's list-scheduling bound the additive form is $(1 - 1/k)$ times the length of the last task to finish, and it appears inside his proof [39]. The pathwise work-difference bound nearest to Lemma 1 of the manuscript is [40, Lemma 2], where the work of jobs of remaining size at most x under k -server preemptive shortest-remaining-processing-time exceeds its single-server counterpart by at most kx ; Lemma 1 differs in comparing two arbitrary non-preemptive work-conserving k -server policies, in bounding total unfinished work, and in being two-sided. In the conservation-law family a weighted sum of mean waiting times is invariant across a class of disciplines in steady state [41], and the multiserver accumulating priority queue orders service by a credit linear in waiting time, with exact class waiting-time distributions under exponential service [42,43].

Non-preemptive real-time feasibility conditions charge a whole uninterrupted execution for the reason the packet bounds do [44]. The sample-path members of the conservation-law line, and of the achievable-region work after it, are aggregate or class-level identities rather than per-job comparisons against a named reference [45,46]. The law does extend to several servers [47]. Under non-preemptive priority with $k > 1$, however, it needs every class to share one service-time distribution [48], which is what the model of Section 3 of the manuscript withholds; the work-gap lemma is the quantitative substitute. Aging disciplines [49,50] and the unfairness analyses of shortest-remaining-processing-time against processor sharing [51,52] describe classes in steady state and not individual jobs, whereas $\text{over}[q]$ accumulates overtaken work and carries a pathwise cap. The word itself needs one caution: in queueing theory *overtaking* means departing before a customer who arrived earlier [53,54], while we measure the work that passes a waiting job. A single-server argument also stops working with several servers. The workload vector recursion is Kiefer and Wolfowitz's [55], and its minimality under first-come first-served is Daley's [56], which holds only where arrivals are allocated to servers in a way that does not depend on their service times; the identity of Section 6 of the manuscript replaces that step with a quantitative comparison.

S9.4. Deployed Systems and Relational Benchmarks

The judging-queue scheduler of Zhang, Wu and Lu [57] cuts the queue into fixed-size windows and dispatches in ascending predicted runtime within each; it reports mean response time against first-come first-served, no per-job bound and no adversarial-prediction test in its abstract or its results, and does not describe its arrival process as recorded. The tail-aware inference scheduler [58] argues about starvation in the crossing-work terms of [32]. SFS orchestrates the Linux first-in first-out and fair schedulers from user space to approximate shortest-remaining-time-first, trading longer durations for long functions against the short ones [59], and ALPS derives its per-function rank and time-slice cap from an offline shortest-remaining-processing-time replay of recent traces and reuses the fair scheduler to keep long functions from starving [60]. In the RelBench study against a human data scientist [61], the relational model matched or beat the expert on 11 of 15 tasks, while the authors record that regression tasks are where it struggles; the successor benchmark [62] keeps the same baseline construction.

S10. The Sealed Terms

This section gives the tables of the sealed run (Section 8.2 of the manuscript) that the manuscript does not print. Every figure in them comes from the three sealed terms of the main log, 2023-1, 2023-2 and 2024-1, which were opened once on 25 September 2026 under the frozen protocol. A first attempt at the run stopped at its fourth command, before any table was written. The rule that removes records with zero cost from the simulated trace listed the development terms by name and not the sealed ones, so a service time of zero reached the simulator. The rule was extended to the sealed terms, the protocol was frozen again, and the run was repeated from its first command. The access ledger in the repository records both attempts.

The tables use the original protocol, as their development counterparts do; the five protocols for Guard(600) are in Table 6 of the manuscript. Every Guard and Fixed row stays within its promise, and the per-job check found no violation among the jobs it examined. The single-server trace reuses the 321 copies chosen on the development pool, because choosing copies by the utilisation they produce would be a design decision taken on sealed data. On the sealed terms those copies load the server to a busy-hour utilisation of 2.26, an overloaded queue in which FCFS reaches a $p99_{dl}$ of 3,692.9 s. No guard

closes more than 0.041 of the gap there (Table S30), and the identity of Theorem 2 of the manuscript holds with zero residual on every job.

Table S27. Sealed terms: ranking scores at the three loads under the original protocol, five overlays. Columns as in Table S16.

Load	Policy	p99 _{dl}	Gap closed	Red. (%)	Mean
$\rho = 0.5, k = 8.2$	FCFS	9.85	0.000 [0.000, 0.000]	0.0 [0.0, 0.0]	0.290
	SJF	3.03	1.000 [1.000, 1.000]	69.8 [65.4, 72.9]	0.162
	SPJF-log	3.78	0.894 [0.870, 0.910]	62.4 [57.3, 66.2]	0.209
	SPJF-E	3.74	0.898 [0.879, 0.909]	62.7 [57.9, 66.2]	0.216
$\rho = 0.8, k = 5.2$	FCFS	67.88	0.000 [0.000, 0.000]	0.0 [0.0, 0.0]	2.012
	SJF	18.91	1.000 [1.000, 1.000]	71.9 [69.0, 74.0]	0.763
	SPJF-log	24.82	0.879 [0.854, 0.901]	63.2 [59.4, 66.1]	1.036
	SPJF-E	23.64	0.903 [0.883, 0.919]	64.9 [61.2, 67.7]	1.052
$\rho = 1, k = 4.2$	FCFS	187.20	0.000 [0.000, 0.000]	0.0 [0.0, 0.0]	5.642
	SJF	30.46	1.000 [1.000, 1.000]	83.2 [78.6, 85.4]	1.445
	SPJF-log	41.82	0.925 [0.898, 0.941]	77.0 [70.7, 80.0]	2.085
	SPJF-E	38.95	0.944 [0.923, 0.955]	78.5 [72.7, 81.4]	2.101

Table S28. Sealed terms: the guard at three promises, its three budget shapes, and the comparators, under the original protocol. Columns as in Table S6; excess and harm are the worst over the five overlays, the rest are means.

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 8.2$; FCFS p99 _{dl} = 9.85 s, SPJF-E 3.74 s with max excess 364.7 s								
	Guard(300)	300	3.88	0.878 [0.863, 0.890]	61.3	132.4	95.4	10.91
	Guard-fixed(300)	300	3.81	0.888 [0.872, 0.899]	62.0	117.4	117.4	9.76
	Guard-age(300)	300	3.88	0.878 [0.863, 0.890]	61.3	132.4	95.4	10.91
	Guard-queue(300)	300	3.80	0.888 [0.870, 0.901]	62.0	137.3	94.6	7.38
	Fixed(300)	300	3.75	0.896 [0.877, 0.906]	62.5	157.9	157.9	7.37
	Skip(300), $\kappa = 29$	300	7.49	0.359 [0.294, 0.445]	25.1	60.3	60.2	42.88
	Guard(600)	600	3.75	0.897 [0.878, 0.907]	62.6	264.0	173.9	4.01
	Guard-fixed(600)	600	3.74	0.898 [0.879, 0.909]	62.7	247.5	241.1	1.04
	Guard-age(600)	600	3.75	0.897 [0.878, 0.907]	62.6	264.0	173.9	4.01
	Guard-queue(600)	600	3.76	0.895 [0.876, 0.907]	62.5	346.3	124.6	0.09
	Fixed(600)	600	3.74	0.898 [0.879, 0.909]	62.7	364.7	241.1	0.00
	Skip(600), $\kappa = 74$	600	5.84	0.599 [0.543, 0.673]	41.9	62.8	62.8	36.05
	Guard(1200)	1200	3.76	0.895 [0.876, 0.907]	62.5	346.3	124.6	0.09
	Guard-fixed(1200)	1200	3.74	0.898 [0.879, 0.909]	62.7	364.7	241.1	0.00
	Guard-age(1200)	1200	3.74	0.898 [0.879, 0.909]	62.7	364.7	241.1	0.00
	Guard-queue(1200)	1200	3.76	0.895 [0.876, 0.907]	62.5	346.3	124.6	0.09
	Fixed(1200)	1200	3.74	0.898 [0.879, 0.909]	62.7	364.7	241.1	0.00
	Skip(1200), $\kappa = 164$	1200	4.64	0.770 [0.741, 0.804]	53.8	82.6	78.2	29.36
$\rho = 0.8, k = 5.2$; FCFS p99 _{dl} = 67.88 s, SPJF-E 23.64 s with max excess 1,613.4 s								
	Guard(300)	300	27.88	0.817 [0.794, 0.844]	58.8	221.4	151.1	19.74
	Guard-fixed(300)	300	28.61	0.801 [0.779, 0.832]	57.6	150.9	130.3	22.66
	Guard-age(300)	300	27.88	0.817 [0.794, 0.844]	58.8	221.4	151.1	19.74
	Guard-queue(300)	300	27.24	0.830 [0.812, 0.855]	59.7	221.4	105.6	17.23
	Fixed(300)	300	25.53	0.864 [0.850, 0.884]	62.1	221.4	167.9	17.31
	Skip(300), $\kappa = 20$	300	66.18	0.038 [0.019, 0.066]	2.7	96.3	61.6	76.17
	Guard(600)	600	24.22	0.891 [0.874, 0.908]	64.1	504.6	189.5	7.01
	Guard-fixed(600)	600	24.43	0.887 [0.871, 0.903]	63.8	311.5	262.9	7.40
	Guard-age(600)	600	24.22	0.891 [0.874, 0.908]	64.1	504.6	189.5	7.01
	Guard-queue(600)	600	24.59	0.884 [0.865, 0.902]	63.5	494.9	166.1	0.63
	Fixed(600)	600	24.01	0.895 [0.878, 0.912]	64.4	491.3	464.5	0.43
	Skip(600), $\kappa = 50$	600	63.69	0.093 [0.053, 0.157]	6.6	97.3	78.1	69.24
	Guard(1200)	1200	24.42	0.888 [0.868, 0.905]	63.8	1064.0	166.1	0.31
	Guard-fixed(1200)	1200	23.91	0.898 [0.879, 0.914]	64.5	564.4	517.3	0.17
	Guard-age(1200)	1200	23.84	0.899 [0.881, 0.915]	64.6	1062.0	384.7	0.30
	Guard-queue(1200)	1200	24.42	0.888 [0.868, 0.905]	63.8	1064.0	166.1	0.31
	Fixed(1200)	1200	23.74	0.901 [0.882, 0.917]	64.8	1092.5	1045.3	0.01
	Skip(1200), $\kappa = 110$	1200	59.38	0.186 [0.125, 0.306]	13.3	97.3	81.7	60.50
$\rho = 1, k = 4.2$; FCFS p99 _{dl} = 187.20 s, SPJF-E 38.95 s with max excess 3,202.5 s								
	Guard(300)	300	75.96	0.725 [0.620, 0.803]	60.2	229.7	149.1	21.58
	Guard-fixed(300)	300	106.11	0.552 [0.377, 0.716]	45.6	150.9	148.1	28.95
	Guard-age(300)	300	75.96	0.725 [0.620, 0.803]	60.2	229.7	149.1	21.58
	Guard-queue(300)	300	76.22	0.726 [0.608, 0.808]	60.2	229.7	90.7	20.07
	Fixed(300)	300	68.00	0.778 [0.639, 0.850]	64.6	229.7	207.1	20.21
	Skip(300), $\kappa = 17$	300	186.55	0.005 [0.001, 0.018]	0.4	94.4	61.4	87.57
	Guard(600)	600	44.33	0.911 [0.890, 0.926]	75.8	522.9	293.2	7.10
	Guard-fixed(600)	600	52.58	0.865 [0.797, 0.894]	71.9	321.9	289.1	11.04
	Guard-age(600)	600	44.33	0.911 [0.890, 0.926]	75.8	522.9	293.2	7.10
	Guard-queue(600)	600	45.38	0.904 [0.884, 0.919]	75.2	522.9	157.0	2.26
	Fixed(600)	600	43.38	0.917 [0.899, 0.930]	76.3	522.9	475.0	2.05
	Skip(600), $\kappa = 42$	600	185.51	0.014 [0.004, 0.046]	1.2	107.7	65.6	82.92
	Guard(1200)	1200	43.19	0.918 [0.898, 0.931]	76.3	1111.7	157.0	0.61
	Guard-fixed(1200)	1200	42.88	0.921 [0.903, 0.933]	76.6	583.9	526.4	1.28
	Guard-age(1200)	1200	40.53	0.935 [0.916, 0.945]	77.7	1105.4	408.6	0.74
	Guard-queue(1200)	1200	43.19	0.918 [0.898, 0.931]	76.3	1111.7	157.0	0.61
	Fixed(1200)	1200	40.45	0.935 [0.917, 0.946]	77.8	1111.6	1061.3	0.13
	Skip(1200), $\kappa = 92$	1200	183.31	0.033 [0.011, 0.097]	2.7	167.8	103.7	76.56

Table S29. Sealed terms: deliberately corrupted scores, unguarded and under Guard(600). Columns as in Table S8.

Load	Score	p99 _{dl}		Max excess		Harm
		unguarded	guarded	unguarded	guarded	guarded
$\rho = 0.5$	reversed	12.15	12.21	366.8	263.3	248.8
	random	4.36	4.37	363.2	263.5	256.0
	longest as shortest	8.89	8.91	364.7	264.0	242.1
$\rho = 0.8$	reversed	143.00	150.56	1861.7	530.7	337.6
	random	40.72	42.61	1850.1	521.6	343.4
	longest as shortest	116.03	125.35	1790.3	525.9	344.7
$\rho = 1$	reversed	437.89	462.06	3525.0	537.7	318.7
	random	120.71	170.55	3520.5	522.4	331.9
	longest as shortest	342.53	418.60	3426.4	528.3	296.8

Table S30. Sealed terms: the single server, with the 321 copies chosen on the development pool and a busy-hour utilisation of 2.26. Columns as in Table S10, except that the unguarded rows print the firing rate as the sealed run emitted it, 100.00 for FCFS and 0.00 for SJF and SPJF-E, where Table S10 prints —.

Policy	Promise	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
FCFS	—	3692.94	0.000 [0.000, 0.000]	0.0	0.0	100.00
SJF	—	67.16	1.000 [1.000, 1.000]	21075.7	4899.8	0.00
SPJF-E	—	660.71	0.836 [0.696, 0.921]	21429.6	5766.5	0.00
Guard(300)	300	3627.74	0.018 [-0.061, 0.082]	296.6	75.9	62.06
Guard-fixed(300)	300	3678.73	0.004 [-0.022, 0.036]	140.9	122.5	80.21
Guard-age(300)	300	3627.74	0.018 [-0.061, 0.082]	296.6	75.9	62.06
Guard-queue(300)	300	3624.59	0.019 [-0.061, 0.082]	296.6	75.9	61.31
Fixed(300)	300	3627.74	0.018 [-0.061, 0.082]	298.0	272.6	61.91
Skip(300), $\kappa = 5$	300	3694.63	-0.000 [-0.001, -0.000]	175.6	53.6	98.73
Guard(600)	600	3543.41	0.041 [-0.121, 0.149]	596.1	149.6	40.87
Guard-fixed(600)	600	3624.07	0.019 [-0.062, 0.082]	302.6	272.6	61.42
Guard-age(600)	600	3543.41	0.041 [-0.121, 0.149]	596.1	149.6	40.87
Guard-queue(600)	600	3545.95	0.041 [-0.121, 0.147]	599.3	75.9	40.36
Fixed(600)	600	3544.86	0.041 [-0.121, 0.147]	596.1	555.0	40.69
Skip(600), $\kappa = 10$	600	3693.38	-0.000 [-0.001, 0.000]	235.4	54.1	98.16
Guard(1200)	1200	3558.60	0.037 [-0.202, 0.257]	1194.4	75.9	20.14
Guard-fixed(1200)	1200	3571.32	0.034 [-0.114, 0.145]	567.0	507.9	42.49
Guard-age(1200)	1200	3555.55	0.038 [-0.204, 0.263]	1194.4	376.0	20.95
Guard-queue(1200)	1200	3558.60	0.037 [-0.202, 0.257]	1194.4	75.9	20.14
Fixed(1200)	1200	3559.78	0.037 [-0.202, 0.270]	1196.7	1140.3	21.05
Skip(1200), $\kappa = 20$	1200	3692.13	0.000 [-0.001, 0.001]	307.8	54.1	97.21

S11. The Comparisons under the Online Replay

Section 8.1 of the manuscript reports Guard(600) under the online protocol, and the comparisons after it use the original one. We replayed the main policies of those comparisons under the online protocol on the same fifteen development cells: SPJF-E, SPJF-log and the aging baseline, and at each promise the guard, its three single-shape winners, Fixed, Skip and Timeout, 24 policies in all. Their parameters are the ones selected before the replay, the guard family's on the original clock and the aging credit under the conservative score (Section S3.8). Every other comparison after Section 8.1 of the manuscript stays on the original clock. The sampled audit of Section S3.7 found no mismatch in any of the 360 policy-cells. Table S31 sets each policy's online figures beside its original ones. The replay keeps each policy's week-block replicates, so a difference between two policies is resampled as one quantity, as it is on the original clock. Table S32 gives the paired differences of Sections 8.3 and 8.4 of the manuscript and of Section S3.11 on both clocks, and Table S33 repeats Table 8 of the manuscript under the online protocol.

No policy with a promise exceeds it under the online replay. The replay checks every job against its bound, and the largest excess over FCFS is 0.945 of the promise, for Guard-age(1200) at the busiest load. At the two higher loads every policy closes less of the gap online than on the original clock. At the lightest load 22 of the 24 do, and SPJF-log and Aging(600) close 0.008 and 0.007 more. Within each promise the order of the seven policies changes only between two policies whose original gaps differ by less than 0.003.

The guarantee costs more under the online replay. Guard(G) minus SPJF-E is negative with an interval clear of zero in all nine cells. At the busiest load it is -0.497 , -0.158 and -0.110 at $G = 300, 600$ and 1200 s, against -0.438 , -0.115 and -0.072 on the original clock. Every difference that clears zero on the original clock keeps its sign online, including the guard's lead over the maximum waiting time in Table S33. At the busiest load and $G = 600$ s that lead is $+0.423$ [$+0.204, +0.664$], against $+0.381$ [$+0.158, +0.650$]. Four differences no longer clear zero: the queue-length shape's lead over the guard at $G = 600$ s and the lightest load, the constant shape's deficit at $G = 1200$ s and the busiest load, which Section S3.11 calls barely clear of zero, and SPJF-E's lead over SPJF-log at the two higher loads. Three newly clear zero, all at the lightest load: SPJF-log's lead over SPJF-E, the age-relative shape's lead of 0.006 over the guard at $G = 1200$ s, and Timeout's lead of 0.005 over the guard at $G = 1200$ s.

Three statements of Section S3.11 therefore hold on the original clock only: that at $G = 1200$ s the constant shape's deficit clears zero at the busiest load, that at a loose promise and a light load the shape does no measurable work, and that the queue-length shape at $G = 600$ s halves the guard's harm at every load. Online its worst-overlay harm is 127.1, 135.4 and 128.6 s against the guard's 248.3, 215.7 and 333.2 s, under half only at the busiest load, and its gap is within 0.013 of the guard's. The shape still matters most where the promise is tight and the load high. The constant shape trails the guard by 0.007, 0.026 and 0.131 at $G = 600$ s as the load rises, and by 0.091 and 0.199 at $G = 300$ s at the two higher loads, each interval clear of zero.

The changes between the two rankings alter the conclusion of Section 8.3 of the manuscript. On the original clock the expected-cost score beats the log-scale one at the two higher loads and ties with it at the lightest. Online, SPJF-E minus SPJF-log is -0.022 [$-0.039, -0.009$], $+0.006$ [$-0.004, +0.015$] and $+0.009$ [$-0.000, +0.015$] at the three loads: the log-scale score is ahead at the lightest load, and the expected-cost score's lead at the two higher loads no longer clears zero. Both are replayed with their own frozen models, so the change comes from the history each score may use and not from a new fit.

Table S31. The 24 main policies on the original clock and under the online replay, at the parameters selected before the replay, five overlays. Gap closed online with 95% intervals from resampled weeks. Largest excess and harm are online, the worst over the overlays; times in seconds.

Load	Policy	p99 _{dl}		Gap, orig.	Gap, online	Max exc.	Harm
		orig.	online				
$\rho = 0.5, k = 7.8$							
	SPJF-E	14.68	14.94	0.754	0.740 [0.708, 0.764]	1237.4	248.3
	SPJF-log	14.71	14.59	0.754	0.762 [0.735, 0.785]	1234.0	305.2
	Aging(600)	17.12	17.06	0.613	0.620 [0.551, 0.659]	888.8	245.8
	Guard(300)	15.58	16.03	0.705	0.681 [0.648, 0.706]	198.9	116.3
	Guard-fixed(300)	15.75	16.12	0.697	0.677 [0.649, 0.701]	143.1	126.9
	Guard-age(300)	15.58	16.03	0.705	0.681 [0.648, 0.706]	198.9	116.3
	Guard-queue(300)	15.43	15.81	0.714	0.693 [0.669, 0.714]	192.9	91.2
	Fixed(300)	15.30	15.72	0.721	0.698 [0.672, 0.719]	197.9	195.0
	Skip(300)	26.38	26.51	0.078	0.071 [0.044, 0.100]	102.7	72.6
	Timeout(300)	16.59	16.96	0.652	0.632 [0.597, 0.660]	168.3	137.3
	Guard(600)	14.97	15.22	0.740	0.725 [0.696, 0.746]	499.5	248.3
	Guard-fixed(600)	15.09	15.35	0.733	0.718 [0.691, 0.739]	313.1	248.3
	Guard-age(600)	14.97	15.22	0.740	0.725 [0.696, 0.746]	499.5	248.3
	Guard-queue(600)	14.88	15.16	0.744	0.729 [0.700, 0.749]	496.5	127.1
	Fixed(600)	14.91	15.17	0.742	0.728 [0.699, 0.750]	491.5	248.3
	Skip(600)	24.64	24.93	0.184	0.168 [0.119, 0.222]	127.2	72.6
	Timeout(600)	15.07	15.34	0.734	0.719 [0.691, 0.740]	425.7	248.3
	Guard(1200)	14.78	15.06	0.750	0.733 [0.703, 0.755]	954.3	127.1
	Guard-fixed(1200)	14.83	15.10	0.747	0.731 [0.701, 0.754]	558.6	248.3
	Guard-age(1200)	14.73	14.96	0.751	0.739 [0.707, 0.762]	1037.7	248.3
	Guard-queue(1200)	14.78	15.06	0.750	0.733 [0.703, 0.755]	954.3	127.1
	Fixed(1200)	14.70	14.95	0.753	0.739 [0.708, 0.763]	1085.6	248.3
	Skip(1200)	21.62	21.98	0.367	0.345 [0.264, 0.412]	131.8	85.5
	Timeout(1200)	14.72	14.97	0.752	0.739 [0.707, 0.762]	1001.4	248.3
$\rho = 0.8, k = 5$							
	SPJF-E	43.33	45.42	0.885	0.861 [0.836, 0.877]	3041.1	394.6
	SPJF-log	44.36	45.92	0.873	0.855 [0.831, 0.869]	3055.1	678.9
	Aging(600)	67.30	72.58	0.608	0.546 [0.448, 0.627]	1401.0	343.0
	Guard(300)	55.94	62.42	0.738	0.662 [0.590, 0.709]	206.0	127.9
	Guard-fixed(300)	64.44	70.27	0.640	0.571 [0.448, 0.657]	155.6	128.2
	Guard-age(300)	55.94	62.42	0.738	0.662 [0.590, 0.709]	206.0	127.9
	Guard-queue(300)	56.13	61.83	0.736	0.669 [0.599, 0.711]	208.0	127.1
	Fixed(300)	53.83	58.51	0.763	0.708 [0.636, 0.743]	213.8	177.9
	Skip(300)	119.20	119.53	0.002	-0.002 [-0.004, 0.005]	143.4	58.8
	Timeout(300)	98.76	113.04	0.244	0.078 [-0.077, 0.431]	183.1	165.8
	Guard(600)	47.47	50.57	0.837	0.800 [0.772, 0.819]	509.3	215.7
	Guard-fixed(600)	49.85	52.80	0.809	0.775 [0.743, 0.794]	308.8	264.1
	Guard-age(600)	47.47	50.57	0.837	0.800 [0.772, 0.819]	509.3	215.7
	Guard-queue(600)	47.69	50.98	0.834	0.796 [0.771, 0.813]	503.7	135.4
	Fixed(600)	47.03	49.91	0.842	0.808 [0.785, 0.823]	523.7	394.6
	Skip(600)	118.69	119.20	0.008	0.002 [-0.007, 0.018]	210.9	66.7
	Timeout(600)	51.55	55.09	0.789	0.748 [0.696, 0.775]	458.0	394.6
	Guard(1200)	46.31	49.30	0.850	0.815 [0.791, 0.832]	1102.5	135.4
	Guard-fixed(1200)	46.76	49.34	0.845	0.815 [0.792, 0.831]	573.8	394.6
	Guard-age(1200)	45.16	47.53	0.864	0.836 [0.813, 0.851]	1113.4	373.4
	Guard-queue(1200)	46.31	49.30	0.850	0.815 [0.791, 0.832]	1102.5	135.4
	Fixed(1200)	45.32	47.78	0.862	0.833 [0.811, 0.848]	1109.4	394.6
	Skip(1200)	117.48	118.11	0.022	0.015 [-0.003, 0.060]	221.3	134.1
	Timeout(1200)	45.77	48.25	0.857	0.828 [0.804, 0.843]	1048.3	394.6
$\rho = 1, k = 4$							
	SPJF-E	62.79	74.91	0.916	0.864 [0.827, 0.890]	5812.7	735.1
	SPJF-log	67.17	77.00	0.897	0.854 [0.821, 0.881]	5806.0	1031.5
	Aging(600)	201.72	214.33	0.313	0.258 [0.222, 0.339]	1808.9	468.9
	Guard(300)	163.40	188.97	0.478	0.367 [0.278, 0.485]	224.6	129.4
	Guard-fixed(300)	223.69	235.16	0.217	0.168 [0.114, 0.263]	161.9	145.9
	Guard-age(300)	163.40	188.97	0.478	0.367 [0.278, 0.485]	224.6	129.4
	Guard-queue(300)	171.79	197.20	0.442	0.331 [0.262, 0.455]	221.9	108.1
	Fixed(300)	169.29	193.74	0.453	0.347 [0.280, 0.501]	230.8	206.1
	Skip(300)	273.14	273.71	0.001	-0.001 [-0.003, 0.001]	146.7	81.3
	Timeout(300)	266.41	268.13	0.031	0.023 [0.008, 0.040]	191.4	191.4
	Guard(600)	89.24	111.25	0.801	0.705 [0.648, 0.768]	517.7	333.2
	Guard-fixed(600)	116.75	141.55	0.682	0.574 [0.475, 0.681]	317.4	298.5
	Guard-age(600)	89.24	111.25	0.801	0.705 [0.648, 0.768]	517.7	333.2
	Guard-queue(600)	91.09	114.26	0.793	0.692 [0.641, 0.753]	520.4	128.6
	Fixed(600)	86.53	106.87	0.813	0.725 [0.674, 0.783]	517.9	469.1
	Skip(600)	272.98	274.06	0.002	-0.002 [-0.007, 0.002]	225.4	82.3
	Timeout(600)	177.08	208.91	0.420	0.282 [0.016, 0.548]	481.3	454.7
	Guard(1200)	79.32	100.11	0.844	0.754 [0.711, 0.799]	1116.4	128.6
	Guard-fixed(1200)	83.32	102.76	0.827	0.743 [0.696, 0.795]	583.4	508.2
	Guard-age(1200)	72.52	90.68	0.874	0.795 [0.755, 0.835]	1134.6	442.3
	Guard-queue(1200)	79.32	100.11	0.844	0.754 [0.711, 0.799]	1116.4	128.6
	Fixed(1200)	72.94	90.51	0.872	0.796 [0.754, 0.836]	1126.9	735.1
	Skip(1200)	273.08	274.43	0.002	-0.004 [-0.012, 0.008]	279.0	164.7
	Timeout(1200)	83.29	103.46	0.827	0.740 [0.683, 0.800]	1085.9	735.1

Table S32. Paired differences in gap closed, left policy minus right, on the original clock and under the online replay, five overlays, with 95% intervals from resampled weeks. The original column is the development package's own. Guard-age(G) at $G = 300$ and 600 s and Guard-queue(1200) are the guard's own grid points, so their rows are zero on both clocks and are omitted.

Load	Difference	Original	Online
0.5	SPJF-E – SPJF-log	−0.000 [−0.016, +0.013]	−0.022 [−0.039, −0.009]
	Guard(300) – Guard-fixed(300)	+0.008 [−0.002, +0.015]	+0.004 [−0.007, +0.012]
	Guard(300) – Guard-queue(300)	−0.009 [−0.016, −0.003]	−0.012 [−0.025, −0.003]
	Guard(300) – SPJF-E	−0.049 [−0.061, −0.036]	−0.059 [−0.076, −0.043]
	Guard(600) – Guard-fixed(600)	+0.007 [+0.003, +0.012]	+0.007 [+0.002, +0.014]
	Guard(600) – Guard-queue(600)	−0.005 [−0.009, −0.002]	−0.003 [−0.007, +0.001]
	Guard(600) – SPJF-E	−0.014 [−0.020, −0.008]	−0.015 [−0.021, −0.008]
	Guard(1200) – Guard-fixed(1200)	+0.003 [−0.000, +0.008]	+0.002 [−0.002, +0.006]
	Guard(1200) – Guard-age(1200)	−0.002 [−0.004, +0.002]	−0.006 [−0.008, −0.002]
0.8	SPJF-E – SPJF-log	+0.012 [+0.004, +0.019]	+0.006 [−0.004, +0.015]
	Guard(300) – Guard-fixed(300)	+0.099 [+0.056, +0.157]	+0.091 [+0.041, +0.150]
	Guard(300) – Guard-queue(300)	+0.002 [−0.008, +0.011]	−0.007 [−0.023, +0.008]
	Guard(300) – SPJF-E	−0.147 [−0.189, −0.122]	−0.199 [−0.262, −0.151]
	Guard(600) – Guard-fixed(600)	+0.028 [+0.020, +0.038]	+0.026 [+0.015, +0.041]
	Guard(600) – Guard-queue(600)	+0.003 [−0.001, +0.007]	+0.005 [−0.001, +0.008]
	Guard(600) – SPJF-E	−0.049 [−0.058, −0.038]	−0.060 [−0.077, −0.046]
	Guard(1200) – Guard-fixed(1200)	+0.005 [−0.001, +0.011]	+0.000 [−0.004, +0.006]
	Guard(1200) – Guard-age(1200)	−0.014 [−0.018, −0.009]	−0.021 [−0.025, −0.016]
1.0	SPJF-E – SPJF-log	+0.019 [+0.011, +0.024]	+0.009 [−0.000, +0.015]
	Guard(300) – Guard-fixed(300)	+0.262 [+0.202, +0.288]	+0.199 [+0.137, +0.234]
	Guard(300) – Guard-queue(300)	+0.036 [+0.016, +0.046]	+0.036 [+0.005, +0.045]
	Guard(300) – SPJF-E	−0.438 [−0.526, −0.314]	−0.497 [−0.565, −0.393]
	Guard(600) – Guard-fixed(600)	+0.119 [+0.074, +0.176]	+0.131 [+0.082, +0.183]
	Guard(600) – Guard-queue(600)	+0.008 [+0.004, +0.013]	+0.013 [+0.001, +0.024]
	Guard(600) – SPJF-E	−0.115 [−0.142, −0.077]	−0.158 [−0.191, −0.110]
	Guard(1200) – Guard-fixed(1200)	+0.017 [+0.002, +0.031]	+0.011 [−0.004, +0.025]
	Guard(1200) – Guard-age(1200)	−0.030 [−0.036, −0.022]	−0.041 [−0.047, −0.033]
	Guard(1200) – SPJF-E	−0.072 [−0.083, −0.052]	−0.110 [−0.124, −0.082]

Table S33. Table 8 of the manuscript under the online replay: Guard(G) against the maximum waiting time Timeout($G - (3 - 2/k)L$), which has the same bound G . Gap closed, mean over the five overlays, with 95% intervals. The difference is Guard minus Timeout, resampled as a difference. Harm is the worst over the overlays, in seconds.

Load	G (s)	Guard(G)	Timeout	Difference	Harm, Guard / Timeout
0.5	300	0.681 [0.648, 0.706]	0.632 [0.597, 0.660]	+0.049 [+0.024, +0.074]	116.3 / 137.3
	600	0.725 [0.696, 0.746]	0.719 [0.691, 0.740]	+0.006 [+0.001, +0.013]	248.3 / 248.3
	1200	0.733 [0.703, 0.755]	0.739 [0.707, 0.762]	−0.005 [−0.009, −0.001]	127.1 / 248.3
0.8	300	0.662 [0.590, 0.709]	0.078 [−0.077, 0.431]	+0.584 [+0.257, +0.709]	127.9 / 165.8
	600	0.800 [0.772, 0.819]	0.748 [0.696, 0.775]	+0.053 [+0.030, +0.092]	215.7 / 394.6
	1200	0.815 [0.791, 0.832]	0.828 [0.804, 0.843]	−0.012 [−0.016, −0.008]	135.4 / 394.6
1.0	300	0.367 [0.278, 0.485]	0.023 [0.008, 0.040]	+0.344 [+0.265, +0.450]	129.4 / 191.4
	600	0.705 [0.648, 0.768]	0.282 [0.016, 0.548]	+0.423 [+0.204, +0.664]	333.2 / 454.7
	1200	0.754 [0.711, 0.799]	0.740 [0.683, 0.800]	+0.014 [−0.008, +0.044]	128.6 / 735.1

References

- Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset, version 1.81. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19.
- Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021.
- Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/l_intel_netbatch/, 2012.
- Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19.
- Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19.
- Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017, pp. 3146–3154.
- Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the 15th European Semantic Web Conference (ESWC 2018), 2018, Vol. 10843, *Lecture Notes in Computer Science*, pp. 593–607. Preprint arXiv:1703.06103, https://doi.org/10.1007/978-3-319-93417-4_38.
- Chen, K.; Huang, F.; Liu, Z.; Yu, H.; Meng, L.; Mo, S.; Zhang, L.; Song, Y. ACcoding: A graph-based dataset for online judge programming. *Scientific Data* **2024**, *11*. See also Author Correction, doi:10.1038/s41597-024-03693-3, <https://doi.org/10.1038/s41597-024-03392-z>.
- Shahrad, M.; Fonseca, R.; Goiri, Í.; Chaudhry, G.; Batur, P.; Cooke, J.; Laureano, E.; Tresness, C.; Russinovich, M.; Bianchini, R. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20). USENIX Association, 2020, pp. 205–218.
- Verma, A.; Pedrosa, L.; Korupolu, M.; Oppenheimer, D.; Tune, E.; Wilkes, J. Large-scale cluster management at Google with Borg. In Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), 2015, pp. 1–17. <https://doi.org/10.1145/2741948.2741964>.
- Harchol-Balter, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*; Cambridge University Press, 2013. <https://doi.org/10.1017/CBO9781139226424>.
- Grosz, I.; Yang, K.; Scully, Z.; Harchol-Balter, M. Nudge: Stochastically Improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2021**, *5*, 1–29. Preprint arXiv:2106.01492, <https://doi.org/10.1145/3460088>.
- Van Houdt, B. On the Stochastic and Asymptotic Improvement of First-Come First-Served and Nudge Scheduling. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2022**, *6*, 1–22. Preprint arXiv:2206.10428, <https://doi.org/10.1145/3570610>.
- Grosz, I.; Harchol-Balter, M.; Scheller-Wolf, A. WCFS: a new framework for analyzing multiserver systems. *Queueing Systems* **2022**, *102*, 143–174. Preprint arXiv:2109.12663, <https://doi.org/10.1007/s11134-022-09848-6>.
- Cruz, R.L. A Calculus for Network Delay, Part I: Network Elements in Isolation. *IEEE Transactions on Information Theory* **1991**, *37*, 114–131. <https://doi.org/10.1109/18.61109>.
- Feitelson, D.G. Resampling with Feedback: A New Paradigm of Using Workload Data for Performance Evaluation. In *Job Scheduling Strategies for Parallel Processing (JSSPP 2021)*; Springer, 2021; Vol. 12985, *Lecture Notes in Computer Science*, pp. 3–32. https://doi.org/10.1007/978-3-030-88224-2_1.
- Shmueli, E.; Feitelson, D.G. On Simulation and Design of Parallel-Systems Schedulers: Are We Doing the Right Thing? *IEEE Transactions on Parallel and Distributed Systems* **2009**, *20*, 983–996. <https://doi.org/10.1109/TPDS.2008.152>.
- Parallel Workloads Archive. The LPC Log. https://www.cs.huji.ac.il/labs/parallel/workload/l_lpc/. Parallel Workloads Archive; data provided by Emmanuel Medernach; accessed 2026-09-23.
- Shai, O.; Shmueli, E.; Feitelson, D.G. Heuristics for Resource Matching in Intel's Compute Farm. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Vol. 8429, *Lecture Notes in Computer Science*, pp. 116–135. https://doi.org/10.1007/978-3-662-43779-7_7.
- Schwiegelshohn, U.; Yahyapour, R. Fairness in Parallel Job Scheduling. *Journal of Scheduling* **2000**, *3*, 297–320. [https://doi.org/10.1002/1099-1425\(200009/10\)3:5<297::AID-JOS50>3.0.CO;2-D](https://doi.org/10.1002/1099-1425(200009/10)3:5<297::AID-JOS50>3.0.CO;2-D).
- Lindsay, A.M.; Galloway-Carson, M.; Johnson, C.R.; Bunde, D.P.; Leung, V.J. Backfilling with guarantees made as jobs arrive. *Concurrency and Computation: Practice and Experience* **2013**, *25*, 513–523. <https://doi.org/10.1002/cpe.2860>.
- Yuan, Y.; Yang, G.; Wu, Y.; Zheng, W. PV-EASY: A Strict Fairness Guaranteed and Prediction Enabled Scheduler in Parallel Job Scheduling. In Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10), 2010, pp. 240–251. <https://doi.org/10.1145/1851476.1851505>.
- Yuan, Y.; Wu, Y.; Zheng, W.; Li, K. Guarantee Strict Fairness and Utilize Prediction Better in Parallel Job Scheduling. *IEEE Transactions on Parallel and Distributed Systems* **2014**, *25*, 971–981. <https://doi.org/10.1109/TPDS.2013.88>.

24. Grosz, I.; Scully, Z.; Harchol-Balter, M. Load Balancing Guardrails: Keeping Your Heavy Traffic on the Road to Low Response Times. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2019**, *3*, 1–31. Preprint arXiv:1905.03439, <https://doi.org/10.1145/3341617.3326157>. 2347–2349
25. Harchol-Balter, M. Task assignment with unknown duration. *Journal of the ACM* **2002**, *49*, 260–288. Conference version: Proceedings of the 20th International Conference on Distributed Computing Systems (ICDCS 2000), pp. 214–224, doi:10.1109/ICDCS.2000.840932, <https://doi.org/10.1145/506147.506154>. 2350–2352
26. Mu'alem, A.W.; Feitelson, D.G. Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling. *IEEE Transactions on Parallel and Distributed Systems* **2001**, *12*, 529–543. <https://doi.org/10.1109/71.932708>. 2353–2354
27. Smith, W.E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly* **1956**, *3*, 59–66. <https://doi.org/10.1002/nav.3800030106>. 2355–2356
28. Sabin, G.; Kochhar, G.; Sadayappan, P. Job Fairness in Non-Preemptive Job Scheduling. In Proceedings of the 2004 International Conference on Parallel Processing (ICPP 2004). IEEE Computer Society, 2004, pp. 186–194. <https://doi.org/10.1109/ICPP.2004.1327920>. 2357–2359
29. Sabin, G.; Sadayappan, P. Unfairness Metrics for Space-Sharing Parallel Job Schedulers. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2005), 2005, Vol. 3834, *Lecture Notes in Computer Science*, pp. 238–256. https://doi.org/10.1007/11605300_12. 2360–2362
30. Rajbhandary, A.; Bunde, D.P.; Leung, V.J. Variations of Conservative Backfilling to Improve Fairness. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Vol. 8429, *Lecture Notes in Computer Science*, pp. 177–191. https://doi.org/10.1007/978-3-662-43779-7_10. 2363–2365
31. Charlet, N.; Van Houdt, B. Tail Optimality and Performance Analysis of the Nudge*(M) Scheduling Algorithm. *Operations Research* **2026**, *74*, 2161–2178. <https://doi.org/10.1287/opre.2024.1061>. 2366–2367
32. Yu, G.; Scully, Z. Strongly Tail-Optimal Scheduling in the Light-Tailed M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2024**, *8*, 1–33. Preprint arXiv:2404.08826, <https://doi.org/10.1145/3656011>. 2368–2369
33. Yu, G.; Harlev, A.; Adakroy, R.; Scully, Z. A Tale of Two Traffics: Optimizing Tail Latency in the Light-Tailed M/G/k. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2025**, *9*, 1–40. <https://doi.org/10.1145/3771561>. 2370–2371
34. Bennett, J.C.R.; Zhang, H. WF²Q: Worst-case Fair Weighted Fair Queueing. In Proceedings of the IEEE INFOCOM '96, 1996, Vol. 1, pp. 120–128. <https://doi.org/10.1109/INFCOM.1996.497885>. 2372–2373
35. Bennett, J.C.R.; Benson, K.; Charny, A.; Courtney, W.F.; Le Boudec, J.Y. Delay Jitter Bounds and Packet Scale Rate Guarantee for Expedited Forwarding. *IEEE/ACM Transactions on Networking* **2002**, *10*, 529–540. <https://doi.org/10.1109/TNET.2002.801404>. 2374–2375
36. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the single-node case. *IEEE/ACM Transactions on Networking* **1993**, *1*, 344–357. <https://doi.org/10.1109/90.234856>. 2376–2377
37. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the multiple node case. *IEEE/ACM Transactions on Networking* **1994**, *2*, 137–150. <https://doi.org/10.1109/90.298432>. 2378–2379
38. Stoica, I.; Abdel-Wahab, H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Technical Report TR-95-22, Department of Computer Science, Old Dominion University, Norfolk, Virginia, 1995. Revised 26 January 1996. 2380–2382
39. Graham, R.L. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal on Applied Mathematics* **1969**, *17*, 416–429. <https://doi.org/10.1137/0117039>. 2383–2384
40. Grosz, I.; Scully, Z.; Harchol-Balter, M. SRPT for multiserver systems. *Performance Evaluation* **2018**, *127–128*, 154–175. Preprint arXiv:1805.07686, <https://doi.org/10.1016/j.peva.2018.10.001>. 2385–2386
41. Kleinrock, L. A conservation law for a wide class of queueing disciplines. *Naval Research Logistics Quarterly* **1965**, *12*, 181–192. <https://doi.org/10.1002/nav.3800120206>. 2387–2388
42. Sharif, A.B.; Stanford, D.A.; Taylor, P.; Ziedins, I. A multi-class multi-server accumulating priority queue with application to health care. *Operations Research for Health Care* **2014**, *3*, 73–79. <https://doi.org/10.1016/j.orhc.2014.01.002>. 2389–2390
43. Li, N.; Stanford, D.A. Multi-server accumulating priority queues with heterogeneous servers. *European Journal of Operational Research* **2016**, *252*, 866–878. <https://doi.org/10.1016/j.ejor.2016.02.010>. 2391–2392
44. Jeffay, K.; Stanat, D.F.; Martel, C.U. On Non-Preemptive Scheduling of Periodic and Sporadic Tasks. In Proceedings of the 12th IEEE Real-Time Systems Symposium (RTSS 1991), 1991, pp. 129–139. <https://doi.org/10.1109/REAL.1991.160366>. 2393–2394
45. Green, T.C.; Stidham, Jr., S. Sample-path conservation laws, with applications to scheduling queues and fluid systems. *Queueing Systems* **2000**, *36*, 175–199. <https://doi.org/10.1023/A:1019183220080>. 2395–2396
46. El-Taha, M. A general workload conservation law with applications to queueing systems. *Queueing Systems* **2017**, *85*, 361–381. <https://doi.org/10.1007/s11134-017-9515-4>. 2397–2398
47. Bartsch, B.; Bolch, G. A conservation law for G/G/m queueing systems. *Acta Informatica* **1978**, *10*, 105–109. <https://doi.org/10.1007/BF00260928>. 2399–2400

48. Federgruen, A.; Groenevelt, H. M/G/c Queueing Systems with Multiple Customer Classes: Characterization and Control of Achievable Performance Under Nonpreemptive Priority Rules. *Management Science* **1988**, *34*, 1121–1138. <https://doi.org/10.1287/mnsc.34.9.1121>. 2401
49. Kleinrock, L. A delay dependent queue discipline. *Naval Research Logistics Quarterly* **1964**, *11*, 329–341. <https://doi.org/10.1002/nav.3800110306>. 2402
50. Stanford, D.A.; Taylor, P.; Ziedins, I. Waiting time distributions in the accumulating priority queue. *Queueing Systems* **2014**, *77*, 297–330. Online first 2013-12-07, <https://doi.org/10.1007/s11134-013-9382-6>. 2403
51. Bansal, N.; Harchol-Balter, M. Analysis of SRPT scheduling. In Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2001, pp. 279–290. <https://doi.org/10.1145/378420.378792>. 2404
52. Wierman, A.; Harchol-Balter, M. Classifying scheduling policies with respect to unfairness in an M/GI/1. In Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2003, pp. 238–249. <https://doi.org/10.1145/781027.781057>. 2405
53. Whitt, W. The amount of overtaking in a network of queues. *Networks* **1984**, *14*, 411–426. <https://doi.org/10.1002/net.3230140305>. 2406
54. Kim, W.S.; Lim, D.E. Analysis of overtaking in M/M/c queues. *Computers & Industrial Engineering* **2016**, *101*, 177–183. <https://doi.org/10.1016/j.cie.2016.09.005>. 2407
55. Kiefer, J.; Wolfowitz, J. On the theory of queues with many servers. *Transactions of the American Mathematical Society* **1955**, *78*, 1–18. <https://doi.org/10.1090/S0002-9947-1955-0066587-3>. 2408
56. Daley, D.J. Certain optimality properties of the first-come first-served discipline for G/G/s queues. *Stochastic Processes and their Applications* **1987**, *25*, 301–308. [https://doi.org/10.1016/0304-4149\(87\)90208-0](https://doi.org/10.1016/0304-4149(87)90208-0). 2409
57. Zhang, E.; Wu, F.; Lu, X. An Intelligent Scheduling System for Large-Scale Online Judging. In Proceedings of the Computer Science and Education. Computer Science and Technology (ICCSE 2023). Springer Nature Singapore, 2024, Vol. 2023, *Communications in Computer and Information Science*, pp. 265–279. https://doi.org/10.1007/978-981-97-0730-0_24. 2410
58. Li, Y.; Chen, Y.; Chen, J.; Choukse, E.; Qiu, H.; Suh, G.E.; Fonseca, R.; Scully, Z.; Gupta, U. Beyond Prediction: Tail-Aware Scheduling for LLM Inference. In Proceedings of the 43rd International Conference on Machine Learning (ICML 2026), Seoul, South Korea, 2026; Vol. 306, *Proceedings of Machine Learning Research*. Preprint arXiv:2606.18431. 2411
59. Fu, Y.; Liu, L.; Wang, H.; Cheng, Y.; Chen, S. SFS: Smart OS Scheduling for Serverless Functions. In Proceedings of the SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2022, pp. 1–16. Preprint arXiv:2209.01709, <https://doi.org/10.1109/SC41404.2022.00047>. 2412
60. Fu, Y.; Shi, R.; Wang, H.; Chen, S.; Cheng, Y. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC 24). USENIX Association, 2024, pp. 19–36. 2413
61. Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J.E.; Yuan, Y.; Zhang, Z.; et al. RelBench: A Benchmark for Deep Learning on Relational Databases. In Proceedings of the Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track (NeurIPS 2024), 2024. Preprint arXiv:2407.20060. 2414
62. Gu, J.; Ranjan, R.; Kanatsoulis, C.; Tang, H.; Jurkovic, M.; Hudovernik, V.; Znidar, M.; Chaturvedi, P.; Shroff, P.; Li, F.; et al. RelBench v2: A Large-Scale Benchmark and Repository for Relational Data, 2026, [2602.12606]. 2415